

Ответы на вопросы

1. Архитектура, микроархитектура: отличия, примеры

Архитектура (Instruction Set Architecture, ISA) — Набор инструкций, регистров, режимов адресации, организации виртуальной памяти, обработки исключений и прерываний. То есть все то, что представляет процессор со стороны программиста.

Микроархитектура: способ, которым данная архитектура набора команд реализована в процессоре.

ISA — это интерфейс между CPU и программами, которые на этом CPU исполняются. ISA определяет количество регистров, которые видны программисту, доступные инструкции, правила выравнивания и так далее. В целом ISA - абстракция для программиста.

Микроархитектура определяет то, как CPU устроен внутри. Из каких функциональных узлов он состоит, какая иерархия кэшей, как декодируются инструкции, сколько стадий в конвейере и так далее.

1 Микроархитектура, не влияет на семантику и результат исполнения программ, но влияет на скорость их исполнения.

2 Микроархитектура влияет на энергопотребление процессора. Например, Intel запустила линейку процессоров Atom, которая была нацелена на мобильные устройства. Эти CPU реализовали ту же архитектуру x86, но их внутреннее устройство позволяло экономить потребление энергии, что делало возможным их использование в переносимых устройствах.

Микроархитектура программисту не видна†, программист работает на уровне ISA.

Если два разных процессора реализуют одну и ту же ISA, то они могут исполнять одни и те же программы. Например, программы можно запускать как на Intel, так и на AMD.

2. Производительность процессора: понятие “производительность”, способы измерения, единицы измерения

Производительность — это характеристика, которая отражает способность процессора выполнять вычисления за определенное время. Она определяется скоростью и эффективностью выполнения инструкций, обработки данных и взаимодействия с другими компонентами системы. Типичные показатели производительности — время отклика и пропускная способность (объем данных, обрабатываемых за единицу времени.).

Способы измерения:

1) Теоретические методы (оценка на основе технических характеристик):

- тактовая частота;
- количество операций на такт (CPI – Cycles Per Instruction);

- количество ядер и их взаимодействие.

2) Бенчмарки (задачи, служащие эталонным тестом производительности):

- Синтетические (оценивают процессор в искусственных задачах):

- Dhrystone – производительность целочисленных вычислений, измеряемая в DMIPS (Dhrystone millions of instructions per second);
- NBench – для измерения производительности целочисленных вычислений, операций памяти и вычислений с плавающей точкой.
- SPECobfr – оценивает производительность вычислений с плавающей запятой, которые часто используются в научных, инженерных и других вычислительно сложных задачах, таких как моделирование, обработка изображений и симуляции.

Единицы измерения

1) Количество операций в секунду:

- FLOPS (Floating Point Operations Per Second) – операции с плавающей запятой.

- MIPS (Million Instructions Per Second) – миллионы инструкций в секунду.

2) Время выполнения задач. Время выполнения реальных задач, таких как рендеринг, компиляция или математические расчеты.

3) Балл производительности. Результаты тестов в бенчмарках.

Например:

- Dhrystone: Процессор с результатом 5000 DMIPS способен выполнять 5 миллиардов целочисленных операций в секунду.

- NBench: В тесте Integer Index процессор набрал 150 баллов, что указывает на его высокую скорость обработки целочисленных операций.

- SPECobfr: Для задач с плавающей запятой процессор может достичь 40 баллов в Base режиме, что делает его подходящим для научных вычислений.

3. Классификация архитектур (CISC, RISC, VLIW, EPIC)

Способы классификации архитектур:

- Control Flow / Data Flow архитектуры;
- по способу хранения операндов:
 - стековая;
 - аккумуляторная (IBM 7090, DEC PDP-8, Агат-8);
 - регистровая;
- по доступу к операндам
- по типу архитектуры: CISC, RISC, VLIW, MIPS, PowerPC, SPARK, x86, x86-64, ARM

CISC архитектура (Complex Instruction Set Computer)

- Малое количество регистров.
- Большое количество режимов адресации
- Сложные инструкции
- Переменная длина инструкции

RISC архитектура (Reduced instruction set computer)

- архитектура загрузки сохранения (load-store)
- большое количество универсальных регистров (register-register)
- простые инструкции
- ограниченное число режимов адресации
- инструкции фиксированной длины

Причины перехода от CISC к RISC:

- конвейеризация процессора
- увеличение объема памяти
- совершенствование компилятора

VLIW (Very Long Instruction Word)

Архитектура, которая специально разработана для повышения количества одновременно исполняемых инструкций. При одинаковом количестве одновременно исполняемых инструкций VLIW архитектура позволяет создать значительно более простой процессор, чем RISC архитектура.

Ключевая идея:

1. множество НЕЗАВИСИМЫХ инструкций группируются в ОДИН пакет (VLIW инструкцию)
2. на одном такте выбирается ОДИН пакет
3. каждая инструкция в пакете соответствует строго определенному функциональному устройству (ФУ)
4. между всеми ФУ делится общий регистровый файл
5. типовая длина пакета – 64-1024 бит (в зависимости от числа ФУ)
6. проверка зависимостей и планирование исполнения возлагается на компилятор

Плюсы:

- аппаратная сложности уменьшена
- отсутствуют блок проверки зависимостей
- существенно упрощены блоки планирования исполнения и декодирования инструкций
- как следствие, сниженное энергопотребление и повышение тактовых частот

Минусы:

- высокие требования к компилятору
- увеличенный размер программного кода
- более высокие требования к пропускной способности памяти и регистрового файла
- незапланированные события (например, кэш-промах) приводят к останову конвейера
- нерациональное использование пропускной способности памяти и функциональных устройств (например, операция por)

Статический VLIW процессор

- особенность: за запуск инструкций отвечает компилятор
- компилятор должен правильно расставить пор-ы
- если инструкция пойдет на исполнение до того момента, как предыдущая инструкция закончится произойдет сбой
- компилятор всегда выбирает максимальную латентность для устройств сослучайной латентностью

недостатки:

- неэффективность кэша
- очень большой размер программы из-за большого количества пор инструкций

Динамический VLIW процессор

- особенность: за запуск инструкций отвечает процессор
- процессор ожидает готовности данных для всех инструкций в пакете и только после этого запускает пакет
- позволяет эффективно использовать кэш
- уменьшает размер программы

Общие проблемы VLIW архитектур

- большой размер скомпилированной программы
- отсутствие бинарной совместимости
- изменение количественного и качественного состава ФУ процессора приводит к бинарной несовместимости с предыдущей версией
- код нужно перекомпилировать для каждого процессора



EPIC (explicit parallel instruction set) – призвана сохранить достоинства VLIW архитектуры и устранить ее недостатки

общие черты VLIW и EPIC:

- возможность параллельного исполнения инструкций задается на уровне архитектуры, а не определяется динамически в процессе исполнения

преимущества EPIC:

- решена проблема бинарной совместимости между различными процессорами

Спекуляция по управлению в EPIC основана на:

- наличии двух типов инструкций: спекулятивных и не спекулятивных
- параллельном исполнении спекулятивных инструкций из двух ветвей
- использовании предикатирования для выбора результат корректной ветви
- использовании третьего состояния регистров для сохранения поведения исключений во время спекуляции

4. Классификация архитектур (x86, x86-64, Power, ARM, IA64, RISC-V, MIPS, Alpha, ...)

Здесь важнее всего понимать, что все эти архитектуры отличаются только структурой инструкций, внутренней схемой соединения элементов, какими то другими конструктивными особенностями и системой команд. Умирали они потому, что у компании заканчивались деньги/не успевали за конкурентами.

- MIPS (умерло)
- SPARC (умирает)
- Power (все сложно)
- 80x86 (все сложно)
- ARM
- RISC-V

MIPS

Область применения - встраиваемые решения
1981-2021 (после переход на RISC-V)

Совершенствование конвейера:

Согласовать каждую подфазу каждой команды (в т.ч. кэширование) в один цикл.

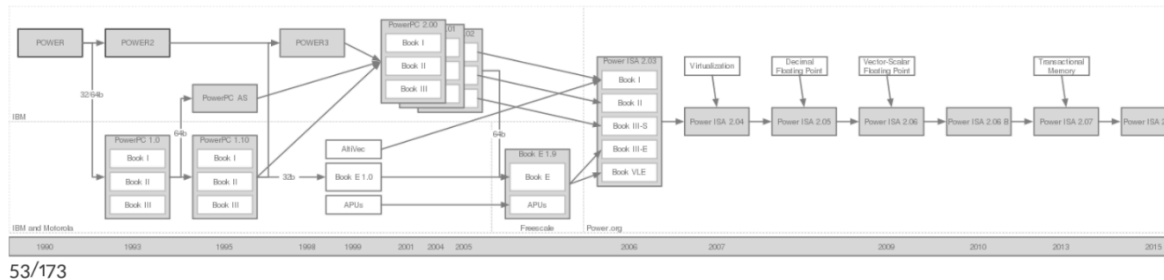
- избегание необходимости в блокировках между стадиями
- пропуск на конвейере только один цикл
- цена: исключение «тяжелых» операций
- исключение регистрового окна

- Малое количество простых инструкций
- Всего 3 формата инструкций (3 способа расположения данных внутри инструкции)
- Широкое распространение в академических кругах

Применялось в сетевых устройствах Cisco, Mikrotik, в старых консолях PlayStation, встраиваемые системы

Power PC

- архитектура POWER
 - Performance Optimization With Enhanced RISC
- область применения
 - высокопроизводительные серверные, настольные, встраиваемые решения
- 1991 г.: альянс AIM: Apple, IBM, Motorola
- 2013 г.: [OpenPOWER Foundation](#)



Юзали древние макбуки, вроде как иногда применяют военной промышленности, инфы очень мало.

80x86/x64

Юзают CISC, отличаются в объёме доступной ОЗУ (4 гб для x86, 2^64 для x86-64), x64 обратно совместима с x86.

В x64 регистры 64 битные, думаю это понятно.

Различные режимы адресации (реальный, защищённый, смешанный).

Расширения SSE1..5, AVX, AVX2, AVX512 и другие подобные.

Производители Intel, AMD, а больше ничего и нет((

- Область применения:
 - высокопроизводительные настольные решения, сервера среднего и начального уровня, суперкомпьютеры
- Основные производители процессоров:
 - Intel, AMD, много мелких компаний из Китая
- В 1978 выпущен 16 битный процессор

ARM (Advanced RISC Machine)

Низкое энергопотребление, поэтому юзаются во встраиваемых решениях. Единое адресное пространство для IO и ОЗУ

Надо покупать лицензию, чтобы использовать, а так по базе рассказываем про особенности RISC.

- Область применения:
 - встраиваемые решения (смартфоны, планшеты)
- Разработчик:
 - ARM Limited
- Разработка: с 1985 г. (ARM1)
- Семейства:
 - ARM7, ARM9, ARM11, Cortex, Neoverse

Alpha (умер юзал RISC)

- Область применения:
 - высокопроизводительные сервера и суперкомпьютеры
- Основные производители процессоров:
 - DEC -> Compaq (2001) -> HP (до 2007 г.)
- Все процессоры 64-х битные
- С 1992 по 2001 год процессоры Alpha являлись самыми быстрыми в мире
- В 1992 выпущен первый процессор Alpha 21064

PA-RISC (Precision Architecture, умерб юзал RISC)

- область применения:
 - высокопроизводительные сервера и суперкомпьютеры
 - наиболее сложной и развитой системой команд среди всех RISC архитектур
- основные производители процессоров:
 - HP
- в 1989 выпущен первый 32-х битный процессор PA-7000
- в 1996 выпущен 64-х битный процессор PA-8000
- системы сняты с продаж в 2003 году, после перехода HP на процессоры Intel Itanium 2

IA-64 (умер, юзал RISC)

- область применения:
 - высокопроизводительные сервера и суперкомпьютеры
- разработка:
 - Intel + HP (с 1994 г.)
- архитектура относится к новому поколению EPIC
- ядра:
 - Intel Itanium (2001 г.)
 - Intel Itanium 2 (2002-2009г.)
 - Itanium 9700 (2017 г.)

RISC-V:

- **Открытая архитектура:** RISC-V — это открытая архитектура (open-source), в отличие от большинства других RISC-архитектур, таких как ARM и MIPS, которые имеют лицензируемые патенты. Это

позволяет производителям создавать свои собственные процессоры, используя RISC-V без платы за лицензии.

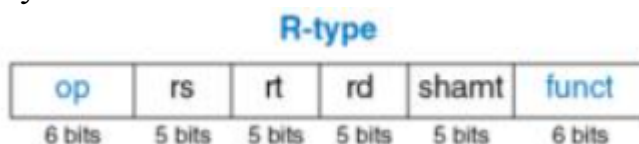
- **Модульность:** RISC-V отличается модульной структурой, что означает, что его можно настроить под конкретные потребности. Например, можно использовать базовую инструкцию (RV32I или RV64I для 32-битных или 64-битных систем) и добавлять дополнительные расширения для нужд конкретного устройства (например, для работы с плавающей запятой или для криптографии).
- **Развитие и поддержка:** RISC-V активно развивается и поддерживается сообществом, в том числе такими крупными организациями, как Google, Western Digital и другие. Это означает, что архитектура постоянно улучшает свою совместимость с новыми технологиями.

5. Кодирование инструкций на примере MIPS и x86

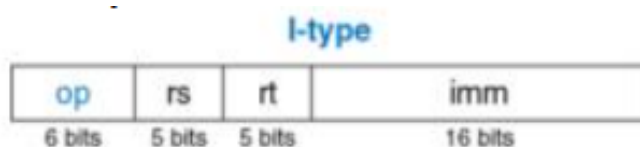
Инструкции бывают:

- постоянной длины
 - простота декодирования
 - RISC: MIPS (4 байта), PowerPC, SPARC, ARM...
- переменной длины
 - занимают меньше памяти
 - CISC: IBM 360, x86(от 1 до 17 байт), Motorola 68k, VAX...
- комбинированный
 - MIPS16, THUMB (2 формата: 2 и 4 байта)

Особенность инструкций MIPS это фиксированная длина(4 байта). В архитектуре MIPS в качестве компромисса используются три формата инструкций: типа R, типа I типа J. Инструкции типа R используют три регистровых операнда. Инструкции типа I используют два регистровых операнда и 16-битную константу. Инструкции типа J (jump) используют 26-битную константой.



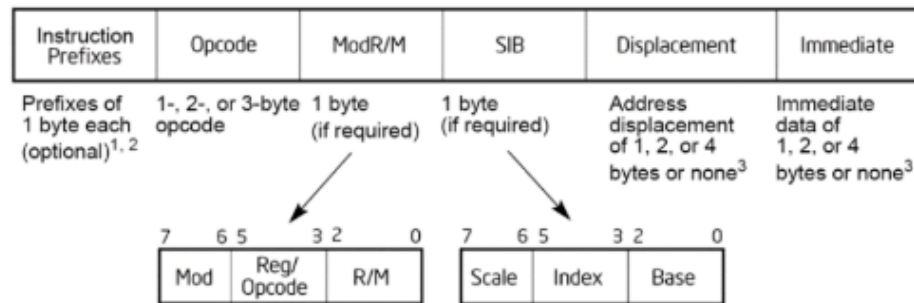
Операция, выполняемая командой, закодирована двумя полями, отмеченными синим цветом: полем op (также называемым opcode или кодом операции) и полем funct (также называемым функцией). У всех команд типа R поле opcode равно нулю. Операнды закодированы тремя полями: rs, rt и rd. Поля содержат номера регистров. Регистры rs и rt являются регистрами-источниками, а rd – регистром-назначением. Поле, shamt, используется только для операций сдвига.



Инструкции типа I используют в качестве операндов два регистра и один непосредственный операнд (константу)



Этот формат используется только для инструкций безусловного перехода и ветвления. 6-бит поле кода операции (opcode). Оставшиеся биты используются для указания адреса перехода (addr).



1. The REX prefix is optional, but if used must be immediately before the opcode; see Section 2.2.1, "REX Prefixes" for additional information.
 2. For VEX encoding information, see Section 2.3, "Intel® Advanced Vector Extensions (Intel® AVX)".
 3. Some rare instructions can take an 8B immediate or 8B displacement.

Главная особенность инструкций x86 – не фиксированная длина инструкций что усложняет архитектуру процессора.

Общий вид инструкции x86:

- Префикс инструкции
- ОPCODE (от 1 до 3 байтов)
- ModR/M - определяет саму операцию
- SIB - байт для операций расширенных (32/64 битных)
- Отклонение - используется для вычисления эффективного адреса инструкции (может быть 1 2 4 байта, его размер определяет поле ModR/M)
- Поле непосредственного операнда имеется в командах, использующих непосредственную адресацию; его наличие определяется по коду операции. Размер этого поля обычно равен размеру операндов данной команды; исключением является 64-разрядный режим

6. Типы инструкций. Примеры

Передача данных

- LD и ST (загрузка и сохранение данных из/в память)
- MFC1 и MTC1 (чтение и загрузка в регистры сопроцессора 1)
- MFC0 и MTC0 (чтение и загрузка в регистры сопроцессора 0)

Арифметико-логические

- ADD и SUB
- AND, OR и XOR
- MUL и DIV
- SLT (Set on Less Than) и LUI (Load Upper Immediate)
- двухадресные (Operand1 = Operand1 OPERATION Operand2) и
 трехадресные (Operand3 = Operand1 OPERATION Operand2)

Управление потоком исполнения

- BEQZ (Branch if Equal to Zero) – условный переход
- JR (Jump Register) и JAL (Jump and Link) – безусловный переход
- TRAP (генерация програм. исключения) и ERET (Exception Return)
- вызов процедур
- возврат из процедуры
- организация циклов

Для работы с плавающей запятой

- ADD.D (Add Double-Precision) и SUB.S (Subtract Single-Precision)
- MUL.D
- C.LT.D (Compare Less Than, Double-Precision) и CVT.S.W (Convert Word to Single-Precision)

Мультимедийные (SIMD)

- ADD.PS и SUB.PS, MUL.PS, C.LT.PS
- .PS (Paired Single) работают с парами чисел с плавающей точкой одинарной точности в одном регистре

Строковые

- REP (Repeat) MOVSB – копирует последовательность байтов из одной области памяти в другую
- REP MOVSW – копирует последовательность 16-битных слов
- REP MOVSD – копирует последовательность 32-битных слов
- обработка последовательностей данных (строк) в памяти без необходимости написания сложных циклов

Системные

- вызовы ОС
- для работы с виртуальной памятью

Для работы с графикой

- операции для работы с пикселями и вертексами
- операции сжатия/восстановления

7. Арифметические инструкции. Инструкции вещественного со-процессора

1. Типы арифметических операций: сложение, вычитание, умножение, деление.

2. Тип арифметики: циклическая или с насыщением (обработка переполнений путем их циклического сброса (wrap-around) или насыщения (saturation)), знаковая или без знаковая (учитывается или нет знак числа).

- количество операндов

— двухадресные

◦ Operand1 = Operand1 OPERATION Operand2

— трехадресные

◦ Operand3 = Operand1 OPERATION Operand2

- разрядность операндов

— для современных 64-ех битных архитектур составляет обычно 32 и 64 бита

— исключение x86/x86-64 (может использовать меньшую разрядность. Например, 8 или 16)

- адресация операндов

- генерация исключений и модификация флагов состояния процессора (Арифметические инструкции могут изменять флаги процессора (например, Zero, Carry, Overflow))

Инструкции умножения и деления. Проблема операндов.

- Как сохранить удвоенный результат умножения.

1. Результат сохраняется в регистр удвоенной размерности. (например, 128 бит для 64-битных операндов).

2. Старшая часть отбрасывается.

3. Результат сохраняется в два регистра заданных неявно (универсальные или специальные)

4. Для вычисления старшей части операции используется отдельная инструкция.

- Деление.

Для деления процессор может возвращать два результата:

1. Только частное: Часто в быстродействующих системах.

2. Частное и остаток: Сохранение в два регистра, задаваемых неявно.

Инструкции для программной реализации

- 1) Сдвиг влево и сложение. Инструкция использовалась для умножения на константу.

Пример (умножение на 0xB):

r1 = A

0xB = 1011

shladd r2, r1, r1, 3

shladd r2, r2, r1, 1

- 2) Инструкции для программной реализации деления.

- DS – divide step. Инструкция вычисляет один бит частного и остаток. Бит частного помещается в carry flag, а остаток в регистр результата.

Инструкции вещественного сопроцессора.

– Инструкции загрузки и сохранения.

- Преобразование входных данных ко внутреннему типу.

– Арифметические инструкции.

- Mull and Add

- Обратное значение.

- Сложные мат. функции (синус, косинус...).

– Инструкции преобразования типов.

– Инструкции сравнения.

Инструкции сравнения

– Результат сравнения помещается во флаги сопроцессора.

Проблемы выполнения переходов на основе флагов сопроцессора

- Копирование флагов сопроцессора в универсальные регистры с их последующим анализом.

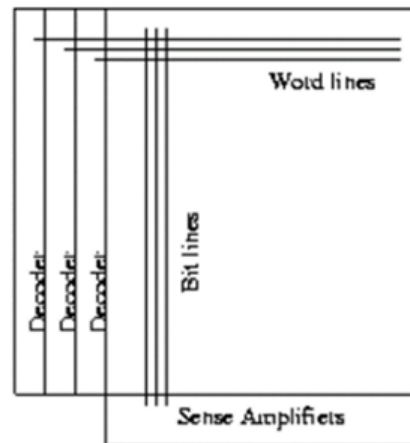
- Инструкции условного перехода, которые вычисляют условие на основе флагов сопроцессора.

- Использование инструкций нуллификаций, которые вычисляют условие на основе флагов сопроцессора.

8. Регистровый файл. Разновидности регистрового файла

Регистровый файл – модуль микропроцессора (CPU), содержащий в себе реализацию регистров процессора. Основные функции: хранение данных, быстрый доступ к ним и управление процессами чтения и записи. Использование регистровых файлов позволяет выполнять внеочередное исполнение машинных инструкций (не по порядку следования в коде, а по их готовности) и переименование регистров для реализации параллелизма на уровне команд.

Реализован как массив запоминающих ячеек, считываемых вертикально.



Слова (данные) расположены в горизонтальных строках (word lines), и при чтении ячейки выдают своё значение на вертикальные битовые линии (Bit lines). В нижней части эти линии подключены к усилителям, которые преобразуют сигналы с ячеек в булевы сигналы полной амплитуды. В левой части расположены декодеры, активирующие строку, соответствующую заказанному регистру.

Классический регистровый файл

- Количество регистров: 8 – 32
- Разрядность регистров: 32 или 64 бита

+: быстрый

-: программное сохр. требуемых регистров при вызове процедур -> увеличение задержки при переключении

Стековый регистровый файл

- Используется в архитектуре x87.
- Позволяет неявно обращаться к операндам на вершине стека.
- Используется в архитектуре вещественных сопроцессов SPARC и PA-RISC

Оконный регистровый файл

- Используется в архитектуре SPARC.
- Большое количество регистров: до 640
- Доступ к регистрам осуществляется через окно фиксированного размера: 32 регистра.
- Окно разбито на 4 части: Глобальные регистры (неизменны при переключении окон), Локальные регистры (для локальных данных), Собственные входные/выходные параметры, Входные/выходные параметры вызываемой функции.
- Регистр CWP (Current Window Pointer) – указывает на активное в данный момент окно
- Альтернативные глобальные регистры – гибкость при переключении окон

Для организации одного локального окна требуется 16 регистров. При переполнении регистрового файла генерируется исключение, по которому ОС может очистить несколько регистровых окон.

+ : Передача параметров между процедурами происходит через регистры. Можно избежать необходимости сохранять регистры в память

- : Низкая скорость регистрового файла; генерация исключения при переполнении регистрового файла; недостаточная гибкость (фиксированный размер окна, фиксированное количество локальных регистров, и регистров, которые используются для передачи параметров)

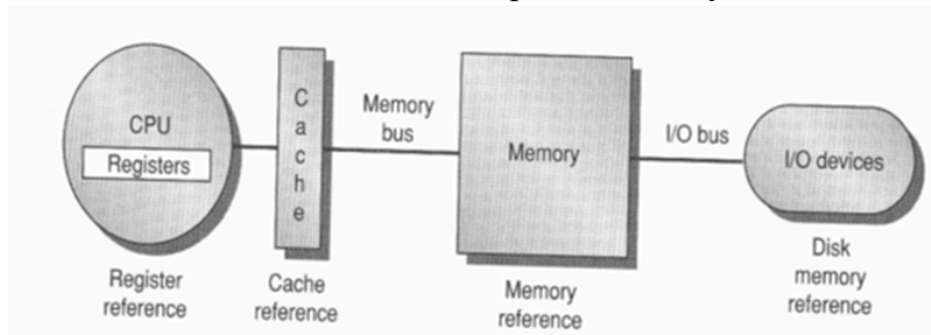
Динамический оконный регистровый файл (регистровый стек)

- Используется в архитектуре IA-64
 - Регистровое окно произвольного размера
 - Количество локальных регистров и регистров входных/выходных параметров произвольно
 - Использование механизма Register Stack Engine (RSE) для автоматического сохранения/загрузки регистров в случае переполнения регистрового файла
 - Количество глобальных регистров: 32, стековых: 96
- + : ПО не нужно заботиться о сохранении/загрузке регистров при вызове процедур; эффективное использование регистров из-за изменяемого размера окна
- : Высокая сложность аппаратной реализации RSE

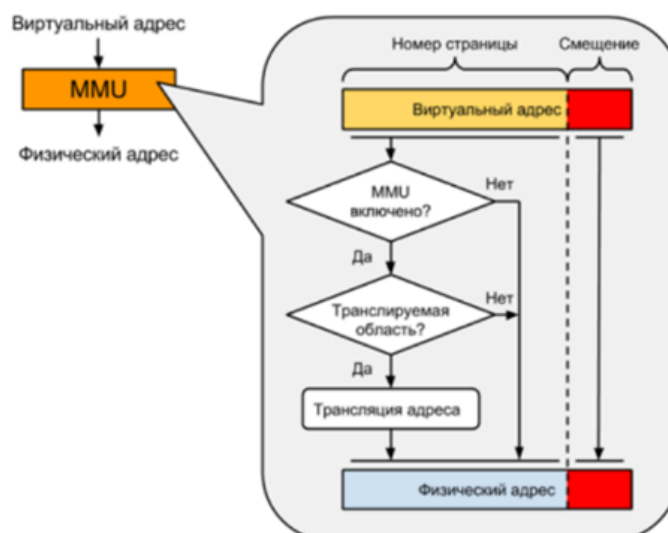
9. Организация памяти. Устройство управления памятью (MMU)

Память организована в виде иерархии. Типичная иерархия памяти выглядит следующим образом (снизу вверх):

1. Регистр процессора – самый быстрый, но ограниченный объём памяти, используемый для хранения данных и инструкций, с которыми работает процессор.
2. Кэш-память – быстрый тип памяти, расположенный между процессором и основной памятью. Кэш бывает разных уровней:
 - **L1** – маленький и быстрый, обычно встроен непосредственно в процессор.
 - **L2** – немного больше и медленнее, часто находится рядом с процессором.
 - **L3** – ещё более ёмкий, но медленный по сравнению с L1 и L2, может быть разделён между несколькими процессорами.
3. Оперативная память (RAM) – основная память системы, куда загружаются данные и программы, с которыми работает компьютер в данный момент. Это более медленный, но большой тип памяти.
4. Внешняя память – хранилище данных на длительный срок. Медленнее, чем RAM, но имеет гораздо большую ёмкость.



MMU (Memory Management Unit) - специализированное аппаратное устройство, которое управляет доступом к памяти и отвечает за преобразование виртуальных адресов в физические. +: использование при необходимости; отсутствие влияния на CPU.



Основные задачи:

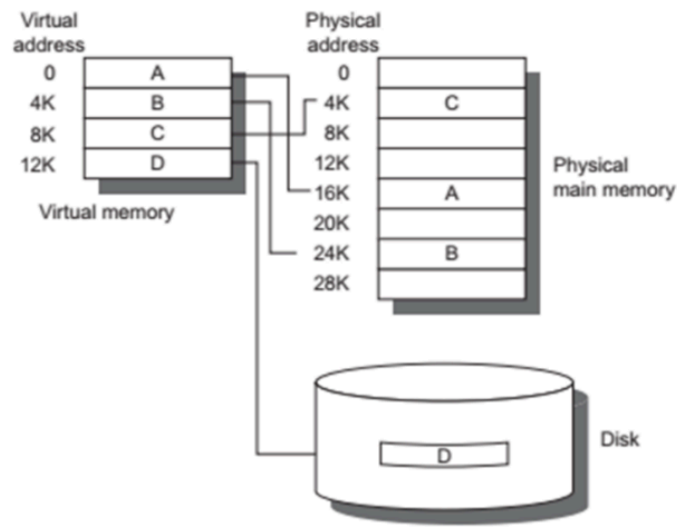
1. **Преобразование виртуальных адресов в физические** для работы с виртуальной памятью, которая позволяет процессам работать с абстракцией памяти, как если бы у каждого процесса был свое изолированное адресное пространство. Но на самом деле процессы используют одну физическую память.
2. **Разделение памяти между процессами:** MMU обеспечивает защиту и изоляцию процессов, чтобы они не имели доступа к ресурсам друг друга.
3. **Управление страницами памяти (paging) и сегментами:** для эффективного использования памяти. MMU управляет переводом виртуальных страниц в физические страницы. Это позволяет выделять память динамически и загружать в память только те части программы, которые необходимы в данный момент.
4. **Защита памяти:** проверка, имеет ли процесс право на доступ к конкретному диапазону памяти.
5. **Использование кэш-памяти для ускорения доступа**

10. Виртуальная память. TLB

Виртуальная память - концепция, которая позволяет уйти от использования физических адресов, используя вместо них виртуальные.

+: расширение реального адресного пространства; создание изолированных адресных пространств для различных процессов; задание различных свойств для разных участков памяти

- все физическое адресное пространство процесса разбито на виртуальные страницы
- вся физическая память разбита на физические страницы такого же размера
- у каждого процессора существует таблица, которая связывает виртуальные страницы с физическими страницами
- кроме оперативной памяти страницы могут находиться в файле подкачки



Оперативная память выступает в качестве кэша для «бесконечной» дисковой памяти; для вытеснения страниц из памяти используется LRU-алгоритм (удаление элементов, которые не использовались долгое время); за перенос страниц между диском и ОЗУ отвечает ОС. При отсутствии требуемой страницы в ОЗУ процессор генерирует исключение. При вытеснении страниц из памяти только модифицированные страницы сохраняются на диск.

TLB (translation look-aside buffer) - это кэш, используемый в устройстве управления памятью (MMU) для ускорения процесса преобразования виртуальных адресов в физические. Он хранит недавние преобразования виртуальных адресов (или страниц) в физические.

Когда процессор обращается к памяти, он генерирует виртуальный адрес. MMU, получив этот виртуальный адрес, должна преобразовать его в физический. Это преобразование осуществляется с использованием таблицы страниц. Для ускорения используется TLB.

Свойства: На вход TLB приходит виртуальный адрес страницы. Элемент TLB - дескриптор страницы. Процессор содержит два TLB: один для данных, второй для инструкций. Кэш TLB обычно полностью ассоциативный. Может иметь несколько уровней. При переключении процесса TLB очищается.

11. Алгоритм выбора строки жертвы, запись данных в кэш, обработка кэш промаха при записи

Когда кэш заполняется, необходимо выбрать строку (line) для замены. Стратегии выбора строки жертвы:

- Случайная. Удаляется случайная строка из множества.
- FIFO. Удаляется строка, которая дольше всех находится в кэше.
- LRU (Least-recently used). Удаляется строка, к которой реже всего обращаются.
- Замена наименее часто использовавшейся строки (LFU — Least Frequently Used). Заменяется та строка в кэш-памяти, к которой было меньше всего обращений.

Запись в кэш

1 Операции записи встречаются в 5 раз реже операций чтения.

2 Операция записи начинается, только после того, как окончательно выяснено есть нужная строка в кэше или нет.

Две политики записи в кэш:

– Сквозная запись (write through). Одновременно выполняется запись данных как в кэш, так и в основную память.

– Обратная запись (write back). Первоначально запись производится только в кэш. Все кэш линии, в которые происходила запись помечаются специальным флагом. Если такая строка выталкивается из кэша, то производится ее запись в память целиком.

Обработка кэш промаха при записи

1 При использовании сквозной записи в память, строка в кэш памяти не выделяется, а сразу производится запись данных в память.

2 При использовании обратной записи в память:

– В кэше выделяется пустая строка, с расчетом на то, что она будет полностью заполнена данными.

– Строка загружается из памяти, а затем происходит ее модификация.

12. Кэш: классификация. Пространственная и временная локальность

Разновидности кэша

- инклюзивный (следующий уровень иерархии включает все данные предыдущего уровня)

- эксклюзивный (нет общих данных между уровнями)

По политике управления

- write-through (изменения в кэше сразу записываются в основную память)

- write-back (данные записываются в основную память только при вытеснении из кэша)

По технологии организации

- с прямым отображением direct mapping (каждый блок в основной памяти отображается на конкретную ячейку в кэше)

- полностью ассоциативная fully associative mapping (каждое место в кэше может содержать данные из любой области памяти)

- set associative mapping (гибрид direct mapping и fully associative mapping; кэш делится на несколько множества (sets), и каждый блок памяти может быть размещен в любом из этих множеств, но только в одном месте)

Пространственная и временная локальность программ и данных

- пространственная локальность состоит в том, что если текущее обращение к памяти произошло по адресу «А», то следующее обращение вероятно произойдет по соседнему адресу.

- временная локальность состоит в том, что если произошло обращение к памяти по адресу «А», то вероятно, что в ближайшее время произойдет еще одно обращение по этому адресу

13. Кэш: логическая организация, ассоциативность. Поиск данных в КЭШе

Кэш – высокоскоростная статическая память, расположенная на процессорном кристалле или в процессорном корпусе.

Иерархия памяти.

Кэш организуется в несколько уровней:

L1: Самый быстрый и маленький уровень кэша, обычно разделенный на данные и инструкции.

L2: Более объемный, но медленный по сравнению с L1.

L3: Общий для всех ядер процессора, самый объемный, но медленный в иерархии кэшей.

Организация.

Весь объем кэша разбит на блоки или кэш линии. Кроме данных в кэше, для каждой кэш линии хранится ее тэг. Тэг кэш линии хранит старшие биты адреса блока памяти, который в ней содержится.

Ассоциативность.

Соответствие между блоком кэша и блоками памяти определяет понятие *ассоциативности*.

- Кэш с прямым отображением (direct mapping) устроен так, что каждая строка ячеек основной памяти может отображаться только в одной, фиксированной для неё строке кэша (под строкой понимается минимальная порция информации, которая может передаваться в кэш). Этот тип обладает минимальным временем поиска, но и минимальной вероятностью присутствия необходимой информации в нём.

- Полностью ассоциативный кэш (fully associative mapping), наоборот, позволяет каждой строке основной памяти отображаться в произвольном месте кэша. Этот кэш, наоборот, обеспечивает максимальную вероятность успеха запросов CPU на требуемую информацию, но при максимальном времени поиска.

- Множественно-ассоциативный кэш (set associative mapping) является сочетанием первых двух типов. Кэш этого типа разделён на фиксированное количество областей, именуемое степенью ассоциативности, и каждая строка RAM может отображаться в произвольном месте только одной из областей кэша. И он позволяет добиться оптимальной производительности. Важным параметром для множественно-ассоциативного кэша является правильный выбор степени ассоциативности или, что то же самое, количества каналов у кэша второго уровня.

Поиск данных в КЭШе

1) Кэш с прямым отображением.

Этапы поиска в кэше:

1. Извлекается средняя часть адреса ($\log_2 m$), определяющая номер кэш-линии в кэше.

2. Тэг кэш-линии с данным номером сравнивается со старшей частью адреса ($\log_2 N$).

Результат:

Если было совпадение по одному из тэгов, то произошло кэш-попадание.

Если не было совпадение ни по одному из тэгов, то произошел кэш-промах.

2) Полностью ассоциативный кэш.

Этапы поиска в кэше:

1. Тэги всех кэш-линий сравниваются со старшей частью адреса одновременно.

Результат, как и в первом случае.

3) Множественно-ассоциативный кэш.

Этапы поиска в кэше:

1. Извлекается средняя часть адреса ($\log_2 m$), определяющая номер сэта в кэше.

2. Тэги всех кэш-линий данного сета сравниваются со старшей частью адреса ($\log_2 N$) одновременно.

Результат, как и в первом случае.

14. Кэш: протоколы когерентности

Когерентность кэша — свойство кэшей, означающее целостность данных, хранящихся в локальных кэшах для разделяемого ресурса.

Когерентность кэшей — частный случай когерентности памяти.

Когда процессы в системе используют кэширование общих ресурсов, например, памяти, могут возникнуть проблемы с противоречивостью данных. Это особенно справедливо в отношении процессоров в многопроцессорной системе. Когерентность кэшей предназначена для управления такими конфликтами путём поддержания согласованности данных в разных кэшах.

Некогерентные кэши: значение одной ячейки памяти различно для двух кэшей.

Когерентность определяет поведение чтений и записей в одно и то же место памяти. Кэш называется когерентным, если выполняются следующие условия:

- информирование о записи: изменение данных в любом из кэшей должно быть распространено на другие копии этих данных (этой линии кэша) в соседних кэшах;
- сериализация транзакций: операции чтения и записи в одну и ту же ячейку памяти должны быть видимы для всех процессоров в одинаковом порядке.

Протокол MSI

3 состояния:

Modified: блок изменен в кэше и не соответствует в ОЗУ. Только один кэш в текущий момент может быть в данном состоянии

Shared: блок не изменен и существует как минимум в одном из кэшей в режиме только чтения

Invalid: блок не существует

Протокол MESI

4 состояния:

Exclusive: значение есть только в этом кеше, и оно пока не было изменено (Это состояние преимущество над MSI т.к не надо проверять актуальность данных в других ядрах процессорах)

Протокол MESIF

идея: попытаться получить данные от других уровней кэша

проблема: состояние S - сообщения приходят от всех

решение: F(orwarded) – разновидность состояния S – «первый среди равных». Состояние получает всегда последний запросивший данные

Протокол MOSI

Проблема MSI (данные в модифицированном состоянии):

отмена транзакции -> сброс данных в основную память -> смена состояния на Invalid -> перезапрос копии

Решение: O(wned) – состояние указывает, что данный кэш владеет эксклюзивными данными (отличие от MSI: в памяти данные не актуальны !!!)

MSI / MOSI: состояние O - оптимизация сброса данных в память (write-back)

MSI / MESI: состояние E - оптимизация числа транзакций, необходимых для чтения/записи от другого процессора (ядра)

15. Кэш: способы оптимизации ПО, prefetching

1. Общие способы оптимизации кэша в ПО

1 Пространственная локальность (Spatial Locality):

Использование данных, которые находятся рядом друг с другом в памяти. Например, считывание массивов последовательными блоками, а не случайными элементами.

2 Временная локальность (Temporal Locality): Повторное использование данных, которые недавно были доступны. Например, использование переменных или структур данных из кэша вместо повторного чтения из основной памяти.

3 Избегать случайного доступа к памяти: Не использовать структуры данных с низкой предсказуемостью доступа, например, хэш-таблицы, если это возможно.

4 Минимизирование кэш-промахов: Использование данных, которые уже находятся в кэше.

2. Prefetching: предзагрузка данных в кэш

Prefetching — это механизм, который загружает данные в кэш заранее, до их фактического использования. Это снижает задержки доступа к памяти.

1 Аппаратный prefetching: Выполняется на уровне процессора. Процессор анализирует паттерны доступа к памяти и автоматически предзагружает данные в кэш. Эффективен для предсказуемых

последовательностей доступа.

2 Программный prefetching: Вызывается из кода с использованием специальных инструкций или методов. Позволяет разработчику явно указать, какие данные нужно загрузить.

16. Векторные архитектуры. Примеры

Векторизация – это генерация вместо скалярных инструкций векторных инструкций. Векторными называются те инструкции, операндами которых являются вектора. Процесс векторизации может выполняться компилятором или человеком.

Возможности векторизации компилятором ограничены, так как сложно на языке высокого уровня представить большинство векторных операций

- Архитектура x86 (x86-64): MMX, SSE-SSE4, SSSE3, 3DNow!
- Архитектура PowerPC: AltiVec, SPE
- Архитектура SPARC: VIS
- Эти расширения объединяет, то что размер вектора для каждой из них равен 128 бит
- 2010 Выпуск AVX расширения для x86, размер вектора 256 бит.

```
for (; i + 15 < size; i += 16) {  
    // Загружаем 16 элементов из массивов a  
    и b в регистры AVX-512  
    __m512 vec_a = _mm512_loadu_ps(&a[i]);  
    __m512 vec_b = _mm512_loadu_ps(&b[i]);  
    __m512 vec_result = _mm512_add_ps(vec_a,  
    vec_b);  
  
    _mm512_storeu_ps(&result[i],  
    vec_result);  
}
```

17. Векторизация. Векторные инструкции. Развертка циклов

Векторизация – это генерация вместо скалярных инструкций векторных инструкций. Частный случай автоматического распараллеливания кода.

Может выполняться компилятором или человеком (компилятором понятно что хуже).

Сложности если в коде появляются операции if, многомерные или разреженные матрицы.

Cray-1 первый векторный процессор.

Векторными называются те инструкции, операндами которых являются вектора. (есть в наборе avx2)

Типы векторных иструкций:

1 Векрикальные - берем условные переменные X0 с 1 источника, Y0 со второго, объединяем их в destination.

2 Горизонтальные - берем условные переменные $X1$, $X0$ с 1 источника, $Y1$, $Y0$ со второго, в destination записываем как $X1 + X0$ и $Y1 + Y0$.

3 Комбинированные - перемножаем n -е элементы с источников, каждые n и $n+1$ складываем и записываем в destination. Начинаем с $n=0$.

Развертка циклов - уменьшается количество исполняемых инструкций связанных с вычислением и проверкой условия.

Вместо:

```
For(int I = 0, I < N, I++)
```

```
Func()
```

Пишем:

```
For(int I = 0, I < N/2, I++) {
```

```
Func()
```

```
Func()
```

```
}
```

Необходимые условия:

1 цикл for

2 кол-во итераций не должно зависеть от данных

3 не известно во время компиляции количества итераций цикла

Недостатки:

1 чем больше инструкций, тем выше вероятность кэш-промаха.

2 может не хватить регистров.

Векторизация выполняется на основе цикла, только для операций, которые внутри цикла

Скалярные элементы объединяются в вектора и скалярные операции заменяются векторными.

Условия автоматической векторизации:

1 Операция в цикле

2 Итерации цикла не зависимы

3 Число итераций не зависит от данных

4 Внутри цикла не должно быть вызова процедур

5 Шаг между элементами в памяти - 1.

18. SIMD-инструкции. Примеры

SIMD (англ. Single Instruction stream / Multiple Data stream) – одиночный поток команд и множественный поток данных. Эти системы обычно имеют большое количество процессоров, от 1024 до 16384, которые могут выполнять одну и ту же инструкцию относительно разных данных в жесткой конфигурации. Инструкции SIMD можно найти, в той или иной степени, на большинстве процессоров, в том числе AltiVec фирмы IBM и SPE для PowerPC, HP PA-RISC Multimedia Acceleration eXtensions (MAX), MMX и iMMXt, SSE, SSE2, SSE3 SSSE3 и SSE4.x компании Intel и так далее.

- Целочисленные векторные инструкции входят в наборы MMX, SSE2, SSE3, SSSE3, SSE4.1

- В качестве аргументов инструкции могут использовать как 64-ех битные регистры MMX, так и 128-ми битные регистры XMM.

- Поддерживается два типа арифметики: с насыщением, без насыщения

(циклическая).

- Целочисленный векторный набора характеризуется отсутствие ортогональности к типам данных.

Примеры :

1) **x86/x86-64:**

- **MMX, SSE, SSE2, AVX, AVX-512** — работа с 64-, 128-, 256- и 512-битными регистрами.

2) **ARM:**

- **NEON** — 128-битные SIMD-инструкции.

- **SVE** — масштабируемые векторы до 2048 бит.

3) **PowerPC:**

- **Altivec** — 128-битные мультимедиа инструкции.

4) **RISC-V:**

- **RVV** — векторы переменной длины.

5) **MIPS:**

- **MSA** — SIMD-инструкции с регистрами 128 бит.

19. Конвейер. Характеристики конвейера. Конвейер инструкций

(в лекциях мало инфы, позже дополню)

1) Конвейер

Первый в 1960 UNIVAC США

Характеристики:

- Латентность (время выполнения одной команды на конвейере)

- Пропускная способность (время между поступлением двух команд на конвейер)

- Не конвейер

- Для быстрого выполнения малого количества операций

- Конвейер

- Для быстрого выполнения большого количества повторяющихся операций.

- Время выполнения одной операции больше чем у не конвейерной реализации.

Конвейер инструкций

- Стадии:
 - Выборка инструкций.
 - Выборка данных из регистров и декодирование инструкции.
 - Исполнение инструкции (вычисление адреса).
 - Обращение к памяти (загрузка и сохранение).
 - Запись данных в регистры

Что препятствует эффективному исполнению инструкций на конвейере?

- Зависимости между инструкциями.
- Наличие зависимостей приводит к возникновению конфликтов (hazard).

20. Предсказание переходов. Локальный предсказатель

Предсказание переходов является одним из наиболее эффективных способов борьбы с конфликтами по управлению. Идея заключается в том, что еще до момента выполнения команды условного перехода или же сразу после ее поступления делается предположение о наиболее вероятном результате выполнения такой команды (переход произойдет или нет). Последующие команды подаются на конвейер в соответствии с предсказанием.

При ошибочном предсказании конвейер необходимо вернуть к состоянию, с которого началась выборка ненужных команд, т.е. очистить конвейер, что эквивалентно приостановке конвейера. Цена ошибки может быть достаточно высокой, но при правильном предсказании имеем большой выигрыш – конвейер функционирует ритмично, без остановок. Выигрыш тем больше, чем выше точность предсказания.

Точность предсказания – процентное отношение числа правильных предсказаний к их общему количеству.

Доказано, что для того, чтобы снижение производительности конвейера из-за приостановок по управлению не превысило 10%, точность предсказания должна быть выше 97,7%.

В самом простом случае рассматривается результат самого последнего выполнения команды (однобитовая схема предсказания ветвлений). Биты предсказания ветвлений в однобитовой схеме называют переключателем – “принято/не принято” (taken /not taken).

В состоянии 0 прогнозируется, что переход не выполняется, в состоянии 1 – выполняемый. Сигналы ПВ (переход выполняется) и ПНВ (переход не выполняется) поступают на вход автомата из блока выполнения команды ветвления (Branch unit). С их помощью осуществляется коррекция прогноза.

Локальный предсказатель – результат тот, который для выдачи результат использует только историю своего перехода/

Предсказание ветвления:

- Две задачи:

- Предсказание выполнимости перехода.
- Предсказание адреса перехода.
- Используется на двух этапах:
 - Выборки инструкций.
 - Планирование и исполнение инструкций.

Обычно под предсказанием инструкции ветвления понимается предсказание выполнимости перехода, так как относительно заданный адрес перехода легко вычислим.

- Предсказатель условного перехода. Два режима работы:
 - Предсказание. На вход адрес инструкции, на выходе результат предсказания.
 - Корректировка. На вход адрес инструкции и направление перехода.

Результат работы – коррекция внутренних состояний.

Однобитная таблица предсказания ветвления:

- Элемент таблицы: однобитная ячейка.
 - Единица – выполнять переход
 - Ноль – пропустить переход
 - Если предсказание неверно, то инвертировать бит.
- Недостатки:
 - Плохо предсказывает поведение коротких циклов. Из-за того, что при выходе из цикла более вероятным становится пропуск перехода.

21. Конфликты исполнения инструкций на конвейере. Типы конфликтов

Конфликты – это некоторая ситуация, при которой невозможно запустить следующую инструкцию непосредственно за предыдущей, не нарушив критерия сохранения корректности программы.

Запуск инструкции происходит спустя n тактов.

Не атрибут программы или алгоритма, это следствие параллельного исполнения зависимых инструкций на процессоре.

Критерии сохранения корректности программы:

1. Сохранение потока данных:
 - поток значений передаваемых между инструкциями производителями и потребителями должен быть сохранен;
 - инструкции ветвления делают поток данных динамическим.
2. Сохранение поведения исключений
 - порядок, количество и тип генерируемых исключений должен сохраниться.

Типы конфликтов:

1. Структурные конфликты - разным инструкциям требуется доступ одному ресурсу процессора.
2. Конфликты по данным - следствие наличия зависимостей по данным именам.

3. Конфликты по управлению - следствие наличия зависимости по управлению.

Способы разрешения конфликтов:

- Программные:
 - обнаружение конфликтов в процессе компиляции и добавление инструкций пор;
 - статическое планирование конвейера.
- Аппаратные:
 - добавление устройства обнаружения конфликтов и останова конвейера;
 - добавление специфичных аппаратных решений.

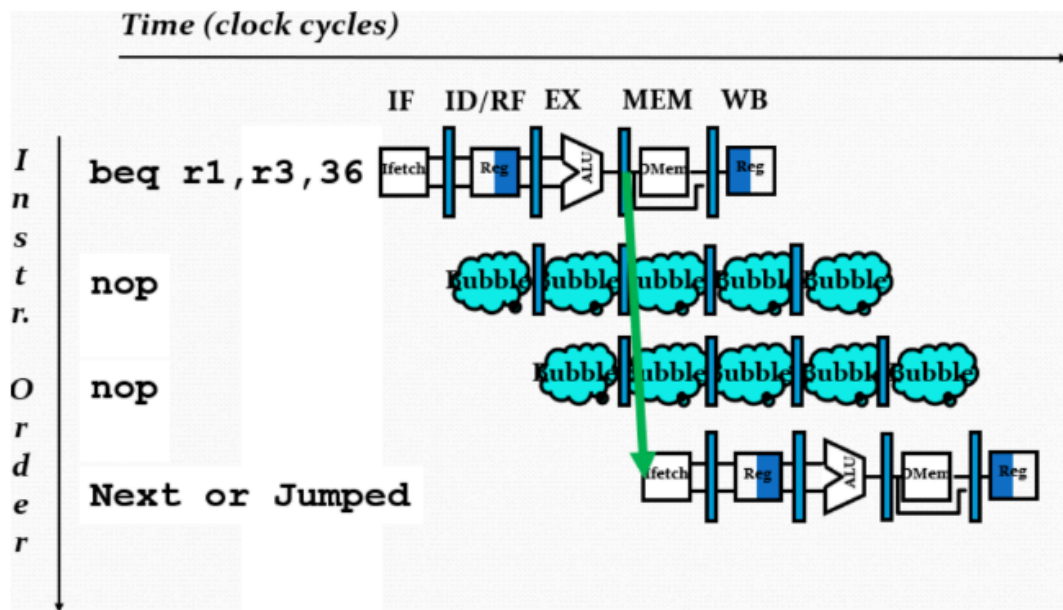
22. Конфликт по управлению, способы устранения, слот ожидания

Конфликты по управлению возникают при конвейеризации команд переходов и других команд, изменяющих значение счетчика команд. Суть конфликтов этой группы наиболее удобно проиллюстрировать на примере команд условного перехода. Пусть в программе команда $i+1$ является командой условного перехода, формирующей адрес следующей команды в зависимости от результата выполнения команды i . Команда i завершит свое выполнение в такте 5. В то же время команда условного перехода уже в такте 3 должна прочитать необходимые ей признаки, чтобы правильно сформировать адрес следующей команды. Если конвейер имеет большую глубину (например, 20 ступеней), то промежуток времени между формированием признака результата и тактом, где он анализируется, может быть еще большим. В инженерных задачах примерно каждая шестая команда является командой условного перехода, поэтому приостановки конвейера при выполнении команд переходов до определения истинного направления перехода существенно скажутся на производительности процессора.

Наиболее эффективным методом снижения потерь от конфликтов по управлению служит предсказание переходов или вставка пор инструкций (менее эффективна). Если после формирования анализируемых признаков оказалось, что направление перехода выбрано верно, все полученные результаты переписываются из буфера по месту назначения, а выполнение программы продолжается в обычном порядке. Если направление перехода предсказано неверно, то буфер результатов очищается. Также очищается и конвейер, содержащий команды, находящиеся на разных этапах обработки, следующие за командой условного перехода. При этом аннулируются результаты всех уже выполненных этапов этих команд. Конвейер начинает загружаться с первой команды другой ветви программы. Так как конвейерная обработка эффективна при большом числе последовательно выполненных команд, то перезагрузка конвейера приводит к значительным потерям производительности.

Конфликт управления

1 Конвейер



Если что, слот ожидания - и есть это время, которое мы заполняем с помощью инструкцию nop(no operation - это ассемблерная инструкция)

23. Статическое и динамическое планирование инструкций на примере VLIW архитектуры

внеочередное исполнение (англ. out-of-order execution)

- исполнение машинных инструкций не в порядке следования в машинном коде, а в порядке готовности к выполнению
- основано на алгоритме Томасуло и его развитии

• переименование регистров (англ. register renaming)

- способ реализации параллелизма на уровне команд
- переименование регистров основано на таком же подходе
 - между архитектурными регистрами (x $\hat{Z}$ $\hat{Z}$: eax, ebx, ...) и физическими нет строгого соответствия
 - при этом физических регистров существенно больше
 - при исполнении на CPU архитектурному регистру назначается не постоянная физическая копия. Если блок исполнения фиксирует, что архитектурный регистр может быть заменен на другой физический без нарушения результата исполнения, данная замена осуществляется (!)

Переименование регистров

Версия 1	Версия 2
r1 = m[1024]	r1 = m[1024]
r1 += 2	r1 += 2
m[1032] = r1	m[1032] = r1
r1 = m[2048]	r2 = m[2048]
r1 += 4	r2 += 4
m[2056] = r1	m[2056] = r2

Технология склейки микроопераций (Micro-fusion)

Объединение в одну микрооперацию несколько зависимых микроопераций при этом результирующая микрооперация должна порождать один результат при внеочередном исполнении нет смысла разделять зависимые операции, т.к. они не могут исполняться параллельно, а всегда будут исполняться последовательно

Наличие нескольких зависимых микроопераций приводит к тому, что станции резервирования и ячейки ROB используются впустую

Объединенная микрооперация будет занимать одну станцию резервирования и одну ячейку ROB

Для поддержки технологии необходимо, чтобы станция резервирования могла хранить дескрипторы для трех и более операндов

После стадии декодирования должно находиться устройство, которое выполняет склейку

Ошибки в микрооперациях

Intel Pentium (1994 г.)

-вещественный сопроцессор

-ошибка в таблицах поиска

а (пример):

$4195835 * 3145727 / 3145727 = 4195579$

Skylake / Kaby Lake :

"Under complex micro-architectural conditions, short loops of less than 64 instructions that use AH, BH, CH or DH registers as well as their corresponding wider register (e.g. RAX, EAX or AX for AH) may cause unpredictable system behavior. This can only happen when both logical processors on the same physical processor are active." "Intel Skylake / Kaby Lake processors: broken hyper-threading" (сообщество Debian)

24. Механизмы ускорения выборки инструкций (внеочередное исполнение инструкций, переименование регистров, технологии микро- и макро-fusion)

VLIW (Very Long Instruction Word) Исторический обзор архитектур. Способы классификации

- архитектура, специально разработанная для повышения количества одновременно исполняемых инструкций
- позволяет создать более простой процессор, чем RISC архитектура

Ключевая идея VLIW Исторический обзор архитектур. Способы классификации

- дано: конкретный CPU с фиксированным числом функциональных устройств (ФУ)
- принцип работы: — инструкция-пакет простых подинструкций (их кол-во = число ФУ) — пакет обрабатывается ЦЕЛИКОМ по одному за такт

Статический VLIW процессор

- особенность: за запуск инструкций отвечает компилятор компилятор должен правильно расставить пор-ы

* если инструкция пойдет на исполнение до того момента, как предыдущая инструкция закончится произойдет сбой

* как быть с устройствами со случайной латентностью (например, устройство загрузки/сохранения при наличие кэша)

* компилятор всегда выбирает максимальную латентность для устройств со случайной латентностью

недостатки:

- неэффективность кэша

- очень большой размер программы из-за большого количества пор инструкций

Динамический VLIW процессор

Особенность: за запуск инструкций отвечает процессор

- процессор ожидает готовности данных для всех инструкций в пакете.

- только после этого запускает пакет

Позволяет эффективно использовать кэш

Уменьшает размер программы

Пример

Ld\St unit 0	Ld\St unit 1	Integer Alu	Branch Unit
Load R1, A	Load R2, B	Nop	nop
nop	nop	Add R3, R2, R1	nop
Store C,R3	nop	nop	nop

Общие проблемы VLIW архитектур

- Большой размер скомпилированной программы.

- Отсутствие бинарной совместимости.

-изменение количественного и качественного состава ФУ процессора приводит к бинарной несовместимости с предыдущей версией.

-это приводит к тому, что код нужно перекомпилировать для каждого процессора.

25. Спекуляция. Спекулятивный суперскалярный процессор. Основные этапы исполнения инструкций

Сокращения:

- ФУ - функциональное устройство
- СР - станция резервирования
- БУ - буфер упорядочивания
- ССП - спекулятивный суперскалярный процессор

- **что-то там** - к вопросу вроде как не относится, но было в ответах прошлых лет

Спекуляция - выполнение некоторой операции, необходимость выполнения которой носит вероятностный характер.

ССП способен выполнять инструкции, которые следуют за невыполненной инструкцией условного перехода, основываясь на данных предсказателя ветвления, и в случае неверного предсказания выполняется переход.

Этапы исполнения инструкции ССП:

1. Выборка инструкции:

- Планирование инструкций осуществляется с вершины очереди планирования.
- Для выполнения планирования должны быть доступны:
 - станция резервирования на требуемом ФУ
 - ячейка в БУ

Если хотя бы один из ресурсов не доступен, то инструкция ожидает в очереди.

2. Выборка инструкции

3. Ожидание готовности операндов:

- Проверяется существование операндов в регистровом файле. Если они существуют, то записываются в СР.
- Если операндов нет в регистровом файле, то проверяется их существование в БУ. Если операнды есть в БУ, то они записываются в СР.
- Если и в БУ операнды не найдены, то в СР записывается номер ячейки БУ, в которой содержится инструкция для вычисления этого операнда(-ов).
- В файл переименования для результирующего регистра инструкции помещается номер ячейки БУ, которая ее содержит.

4. Исполнение инструкции:

- Инструкции запускаются на исполнении по мере готовности их операндов и доступности ФУ.
- По общей шине передается результат инструкции и номер ячейки БУ, которая ее содержит.

5. Запись результата:

- Запись результата производится одновременно во все ожидающие СР и БУ.
- Инструкция загрузки выполняется в две стадии:
 - Вычисление адреса.

- ii. Чтение из памяти. Запускается если нет конфликтов с предыдущими инструкциями загрузки и сохранения.
- iii. Инструкция сохранения на этапе исполнения вычисляет адрес операнда и ожидает готовности результата, запись в память произойдет на стадии завершения.

6. Завершение (commit):

- a. Инструкция после вычисления результата ожидает в БУ своего завершения.
- b. Завершаются инструкции, которые находятся на вершине БУ. БУ поддерживает поведение очереди FIFO.
- c. На этапе завершения:
 - Для арифметических инструкций и инструкций загрузки проверяется наличия исключений и если исключений нет, то результат записывается регистровый файл.
 - Для инструкции сохранения проверяется наличия исключений и если исключений нет, то результат записывается в память.
 - Для инструкции условного перехода проверяется правильно она была предсказана или нет, если да, то инструкция удаляется из БУ, если нет, происходит сброс процессора: очищаются все стадии, на планирование отправляется первая инструкция из правильной ветви.

Изменения в аппаратной схеме:

- 1. Добавлен буфер упорядочивания (БУ) (record buffer).
 - a. Используется для:
 - i. Хранения результатов работы инструкций между стадией записи результата и стадией завершения.
 - ii. Поддержание программного порядка инструкций на стадии завершения.
 - b. В состав БУ входят:
 - i. Выполняемая инструкция. Это может быть арифметическая операция, загрузка, сохранение, условный переход и другие.
 - ii. Адрес результата.
 - iii. Поля для временного хранения результата выполнения инструкции.
- 2. Удален буфер сохранения.

Ограничения спекуляции:

- 1. Если инструкция в спекулятивном режиме вызвало кэш промах стоит его обрабатывать или нет?

Обрабатываются обычно только самые легкие события, такие как промах в кэш первого уровня, остальные события откладываются до выхода инструкции из спекулятивного режима.

2. Ошибочная спекуляция приводит к выполнению бессмысленной работы и трате энергии.

Количество инструкций условного перехода над которыми производится спекуляция может быть ограничена.

Спекуляция по управлению:

- **Спекуляция по управлению** - возможность корректно исполнять инструкции в ветви, для которой еще не вычислено условие.
- Количество инструкций условного перехода над которыми производится спекуляция может быть ограничена.
- Поддержка спекуляции по управлению включена в состав архитектуры EPIC.

Спекуляция по данным:

- Спекулятивный суперскалярный процессор может маскировать большую латентность инструкций загрузки за счет внеочередного исполнения других инструкций.
- EPIC процессор обрабатывает последовательности независимых инструкций в программном порядке и поэтому не может пропустить зависимые инструкции.

Чтобы скрыть латентность инструкций загрузки он должен выполнять их как можно раньше. Для этого компилятор должен перенести инструкцию загрузки как можно выше по коду, относительно инструкции которая от нее зависит.

26. Алгоритм Томасуло. Планирование инструкций

Томасуло - алгоритм динамического планирования инструкций, который позволяет исполнять инструкции в порядке отличном от программного. При одновременном исполнении двух и более инструкций позволяет разрешить RAW (read after write), и устранить WAR (write after read) и WAW (write after write) конфликты

- Достоинства:
 - повышение пропускной способности
 - уменьшение времени простоя процессора
- Недостатки:
 - большие аппаратные затраты на реализацию дополнительных устройств

(синим текстом - для лучшего понимания с, писать необязательно)

Состав процессора:

- очередь планирования (Instruction queue)[ОП]
- регистровый файл (FP Registers) [РФ]
- станции резервирования (Reserve Station) [СР]
- простое вещественное устройство (FP adder)
- сложное вещественное устройство (FP Multiplier)
- общая шина данных (Common Data Bus)[ОШД]
- устройство вычисления адреса (Address Unit)
- буфера загрузки (Load buffer)
- буфера сохранения (Store buffer)
- устройство работы с памятью (Memory Unit)

Планирование инструкций:

1 Выборка с вершины ОП

- выборка происходит по 1 инструкции за такт
- выборка осуществляется в программном порядке, так как очередь

FIFO

2 Декодирование

- назначение на исполнительное устройство
- если все СР устройства заняты, то инструкция возвращается в ОП и ожидает освобождения RS

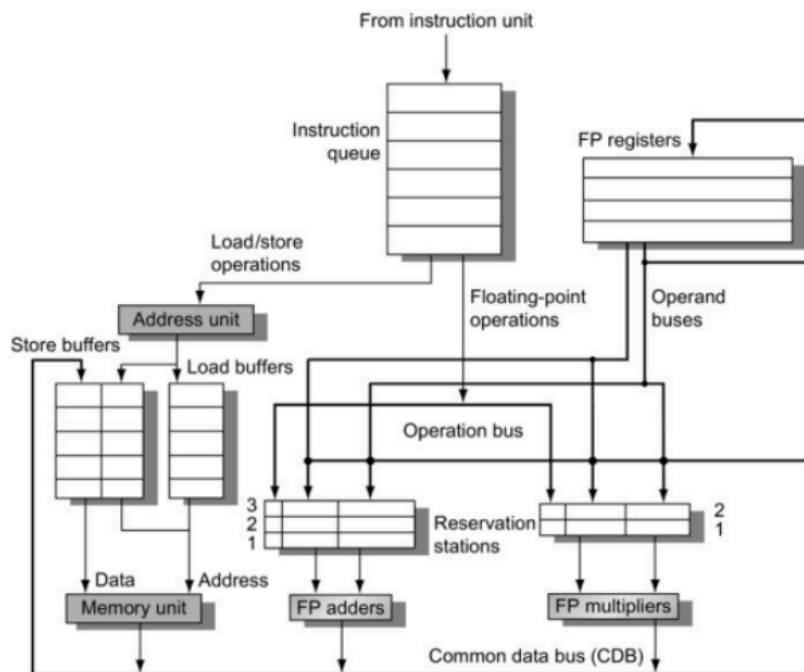
3 Выборка операндов

- если операнды вычислены, то они выбираются из РФ, если нет то в дескрипторах СР устанавливается ссылки на другие СР

27. Алгоритм Томасуло. Состав процессора. Этапы исполнения инструкций

Состав процессора:

- очередь планирования (Instruction queue)[ОП]
- регистровый файл (FP Registers) [РФ]
- станции резервирования (Reserve Station) [СР]
- простое вещественное устройство (FP adder)
- сложное вещественное устройство (FP Multiplier)
- общая шина данных (Common Data Bus)[ОШД]
- устройство вычисления адреса (Address Unit)
- буфера загрузки (Load buffer)
- буфера сохранения (Store buffer)
- устройство работы с памятью (Memory Unit)



Этапы исполнения:

Классический конвейер:

- выборка инструкции
- выборка операндов
- исполнение
- сохранение результата

Томасуло:

- выборка инструкции
- планирование инструкции
- ожидание готовности операндов
- исполнение
- сохранение результата

Обработка инструкций загрузки и сохранения

- инструкция загрузки:
 - вычисление адреса
 - выполнение загрузки по адресу
- инструкция сохранения:
 - вычисление адреса
 - ожидания готовности операнда
 - выполнение сохранения по адресу
- буфер загрузки: поле адреса
- буфер сохранения: поле адреса, СР для операнда

Ожидание готовности операндов, исполнение, сохранение результатов

- инструкция ожидает в СР до тех пор, пока не будут вычислены все ее операнды и записаны в соответствующие дескрипторы
- передача вычисленного операнда происходит по ОШД вместе с номером СР, который его содержал

- каждая СР слушает ОШД и сравнивает значение номер передаваемого по ней СР с ожидаемым. Если номера совпадают то она забирает значение операнда с ОШД
- передаваемые по ОШД данные сохраняются в РФ
- если все операнды находятся СР, то инструкция отправляется на исполнение

Порядок исполнения инструкций загрузки и сохранения

- определяется наличием зависимостей между инструкциями по ячейкам памяти
- адреса ячеек вычисляются на первом этапе
- инструкция загрузки ожидает завершения всех предшествующих инструкций сохранения по данному адресу
- инструкция сохранения ожидает завершения всех предшествующих инструкций загрузки и сохранения по данному адресу

28. Спекулятивный суперскалярный процессор

Спекуляция – выполнение некоторой операции, необходимость выполнения которой носит вероятностный характер.

СПП - способен выполнять инструкции, которые следуют за невыполненной инструкцией условного перехода, основываясь на данных предсказателя ветвления, и в случае неверного предсказания выполнять откат.

Этапы исполнения инструкции:

1. Выборка инструкции
2. Планирование инструкции
3. Ожидание готовности операндов
4. Исполнение
5. Запись результата
6. Завершения (commit)

Изменения в аппаратной схеме:

- Добавлен буфер упорядочивания (для хранения результатов инструкций между стадией записи результата и стадией завершения) (содержит выполняемую инструкцию и адрес результата инструкции).
- Поля для временного хранения результата инструкции
- Убран буфер сохранения

Этап планирования:

- Инструкции планируются сверху очереди.
- Инструкция выполняется, если есть свободный ресурс (СР) и ячейка в блоке управления (БУ). Если ресурсов нет – инструкция ждёт.
- Операнды выбираются в три этапа:
 - Проверка регистров – если данные есть, они копируются в СР.
 - Если в регистрах нет – проверка БУ. Данные копируются, если найдены.

- Если нигде нет – записывается номер ячейки БУ, которая вычислит операнд

Этап исполнения и записи результата:

- Инструкции выполняются, когда готовы операнды и есть свободный ФУ.
- Результат и номер ячейки БУ передаются по общей шине.
- Запись идёт сразу во все ожидающие СР и БУ.
- Загрузка выполняется в два этапа:
 - Адрес вычисляется.
 - Данные читаются из памяти (если нет конфликтов с другими загрузками/сохранениями).

Этап завершения:

- Инструкция ждёт завершения в БУ после вычисления результата.
- Завершаются инструкции сверху очереди БУ (FIFO).
- Если не спекуляция, то проверка на исключения. Если их нет — запись в регистры.
- Если условный переход, то проверка предказания. Если верно — удаление из БУ, если нет — сброс процессора и запуск с правильной ветви

Ограничения спекуляции:

- Ошибочная спекуляция тратит ресурсы и энергию, НО кол-во инструкций с условным переходом может быть ограничено.
- Инструкция в спекулятивном режиме может вызвать кэш-промах, НО обрабатываются только самые легкие события (например, промах L1), а остальные откладываются до выхода из спекулятивного режима.

29. EPIC. Механизмы поддержки спекуляции

EPIC (*англ. explicitly parallel instruction computing* — «вычисление с явным параллелизмом машинных команд»)

- Призвана сохранить достоинства VLIW архитектуры и устранить ее недостатки.
- Достоинства VLIW и EPIC: возможность параллельного исполнения инструкций задается на уровне архитектуры, а не определяется динамически в процессе исполнения.
- Недостаток VLIW/ преимущества EPIC: решена проблема бинарной совместимости между различными процессорами.

(синим текстом - для лучшего понимания, писать необязательно)

• VLIW:

- параллельное выполнение инструкции задается в рамках одного пакета.
- количество инструкций в одном пакете определяется количеством исполняемых устройств в процессоре.

• EPIC:

- параллельно выполняемые инструкции задаются специальными шаблонами в пакете.
- количество параллельно исполняемых инструкций может быть больше количества инструкций в пакете.
- количество инструкций в пакете определяется архитектурой и не зависит от аппаратной реализации.

Проблема бинарной совместимости:

- Программы, скомпилированные для одного процессора, могут не работать оптимально или вовсе не работать на другом процессоре с другим количеством функциональных устройств (ФУ).
- В архитектуре EPIC (включая Itanium) задача назначения инструкций на ФУ перекладывается на процессор, а не на компилятор.
- Процессор сам определяет, какие инструкции запускать на каком устройстве, учитывая своё текущее количество ФУ и их доступность.
- Это означает, что программа не "зависит" от конфигурации конкретного процессора, что обеспечивает бинарную совместимость.

Примеры (??)

- разработки архитектуры следующего за VLIW поколения начались в компании HP в конце 80-ых годов.
- в 1993 году разработана первая EPIC архитектура PlayDoh
- в 1997 году анонсирована архитектура Intel IA-64 (Intel и HP)
- в 2001 выпущен первый процессор Intel Itanium
- в 2002 выпущен процессор Intel Itanium 2

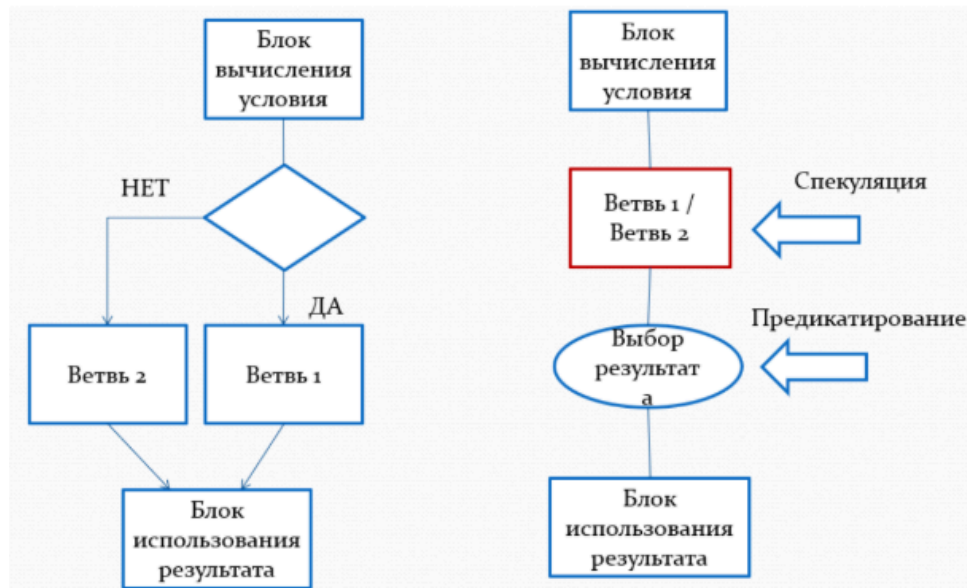
Спекулятивное выполнение — это метод оптимизации, при котором компьютерная система выполняет некоторую задачу, которая может оказаться ненужной. Работа выполняется до того, как станет известно, действительно ли она необходима, чтобы предотвратить задержку, которая может возникнуть при выполнении работы после того, как станет известно, что она необходима. Если выясняется, что работа в конце концов не нужна, большинство изменений, внесённых в работу, отменяется, а результаты игнорируются.

Механизмы поддержки спекуляции:

• **Спекуляция по управлению**

- это возможность корректно исполнять инструкции в ветви, для которой еще не вычислено условие.
 - процессоры с архитектурой RISC и CISC выполняют спекуляцию на аппаратном уровне. Первоначально такой возможности в эти архитектуры не закладывалось.
 - поддержка спекуляции по управлению включена в состав архитектуры EPIC
- спекуляция по управлению в EPIC основано на:
- наличие двух типов инструкций: спекулятивных и не спекулятивных;
 - параллельном исполнении спекулятивных инструкций из двух ветвей;
 - использование предикатирования для выбора результат корректной ветви;

- использование третьего состояния регистров для сохранения поведения исключений во время спекуляции;
- все исключения генерируемые спекулятивными инструкциями являются отложенными;
- спекулятивная инструкция, сгенерировавшая отложенное исключение, помещает в свой регистр результата значение NaT (Not a Thing)
(реализуется добавлением дополнительного бита к каждому регистру процессора; спекулятивными являются только те инструкции, которые сохраняют данные в регистры с поддержкой состояния NaT; не спекулятивные инструкции сохраняют результаты в память или обычные регистры)



Программная спекуляция по управлению в EPIC:

- достоинства:
 - маленькие аппаратные затраты на реализацию
 - нет затрат на неверное предсказание
- недостатки:
 - требует исполнения большего количества инструкций
 - проигрывает по скорости аппаратной спекуляции в случае низкой вероятности неверного предсказания перехода

• Спекуляция по данным

Это выполнение инструкции загрузки данных (чтения из памяти) раньше времени, даже если процессор ещё не знает, понадобится ли результат этой загрузки или нет.

Процессор не может просто "перепрыгнуть" через инструкции, которые зависят друг от друга.

Чтобы ускорить выполнение, процессор или компилятор должны попытаться загрузить данные раньше, но при этом нельзя нарушить порядок выполнения: например, нельзя прочитать данные, которые ещё не были записаны.

"Компилятор переносит загрузку вверх"

Компилятор старается вставить инструкцию загрузки (чтения) как

можно раньше в коде, до того, как данные понадобятся. Это помогает "скрыть" время ожидания загрузки данных.

Проблема совпадения адресов

Если в коде сначала идёт запись (st) по адресу, а потом чтение (ld) с того же адреса, их нельзя менять местами. Компилятор должен проверить, совпадают ли адреса записи и чтения. Но эта задача слишком сложная для компилятора (он не всегда может заранее узнать адреса), поэтому её решают на уровне процессора.

Решение на стороне процессора

- Компилятор всё равно переносит загрузку вверх, как будто конфликта адресов нет.

- Перед использованием загруженных данных процессор вставляет специальную инструкцию проверки.

- Если произошёл конфликт (данные изменились после загрузки), процессор может перезапустить загрузку данных или сгенерировать исключение для исправления ситуации.

30. EPC. Пакет инструкций – способ явного задания параллелизма уровня команд

Параллелизм уровня команд (Instruction Level Parallelism, ILP) – это потенциальное перекрытие нескольких инструкций во времени. Благодаря ILP такие инструкции могут выполняться параллельно, повышая производительность.

Базовый блок – это последовательность инструкций, имеющая одну точку входа и одну точку выхода. Параллелизм присутствует среди инструкций базового блока.

Для получения существенного прироста производительности необходимо использовать параллелизм не только внутри, но и между базовыми блоками.

Простейший способ увеличения ILP – использование параллелизма итераций цикла. Разворачивание циклов (Loop Unrolling) – увеличение числа инструкций базового блока (копирование тела цикла несколько раз).

Определение степени зависимости между инструкциями – критический фактор для определения уровня параллелизма в программе. Независимые инструкции могут выполняться одновременно. Зависимые должны выполняться по порядку, однако они могут частично перекрываться во времени.

Зависимость – свойство программы.

Конфликт, возникший (или не возникший) из зависимости – свойство организации архитектуры.

Существует 3 типа зависимостей:

- Зависимости по данным.
- Зависимости по именам.
- Зависимости по управлению.

31. Технология OpenMP

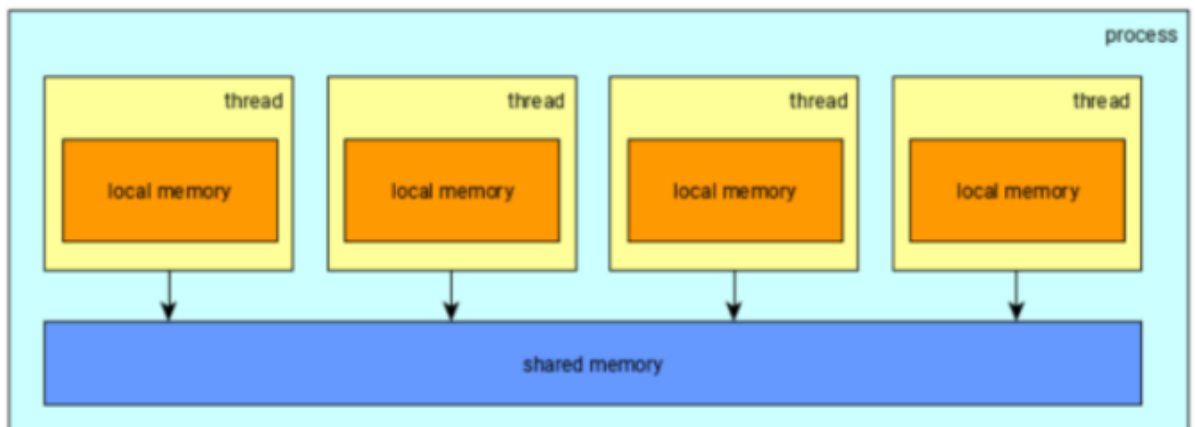
OpenMP – библиотека для параллельного программирования вычислительных систем с общей памятью.

Поддерживаемый языки программирования:

C, C++, Fortran

Поддержка версий:

- актуальная версия – OpenMP 5.2
- MS Visual Studio – OpenMP 2.0 (!!!!) планов по актуализации нет
- GCC 6 – базовая поддержка OpenMP 4.5
- GCC 11 – ожидается полная поддержка OpenMP 4.5 и частичная поддержка OpenMP 5.0



Формат директив:

```
#pragma omp <директива> [раздел [ [,] раздел]...]
```

директивы:

- parallel
- for
- parallel for
- section или sections
- single
- master
- critical
- flush
- ordered
- atomic

parallel - распараллеливание потока в отмеченной директивой области видимости.

```
// A - single thread
#pragma omp parallel
{
    // B - many threads
}
```

parallel for - директива распараллеливания цикла

```
#pragma omp parallel for
for (i=0; i < 100; i++)
{
    temp = array[i];
    array[i] = do_something(temp);
}
```

*опция `private` позволяет сделать внешнюю общую переменную приватной, т.е. значения переменной между потоками никак не связаны.

reduction

Формат директивы: `reduction(оператор: список)`

Возможные операторы — "+", "*", "-", "&", "|", "^", "&&", "||".

Список — перечисляет имена общих переменных. У переменных должен быть скалярный тип (например, `float`, `int` или `long`, но не `std::vector`, `int []` и т. д.).

Принцип работы:

Для каждой переменной создаются локальные копии в каждом потоке.

Локальные копии инициализируются соответственно типу оператора. Для аддитивных операций — 0 или его аналоги, для мультипликативных операций — 1 или ее аналоги.

Над локальными копиями переменных после выполнения всех операторов параллельной области выполняется заданный оператор. Порядок выполнения операторов не определен.

планирование циклов

```
#pragma omp parallel for schedule(kind [, chunk size])
```

два основных режима планирования:

статический

- `schedule(static)`
- `schedule(static, 10)`

динамический

- `schedule(dynamic)`
- `schedule(dynamic, 10)`

1. Тип `static`:

Если размер не указан, все множество итераций цикла делится на примерно равные части (по числу входящих в них итераций) в соответствии с числом потоков и каждому потоку назначается своя часть. Минимальные затраты на планирование, но возможна несбалансированность нагрузки по потокам.

Размер указан. В этом случае все множество итераций цикла делится на части указанного размера (по числу входящих в них итераций) и полученные части назначаются потокам циклически.

2. Тип `dynamic`. Части множества итераций (количество итераций в каждой части равно указанному размеру, а если он не указан, то 1 по умолчанию) назначаются каждому потоку по запросу. Т.е., что после начального распределения частей очередная часть достается тому, кто раньше закончил свою работу. Хорошая балансировка нагрузки по потокам, но большие временные затраты на планирование (поскольку операция запроса потока на очередную порцию итераций цикла и его удовлетворения требует относительно больших временных затрат).

Директива `sections`

Директива `omp sections` распределяет работу между потоками, привязанными к определенной параллельной области.

```
#pragma omp parallel
{
    #pragma omp for
    for (i=0; i<x; i++)
        Fn1();
    #pragma omp sections
    {
        #pragma omp section
        {
            TaskA();
        }
        #pragma omp section
        {
            TaskB();
        }
        #pragma omp section
        {
            TaskC();
        }
    }
}
```

Также существуют директивы `master` для указания области исполнения главным потоком без определения критерия главного потока вручную и `nowait`, не производящий синхронизацию потоков в конце блока.

Примеры функций `omp`:

`int omp_get_num_threads(void);` позволяет получить количество потоков, задействованных для выполнения данной задачи. (возвращает 1 вне многопоточной секции)

`int omp_set_num_threads(int NumThreads);` устанавливает число потоков, которое будет создано при входе в параллельную секцию.

`int omp_get_num_procs(void);` возвращает число доступных физических потоков (учитывая многопоточность ядра процессора).

32. Задание ядра, потоки и блоки потоков на примере перемножения двух матриц в CUDA

Разделяемая(общая) память расположена в том же устройстве, что и кэш L1, совместно используется (разделяется) всеми нитями виртуального блока. Если на мультипроцессоре работает несколько блоков, общая память делится между ними поровну. У каждого блока своё адресное пространство общей памяти конфигурации:

16KB общая память, 48KB L1

48KB общая память, 16KB L1 – по умолчанию

32 KB общая память, 32 KB L1

Выделение разделяемой памяти - в GPU коде объявляем статический массив или переменную с атрибутом `__shared__`

Особенности использования:

Переменные с атрибутом `__shared__` с точки зрения программирования:

- существуют только на время жизни блока
- недоступны с хоста или из других блоков
- индивидуальны для каждого блока и привязаны к его личному пространству общей памяти
- каждый блок нитей видит «свое» значение
- не могут быть проинициализированы при объявлении

Пример перемножения матриц с моделью куды:

```
#include <math.h>
#include <iostream>
#include "cuda_runtime.h"
#include "kernel.h"
#include <stdlib.h>
using namespace std;

__global__ void matrixMultiplicationKernel(float* A, float* B,
float* C, int N)
{
    int ROW = blockIdx.y * blockDim.y + threadIdx.y;
    int COL = blockIdx.x * blockDim.x + threadIdx.x;
    float tmpSum = 0;
    if (ROW < N && COL < N) {
        // each thread computes one element of the block sub-matrix
        for (int i = 0; i < N; i++) {
            tmpSum += A[ROW * N + i] * B[i * N + COL];
        }
    }
    C[ROW * N + COL] = tmpSum;
}

void matrixMultiplication(float *A, float *B, float *C, int
N){
    // declare the number of blocks per grid and the number of
    threads per block
    // use 1 to 512 threads per block
    dim3 threadsPerBlock(N, N);
    dim3 blocksPerGrid(1, 1);
    if (N*N > 512){
        threadsPerBlock.x = 512;
        threadsPerBlock.y = 512;
```

```

        blocksPerGrid.x =
ceil(double(N)/double(threadsPerBlock.x));
        blocksPerGrid.y =
ceil(double(N)/double(threadsPerBlock.y));
    }
    matrixMultiplicationKernel<<<blocksPerGrid,threadsPerBlock>>>(
A, B,          C, N);
}

```

33. Структура ядра и адресация на примере перемножения двух матриц в CUDA

```

__global__ void mm_kernel(float* A, float* B, float* C,
int n)
{
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    if (row < n && col < n) {
        for (int i = 0; i < n; ++i) {
            C[row * n + col] += A[row * n + i] * B[i * n + col];
        }
    }
}

```

34. Синхронизация потоков, дивергенция потоков, функции голосования в CUDA. Примеры

Взаимодействие между потоками одного блока следует стараться осуществлять через их общую быструю разделяемую память, а между потоками двух различных блоков — только с использованием глобальной памяти. Тут и возникает обозначенная во введении проблема: слежение за актуальностью данных в общедоступной разным потокам для чтения и записи области памяти. Иначе говоря — *проблема синхронизации потоков*. Как уже отмечалось, в рамках одного блока потоки каждого warp'a синхронизированы между собой. Для синхронизации потоков блока вне зависимости от принадлежности к warp'ам существует несколько команд барьерного типа:

- **__syncthreads()** — самый верный способ. Эта функция заставит каждый поток ждать:
 - пока все остальные потоки этого блока достигнут этой точки
 - все операции по доступу к разделяемой и глобальной памяти, совершенные потоками этого блока, завершатся и станут видны потокам этого блока.

Не надо размещать эту команду внутри условного оператора *if*, но следует обеспечивать безусловный вызов этой функции всеми потоками блока.

- **__threadfence_block()** - будет заставлять ждать вызвавший её поток, пока все совершенные операции доступа к разделяемой и глобальной памяти завершатся и станут видны потокам этого блока.

- **__threadfence()** - будет заставлять ждать вызвавший её поток, пока все совершенные операции доступа к разделяемой памяти станут видны потокам этого блока, а операции с глобальной памятью — всем потокам на «устройстве». Под «устройством» понимает графический адаптер.
- **__threadfence_system()** - подобна **__threadfence()**, но включает синхронизацию с потоками на CPU («хосте»), при использовании весьма удобной page-locked памяти. Подробнее в [1,2] и некоторых других источниках, приведенных в списке ниже.

Первая команда ставит единый барьер для всех потоков одного блока, три остальные - для каждого потока свой независимый барьер.

Дивергенция варпов - ситуация, возникающая когда в нитях одного варпа одна и та же инструкция условного перехода отработала по разному (например, для одной нити отработал блок *if*, а для другой нити отработал блок *else*). Другими словами, в одной группе нитей переход выполняется а в другой нет. Часто возникает в циклах, содержащих специфические условия.

Из-за того, что инструкции всех тредов внутри варпа выполняются синхронно, то исполнение всех тредов, которые выполнили прямой переход будет приостановлено, до того момента пока остальные треды не достигнут той же инструкции.

Дивергенция приводит к тому, что время исполнения одного варпа определяется суммарным временем исполнения всех выполненных ветвей.

С точки зрения производительности лучше, когда несколько тредов в варпе делают меньше работы по сравнению с остальными, чем когда несколько тредов делают работы больше чем остальные.

Функции голосования (начиная с CUDA 2.x) - набор встроенных функций, которые предоставляют возможность “общения нитей” в пределах варпа.

- **__all(bool value)** - возвращает 1, если все нити в варпе имеют значение 1, иначе 0.
- **__any(bool value)** - Возвращает 1, если хотя бы одна нить в варпе имеет значение 1, иначе 0.
- **__ballot(bool(неточно) value)** - Возвращает 32-битное значение, где каждый бит соответствует одной нити в варпе и содержит ее значение.

Пример дивергенции варпов (ГПТ)

```
__global__ void kernel(int *data, int n) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
```

```

        if (tid < n) {
            if (data[tid] > 0) {
// Операции, выполняемые только для положительных элементов
                data[tid] *= 2;
            } else {
// Операции, выполняемые только для отрицательных элементов
                data[tid] /= 2;
            }
        }
    }
}

```

Пример использования функций голосования (ГПТ)

Задача: найти минимальный элемент числового массива.

```

#include <cuda_runtime.h>

__global__ void findMin(int *data, int *minVal, int n) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;

    // Проверяем границы массива
    if (tid < n) {
        int myVal = data[tid];

        // Инициализируем локальное минимальное значение
        int localMin = myVal;

        // Итеративно находим минимальное значение в варпе
        for (int s = warpSize / 2; s > 0; s >>= 1) {
            int neighbor = myVal + s;
            if (neighbor < n) {
                localMin = __min(localMin, data[neighbor]);
            }
        }

        // Все нити в варпе записывают свое локальное
        // минимальное значение
        data[tid] = localMin;

        // Используем функцию голосования для определения
        // глобального минимума
        *minVal = __all(data[tid] == localMin) ? localMin :
        *minVal;
    }
}

```

35. Архитектура современного GPU

Графический процессор (GPU) — это специализированное устройство для параллельной обработки больших объемов данных.

Основное назначение современных GPU — выполнение задач, связанных с графикой, вычислениями в реальном времени, машинным обучением, научными вычислениями и другими интенсивными вычислительными нагрузками.

Архитектура GPU разработана с учетом максимальной эффективности выполнения параллельных вычислений.

Основные компоненты архитектуры GPU:

1. Множество ядер (SIMD-архитектура):

- GPU состоит из сотен или тысяч вычислительных ядер.
- Каждое ядро способно выполнять одну и ту же инструкцию на разных данных (Single Instruction, Multiple Data, или SIMD).
- Такая архитектура подходит для задач, где нужно одновременно обработать массивы данных, например, пиксели изображения или элементы матрицы.

2. Многоуровневая организация вычислительных блоков:

- Streaming Multiprocessors (SMs): GPU состоит из нескольких блоков SM, каждый из которых содержит:
 - Множество ядер (т.н. CUDA-ядра в GPU от NVIDIA).
 - Специализированные блоки для операций с плавающей запятой (FP32/FP64) и целыми числами.
 - Тензорные ядра (Tensor Cores) для ускорения матричных операций, используемых в машинном обучении.
 - RT-ядра (Ray Tracing Cores) для аппаратного ускорения трассировки лучей (в графических задачах).
- Warp-архитектура: Поток объединяется в группы (обычно 32 потока), которые выполняются синхронно. Это позволяет эффективно распределять вычисления.

3. Память: GPU имеет сложную иерархию памяти, оптимизированную для высокой пропускной способности:

- Регистры: Локальная память для каждого ядра, обладающая самой высокой скоростью доступа.
- Shared memory: Быстрая локальная память, общая для потоков одного блока, используется для обмена данными между потоками.
- Global memory: Основная память GPU, с более высокой задержкой, но большим объемом.
- Texture и Constant memory: Оптимизированы для специфичных задач, таких как доступ к двумерным данным или неизменяемым константам.
- Кэш-память: Современные GPU оснащены многоуровневой системой кэширования (L1, L2), что позволяет уменьшить задержки доступа к глобальной памяти.

4. Высокая пропускная способность (High Bandwidth):

- Современные GPU используют быстрые типы памяти, такие как GDDR6, HBM (High Bandwidth Memory), которые обеспечивают высокую пропускную способность (до сотен ГБ/с).

5. Аппаратное управление потоками:

- GPU поддерживает тысячи потоков одновременно благодаря аппаратному переключению между ними. Это позволяет скрывать задержки при обращении к памяти, обеспечивая высокую загрузку вычислительных блоков.

6. Скалируемость:

- GPU могут масштабироваться от компактных версий для ноутбуков до серверных моделей (например, NVIDIA A100), объединяющих несколько GPU через NVLink.

Программируемые компоненты:

Современные GPU поддерживают гибкую программируемость благодаря платформам:

- CUDA (от NVIDIA) — для программирования общего назначения.
- OpenCL — стандартный фреймворк для работы с GPU различных производителей.
- DirectX, Vulkan и OpenGL — графические API для работы с 3D-графикой.
- ROCm — платформа для GPU от AMD.

Особенности современных GPU:

1. Гетерогенные вычисления:

- GPU часто используются в связке с CPU (гетерогенные системы). CPU управляет задачами, требующими сложной логики, а GPU выполняет параллельные вычисления.

2. Акселерация AI:

- Специализированные блоки, такие как Tensor Cores, значительно ускоряют обучение нейронных сетей и выполнения инференса.

3. Ускорение графики:

- RT-ядра и специализированные технологии позволяют воспроизводить сложные графические сцены с реалистичными эффектами, такими как трассировка лучей.

4. Поддержка виртуализации:

- Современные GPU поддерживают виртуализацию, что делает их полезными для облачных вычислений.

36. Понятие occupancy в CUDA. Пример расчета

В CUDA термин «occupancy» (заполняемость) описывает степень использования вычислительных ресурсов графического процессора (GPU). Он определяется как отношение числа активных варпов (warps) на одном потоковом мультипроцессоре (SM) к максимальному числу варпов, которые могут быть одновременно активны на этом SM.

Для расчета **occupancy** (загруженности) GPU необходимы следующие параметры,

Параметры устройства (GPU)

1. Максимальное количество активных потоков на мультипроцессор (SM).
2. Максимальное количество активных блоков на мультипроцессор (SM).
3. Максимальное количество активных warp на мультипроцессор.
4. Общее количество регистров на мультипроцессор.
5. Размер единицы выделения регистров (Register Allocation Unit Size).
6. Объём разделяемой памяти (Shared Memory) на мультипроцессор.
7. Гранулярность выделения warp (Warp Allocation Granularity).
8. Гранулярность выделения регистров (Register Allocation Granularity).
9. Гранулярность выделения разделяемой памяти (Shared Memory Allocation Granularity).

Параметры ядра (Kernel)

1. Количество потоков в блоке (Threads per Block).
2. Число регистров, используемых каждым потоком (определяется аппаратно).
3. Объём разделяемой памяти (Shared Memory), используемый блоком.

Пример :

На устройстве с вычислительной мощностью (compute capability) 7.0 рассмотрим ядро с блоками из 128 потоков (128-thread blocks), где каждый поток использует 37 регистров.

Из раздела "**Физические ограничения для вычислительной мощности GPU**" (Physical Limits for GPU Compute Capability) известно, что максимальное количество активных warps (групп потоков) равно 64 для вычислительной мощности 7.0.

Количество регистров, необходимых для одного warp, определяется по формуле:

Необходимые регистры для одного warp = (число потоков в warp × число регистров на поток / 256) * 256

$$\left\lceil \frac{37 \times 32}{256} \right\rceil \times 256 = 1280$$

Максимальное количество активных warp на мультипроцессоре (SM) при заданной гранулярности выделения warp рассчитывается как:

Максимум активных warp = (Общее количество регистров / Необходимые регистры для одного warp / 4) * 4

$$\left\lfloor \frac{65536/1280}{4} \right\rfloor \times 4 = 48$$

Осцирапсу=Максимум активных warp/общее количество warp.

48/64 = 75%

37. Типы памяти в CUDA. Примеры создания и организации доступа

Глобальная память:

Расположение – DRAM GPU.

Предназначена для хранения данных, доступных всем нитям на устройстве. Высокая латентность доступа. Кешируется в уровнях L1 и L2. Оптимизация за счет совмещения запросов в рамках одного варпа (memory coalescing).

Примеры:

Динамически с хоста:

```
int *deviceArray;
cudaMalloc(&deviceArray, size);
cudaMemcpy(deviceArray, hostArray, size,
cudaMemcpyHostToDevice);
```

Статически:

```
__device__ int deviceArray[100];
```

Разделяемая (shared) память

Расположение – на том же чипе, что и ядра CUDA.

Объем ограничен, обычно до нескольких десятков килобайт. Используется для обмена данными между нитями одного блока. Низкая латентность. Каждому блоку нитей выделяется свой участок памяти. Память доступна только в рамках времени выполнения блока.

Пример:

```
__global__ void kernel() {
    __shared__ int sharedArray[256];
    sharedArray[threadIdx.x] = threadIdx.x;
    __syncthreads();
}
```

Константная память

Расположение – DRAM GPU, кешируется в read-only кеш Uniform Cache.

Объем до 64 КБ.

Хранение неизменяемых данных, общих для всех нитей.

Высокая производительность при однородных запросах.

Оптимизирована для передачи параметров в ядра.

Пример:

```
__constant__ float constData[256];
```

Копирование с хоста:

```
cudaMemcpyToSymbol(constData, hostData, sizeof(hostData));
```

Регистровая память

Расположение на мультипроцессоре.

Объем ограничен, делится между всеми активными нитями.

Хранение временных переменных для каждой нити. Самая быстрая память. Недоступна явно, управляется компилятором. При переполнении регистров используется локальная память.

Пример:

```
__global__ void kernel() {  
    int localVar = threadIdx.x; - размещается в  
    регистрах  
}
```

Локальная память

Расположение – DRAM GPU.

Хранение данных, специфичных для каждой нити, когда регистров недостаточно. Высокая латентность, но данные кешируются. Используется для массивов с неконстантными индексами или больших структур.

Пример:

```
__global__ void kernel() {  
    int largeArray[1024]; - размещено в локальной  
    памяти  
}
```

38. Механизм транзакций в CUDA. Пример

Транзакция – выполнение загрузки из глобальной памяти сплошного отрезка данных размерностью 128 байт с адреса кратного 128. Размер кэш-линии – 128 байт, на которые проецируется соответственно 128-байтный выравненный сегмент в глобальной памяти.

В случае если размер слова, запрашиваемого нитью, больше 4 байт, запрашиваемый варпом, блок данных разделяется на 128-байтные запросы, обрабатываемые независимо. Транзакции по 128 байт будут формироваться до тех пор, пока все нити варпа не получат необходимые данные.

```
cudaMalloc(&d_src, 1024*sizeof(int));
```

```
...
```

```
__global__ void kernel(int *src, int size) {
    offset = ... ;
    int a = src[offset];
    // действия
}
```

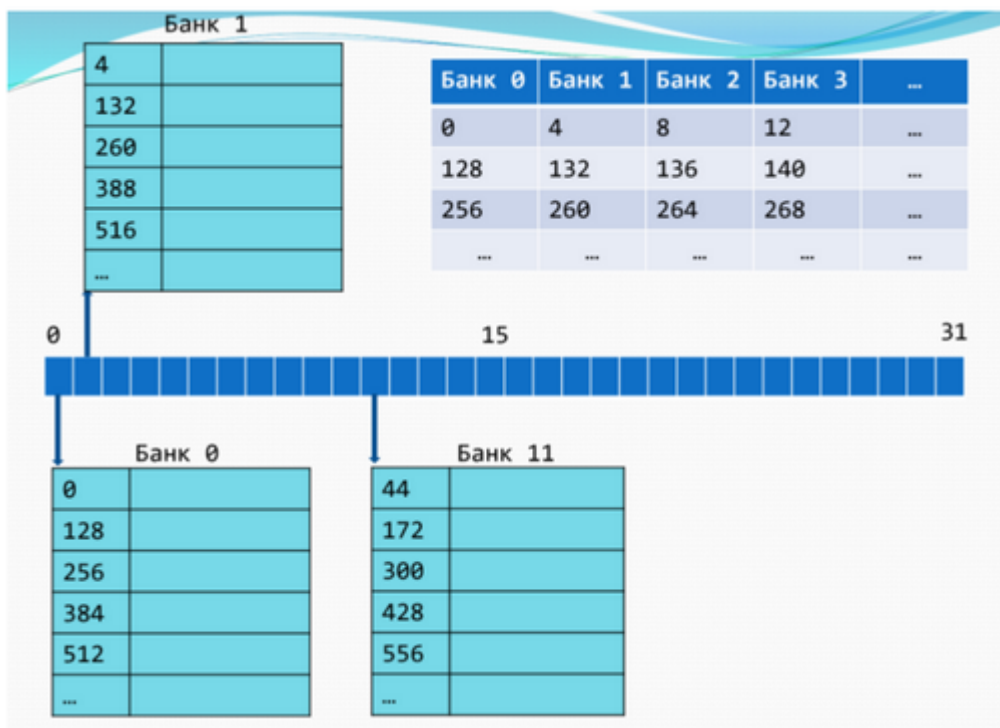
Если $\text{offset} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$,
транзакции +
Если $\text{offset} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x} * 3$,
транзакции $\times 3$

39. Конфликт по банкам в разделяемой памяти в CUDA. Пример

Для увеличения полосы пропускания блок, на котором расположена общая память, разделен на подмодули (банки).

Пусть n - число банков, m - количество последовательных байтов, которое один банк может отдать за 1 цикл.

Адресное пространство общей памяти разделено на n непересекающихся подмножеств, расположенных в разных банках. Банки работают независимо друг от друга и могут вместе выдать максимум $n \cdot m$ байтов за один цикл.



Если хотя бы два нужных слова расположены в одном банке, такая ситуация называется «банк конфликтом». Такое обращение аппаратно разбивается на серию обращений, не содержащих банк конфликтов. Если число обращений, на которое разбит исходный запрос, равно n , то такая ситуация называется банк-конфликтом порядка n . Пропускная способность при этом падает в n раз. Наиболее оптимальный способ обращения к разделяемой памяти - когда каждая нить в пределах варпа оперирует данными в одном банке.

Латентность доступа к глобальной памяти крайне высока, поэтому в правильно спроектированном алгоритме чтение и запись элемента данных должно осуществляться только один раз, при этом должны формироваться транзакции. Поэтому, как правило, ядро начинается с загрузки данных из глобальной в разделяемую память и заканчивается сохранением результата работы из разделяемой в глобальную память. Латентность доступа к разделяемой памяти в сравнении с глобальной гораздо ниже, однако она не нулевая. Поэтому при использовании одной нити значений из разделяемой памяти более одного раза может оказаться эффективной программная реализация дополнительного кэша, построенного на уровне регистров (локальных переменных). В данные переменные размещаются наиболее часто используемые данные и в дальнейшем могут использоваться для аккумуляции промежуточного результата.

Пример конфликта банков (ГПТ)

```
__shared__ float sharedData[1024];
__global__ void kernel(float *data, int n) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;

    // Каждый поток записывает в свою ячейку, но с шагом
1    sharedData[tid] = data[tid];

    __syncthreads();

    // Каждый поток считывает соседнее значение
    float sum = sharedData[tid] + sharedData[tid + 1];
    // ...
}
```

Более хороший код:

```
__shared__ float sharedData[1024];
__global__ void kernel(float *data, int n) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;

    // Каждый поток записывает в свою ячейку, но с
шагом, кратным
    // числу банков
    int bankSize = 32; // Предположим, что в банке 32
элемента
    sharedData[tid * bankSize] = data[tid];

    __syncthreads();

    // Каждый поток считывает значение из соседнего
банка
    float sum = sharedData[tid * bankSize] +
sharedData[(tid + 1) * bankSize];
    // ...
}
```

Во втором примере каждый поток обращается к разным банкам, что минимизирует конфликты.

40. Алгоритм редукции в CUDA. Пример

```
__global__ void reduce0(int* g_idata, int* g_odata) {
extern __shared__ int sdata[];
// each thread loads one element from global to shared
mem
unsigned int tid = threadIdx.x;
unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
sdata[tid] = g_idata[i];
__syncthreads();
// do reduction in shared mem
for (unsigned int s = 1; s < blockDim.x; s *= 2) {
if (tid % (2 * s) == 0) {
sdata[tid] += sdata[tid + s];
}
__syncthreads();
}
// write result for this block to global mem
if (tid == 0) g_odata[blockIdx.x] = sdata[0];
}
```

41. Алгоритм свертки в CUDA. Пример

```
__global__ void convolution2D(float *input, float *output,
float *kernel, int width, int height, int kernelWidth, int
kernelHeight) {
// Индексы текущего элемента
int x = blockIdx.x * blockDim.x + threadIdx.x;
int y = blockIdx.y * blockDim.y + threadIdx.y;

// Индексы в shared memory
int tx = threadIdx.x;
int ty = threadIdx.y;

// Размеры shared memory
extern __shared__ float sharedData[];

int sharedWidth = blockDim.x + kernelWidth - 1;
int sharedHeight = blockDim.y + kernelHeight - 1;

// Загрузка данных в shared memory
for (int i = ty; i < sharedHeight; i += blockDim.y) {
for (int j = tx; j < sharedWidth; j += blockDim.x) {
int globalX = blockIdx.x * blockDim.x + j -
kernelWidth / 2;
int globalY = blockIdx.y * blockDim.y + i -
kernelHeight / 2;

if (globalX >= 0 && globalX < width && globalY
>= 0 && globalY < height) {
sharedData[i * sharedWidth + j] =
input[globalY * width + globalX];
} else {
```

```

sharedData[i * sharedWidth + j] = 0.0f; //
Паддинг
    }
}
__syncthreads();

// Вычисление свертки
if (x < width && y < height) {
    float sum = 0.0f;
    for (int i = 0; i < kernelHeight; ++i) {
        for (int j = 0; j < kernelWidth; ++j) {
            sum += sharedData[(ty + i) *
sharedWidth + (tx + j)] *
                kernel[i * kernelWidth + j];
        }
    }
    output[y * width + x] = sum;
}
}

```

42. Алгоритм операции инклюзивного scan в CUDA. Пример

Базовый алгоритм последовательного scan (на CPU для понимания),
f - функция (например, сложение, min, max, произведение)

```

output[0] = input[0];
for(i = 1; i < N; ++i) {
    output[i] = f(output[i-1], input[i]);
}

```

Kogge-Stone Parallel (Inclusive) Scan Using Shared Memory Code

```

unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
__shared__ float buffer_s[BLOCK_DIM];
buffer_s[threadIdx.x] = input[i];
__syncthreads();
for(unsigned int stride = 1; stride <= BLOCK_DIM/2;
stride *= 2) {
    float v;
    if(threadIdx.x >= stride) {
        v = buffer_s[threadIdx.x - stride];
    }
    __syncthreads();
    if(threadIdx.x >= stride) {
        buffer_s[threadIdx.x] += v;
    }
    __syncthreads();
}
if(threadIdx.x == BLOCK_DIM - 1) {
    partialSums[blockIdx.x] = buffer_s[threadIdx.x];
}
output[i] = buffer_s[threadIdx.x];

```

43. Алгоритм операции эксклюзивного scan в CUDA. Пример

Базовый алгоритм последовательного scan (на CPU для понимания),
f - функция (например, сложение, min, max, произведение),
IDENTITY - обычно просто 0

```
output[0] = IDENTITY;
for(i = 1; i < N; ++i) {
    output[i] = f(output[i-1], input[i-1]);
}
Exclusive Scan Code

unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
__shared__ float buffer1_s[BLOCK_DIM];
__shared__ float buffer2_s[BLOCK_DIM];
float* inBuffer_s = buffer1_s;
float* outBuffer_s = buffer2_s;
if(threadIdx.x == 0) {
    inBuffer_s[threadIdx.x] = 0.0f;
}
else {
    inBuffer_s[threadIdx.x] = input[i - 1];
}
__syncthreads();

for(unsigned int stride = 1; stride <= BLOCK_DIM/2;
stride *= 2) {
    if(threadIdx.x >= stride) {
        outBuffer_s[threadIdx.x] =
inBuffer_s[threadIdx.x] + inBuffer_s[threadIdx.x - stride];
    }
    else {
        outBuffer_s[threadIdx.x] =
inBuffer_s[threadIdx.x];
    }
    __syncthreads();
    float* tmp = inBuffer_s;
    inBuffer_s = outBuffer_s;
    outBuffer_s = tmp;
}
if(threadIdx.x == BLOCK_DIM - 1) {
    partialSums[blockIdx.x] = inBuffer_s[threadIdx.x] +
input[i];
}
output[i] = inBuffer_s[threadIdx.x];
```

44. Асинхронное и синхронное копирование в CUDA. Pinned память. Способы выделения

Время выполнения задачи = время работы ядра + обмен данными между CPU и GPU

Для копирования памяти без участия ЦП применяется zero-copy операции. При этом неявно используется копирование памяти сначала в pinned буфер, потом в dram gpu. Разработчик может самостоятельно

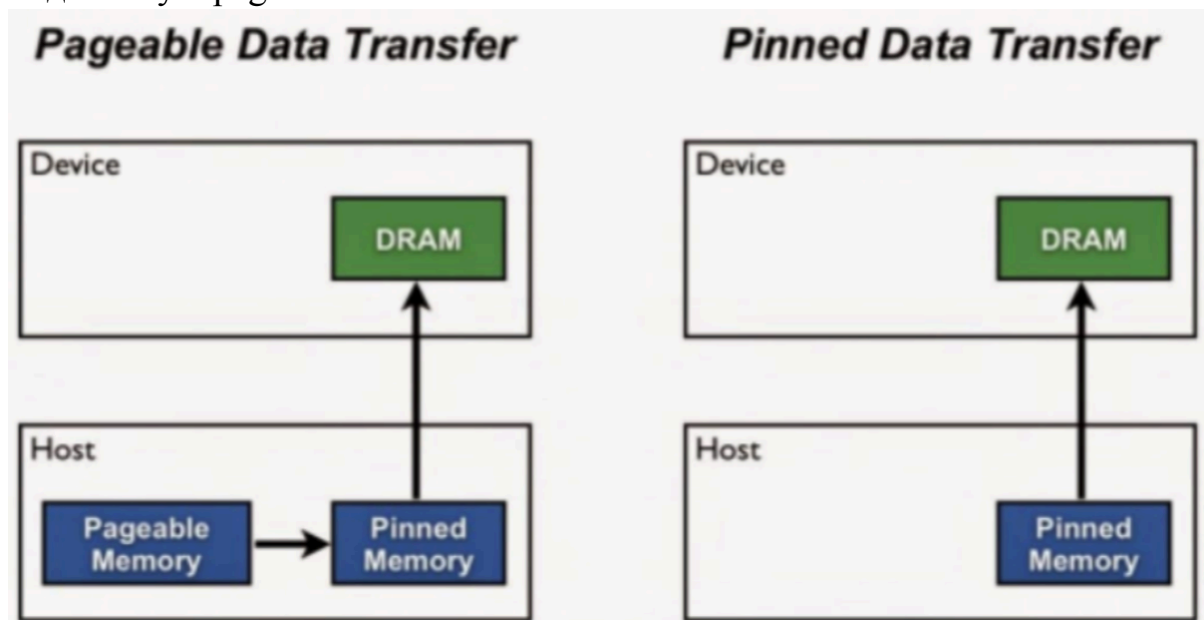
организовать логику получения копируемой на gpu pinned памяти, что позволяет ускорить процесс копирования.

Это проявляется при вызове асинхронных функций копирования памяти.

Основная разница во времени выполнения асинхронных и синхронных версий команд копирования проявляется при работе с pinned памятью в качестве входного параметра. (также влияет неявный вызов синхронизации в синхронной версии, поэтому и основная).

Pinned-память - page-locked память, т.е. страницы виртуальной памяти становятся жёстко привязанны к физической. С такой памятью возможны асинхронные операции с перемещением памяти host->dev/dev->host (смотреть таблцу).

Копирование такой памяти происходит быстрее, т.к. при копировании pageable памяти под капотом происходит перенос её содержимого в pinned буфер. С такой памятью работает dma, из-за чего и достигается мгновенное возвращение из функций асинхронного копирования. Её возможно не только выделять, но и делать таковой уже выделенную pageable память.



Когда возвращается управление хостовой нити

	Host->device		Device->host		host-host	dev-dev
	pageable	pinned	pageable	pinned		
memcpy	После копирования в буфер*	После полного завершения	После полного завершения	После полного завершения	После полного завершения	сразу
memcpyAsync	После копирования в буфер	сразу	После полного завершения	сразу	После полного завершения	сразу

*В начале работы неявно вызывается cudaDeviceSynchronize

45. CUDA Stream. Создание, инициализация и синхронизация

Технология *CUDA stream* позволяет при соблюдении ряда условий (как аппаратных, так и программных) организовать параллельное выполнение некоторых задач. Особенно удобно при работе с более чем одним *gri*. Последовательность команд для GPU (запуски ядер, копирования памяти и т.д.), исполняемая строго последовательно - следующая команда выполняется после полного завершения предыдущей. Пользователь сам создает потоки и распределяет команды по ним

Все команды, принимающие опциональный параметр типа *cudaStream* по-умолчанию используют поток по умолчанию (равный нулю).

Только команды из разных потоков, отличных от потока по-умолчанию, могут исполняться параллельно в зависимости от аппаратных возможностей. Пользователь сам задает необходимую синхронизацию между командами из разных потоков (при наличии зависимостей).

Возможные случаи:

- Параллельные копирование и выполнение ядра;
- Параллельные выполнение ядер;
- Параллельные копирования с хоста на устройство и с устройства на хост.

Возможности параллельного выполнения можно проверить при получении свойств устройства (структура *cudaDeviceProp*).

`cudaDeviceProp::asyncEngineCount > 0` - устройство может выполнять параллельно копирование и счет ядра (счёт ядра == выполнение ядра?)

`cudaDeviceProp::concurrentKernels > 0` устройство может выполнять ядра параллельно

`cudaDeviceProp::asyncEngineCount == 2` устройство может выполнять параллельно копирование в обе стороны и счет ядра

В зависимости от этих параметров имеет смысл планирование очередности вызова команд копирования и вызовов ядер.

Создание потока:

```
cudaStream_t stream;  
cudaStreamCreate(&stream);  
...  
cudaStreamDestroy(stream);
```

Поток привязывается к текущему активному устройству: перед отправлением команды нужно переключаться на устройство, к которому привязан поток, если попробовать отправить в него команду при другом активном устройстве, будет ошибка.

Инициализация stream'a выглядит следующим образом (все представленные команды будут относиться к одному stream'у):

```
cudaStream_t stream1;  
// создаем stream1  
cudaStreamCreate(&stream1);  
// асинхронное копирование на устройство
```

```

    cudaMemcpyAsync(d_a, a, N,
    cudaMemcpyHostToDevice, stream1);
    // запуск ядра
    kernelStream<<<1, N, 0, stream1>>>(d_a);
    ... // дальнейшая работа со stream1, CPU
    // разрушаем определенный stream1
    cudaStreamDestroy(stream1);

```

Для определения того, что использование CUDA Stream может обеспечить повышение производительности, на этапе инициализации рекомендуется выполнить следующие действия:

1. Проверить флаг `deviceOverlap` в структуре `cudaDeviceProp`. В соответствии с документацией все устройства с Compute Capability 1.1 и новее должны иметь этот флаг установлен в истинное значение.
2. Память на хосте должна быть определена как `pinned`.

Для *синхронизации*, например, со `stream1` введена функция `cudaStreamSynchronize(stream1)`, которая блокирует хост до тех пор, пока не будут полностью выполнены все команды для `stream1`. Для того чтобы узнать завершены команды `stream`'а или нет без блокировки хоста используется функция `cudaStreamQuery(stream)`.

Неявная синхронизация (ожидание завершения всех команд на устройстве) выполняется перед:

- Выделением `page-locked` памяти / памяти на устройстве;
- `cudaMemSet`;
- Копированием между пересекающимися областями памяти на устройстве;
- Отправкой команды в поток по-умолчанию;
- Переключением режима кеша L1.

Если между отправкой двух команд в разные потоки стоит что-то из этого списка – параллельного выполнения не будет.

Синхронизация на GPU:

```

cudaError_t cudaStreamWaitEvent(
    (cudaStream_t stream,      cudaEvent_t event,
    unsigned int flags) ).

```

Команды, отправленные в `stream`, начнут выполняться после наступления события `event`

— Синхронизация будет эффективно выполнена на GPU;

— При `stream == NULL` будут отложены все команды всех потоков.

Событие `event` может быть записано на другом GPU: - синхронизация между GPU.

Синхронизация по потоку:

```

cudaError_t cudaStreamQuery      (cudaStream_t
stream);

```

— Возвращает `cudaSuccess`, если выполнены все команды в потоке `stream`, иначе `cudaErrorNotReady`.

```

cudaError_t cudaStreamSynchronize (cudaStream_t
stream);

```

— Возвращает управление хостовой нити, когда завершится выполнение всех команд, отправленных в поток `stream`.

46. Микроархитектура Intel Knights Landing и ее наследники

17 июня 2013 года (релиз 20 июня 2016 года).

Техпроцесс – 14-нм Tri-gate.

Первая версия, работающая как самостоятельный процессор, а не сопроцессор. До 72 ядер с архитектурой, основанной на микроархитектуре Silvermont.

Структура Tile:

- Содержит 2 физических ядра с внеочередным выполнением
- 4-поточковый SMT (Simultaneous Multithreading) на ядро
- Каждое ядро оснащено 2 векторными ALU (VPU, Vision Processing

Unit)

- 1 МБ L2-кэша на Tile

2D Mesh Interconnect:

- Соединяет все tiles и контроллеры
- Использует протокол кэш-когерентности MESI

Поддерживаемые типы памяти:

- MCDRAM (Multichannel DRAM) – 16 ГБ (8x2 ГБ) с суммарной пропускной способностью 450 ГБ/с

- DDR4 – 6-канальная память с частотой до 2,4 ГГц, пропускная способность: 90 ГБ/с

Режимы работы памяти:

- Cache Mode: MCDRAM кэширует DDR
- Flat Mode: MCDRAM расширяет адресное пространство DDR
- Hybrid Mode: Гибридное использование

Наследники Knights Landing:

- Основаны на усовершенствованной 2D Mesh Interconnect
- Улучшена пропускная способность и энергоэффективность
- Используют более современные версии памяти (HBM или DDR5) и оптимизированные архитектуры кэша

47. Микроархитектура Intel Knights Mill

Структура тайла (плитки):

- 2 физических ядра на тайл
- Каждое ядро имеет по 2 векторных АЛУ (VPU)
- 1 МБ общей кэш-памяти L2
- Основано на микроархитектуре Silvermont (Atom)
- Поддержка 4-way SMT (многопоточности)
- Улучшенные буферы и пропускная способность

Системная архитектура (SoC):

- 36 тайлов, соединенных 2D сеткой
- Интегрированная память: 6 каналов DDR4-2400 (до 384 ГБ)
- 16 ГБ встроенной MCDRAM памяти
- 36 линий PCIe Gen3

- Поддержка AVX-512 с 512-битными SIMD операциями
- Уровни параллелизма:
- TLP (Thread Level Parallelism)
 - ILP (Instruction Level Parallelism) с 2-way суперскалярным исполнением
 - DLP (Data Level Parallelism) с поддержкой AVX-512
 - PLP (Pipeline Level Parallelism) с новыми Quad FMA инструкциями
 - Поддержка переменной точности (VNNI)
- Конвейер процессора включает:
- Блок выборки команд (Fetch)
 - Декодирование (Decode)
 - Планирование микроопераций (MS)
 - Распределение ресурсов (RAT/Alloc)
 - Исполнительные устройства (IEC, MEC, VPU)

48. Микроархитектура Intel Sunny Cove

Когда Sunny Cove была представлена, предполагалось, что она придёт на смену актуальной микроархитектуре Skylake и будет впервые реализована в продуктах семейства Ice Lake.

Микроархитектура Sunny Cove должна обеспечить прирост производительности процессоров на такт, а также повысить энергоэффективность в задачах общего назначения. Также производитель позаботился об ускорении своих будущих процессоров в задачах специального назначения, например, связанных с искусственным интеллектом или шифрованием.

Новая микроархитектура Sunny Cove получила значительные улучшения по части IPC во входной и исполнительных частях конвейера. Производитель отмечает возможность исполнения пяти инструкций за такт вместо четырёх, и увеличение числа исполнительных портов с 8 до 10, что обеспечит одновременную обработку до 10 микрокоманд. Также была увеличена пропускная способность кеша первого уровня за счёт добавления четвёртого блока генерации адресов и второго устройства, способного сохранять данные.

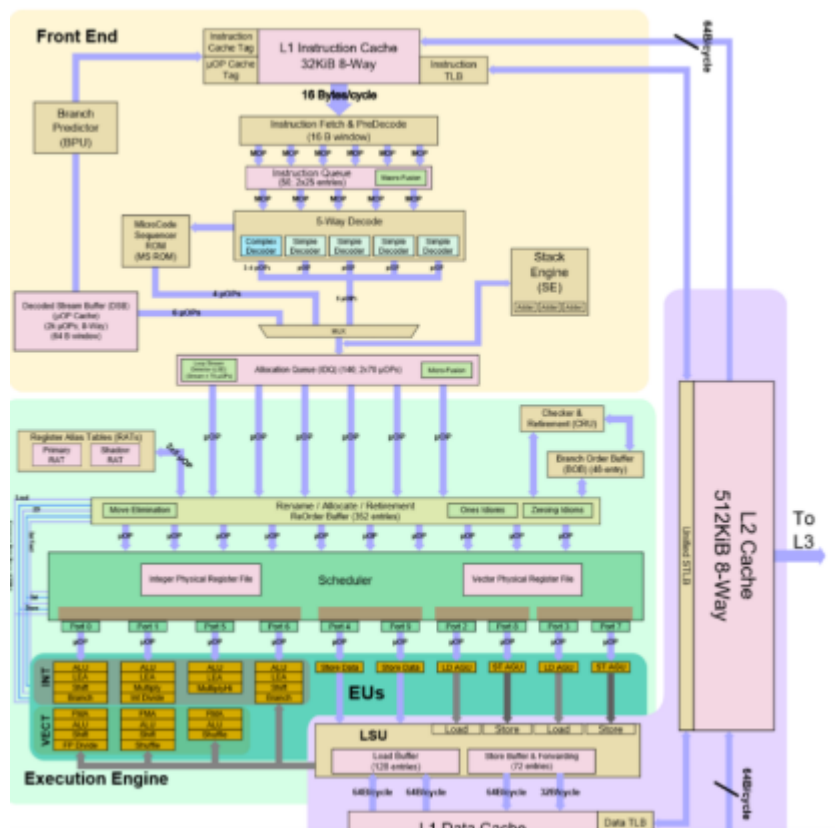
Наконец, отмечается повышение универсальности исполнительных портов. Например, для SIMD Shuffle и LEA стало вдвое больше путей исполнения.

В дополнение к этому отмечается, что Sunny Cove предложит увеличенный размер кешей для оптимизации рабочих нагрузок, подразумевающих обработку больших объёмов данных. В частности, на 50 % увеличится кеш первого уровня для инструкций, вырастет объём кеша микроопераций и унифицированного кеша второго уровня (насколько, будет зависеть от класса CPU), а также увеличится объём буфера ассоциативной трансляции (TLB) второго уровня.

В итоге с использованием Sunny Cove был выпущен только Intel Ice Lake. **Intel Ice Lake:**

- анонс: 2018 г.

- release date: 27.05.2019 (только мобильный сегмент)
- серверный сегмент: апрель 2021 г.
- особенности:
 - техпроцесс: 10 нм+
 - ядро: Sunny Cove
 - новые инструкции: AVX-512
 - Gaussian Neural Accelerator (GNA)
 - Image processing unit (IPU)
 - GPU Gen 11
 - DisplayPort 1.4a с Display Stream Compression
 - HDMI 2.0b



Хе-НРС (High Performance Compute (высокопроизводительные вычисления))

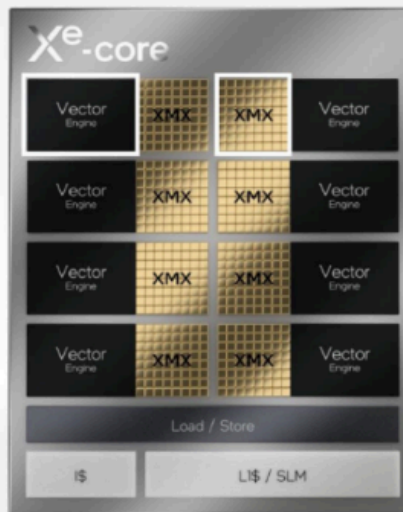
Intel Alchemist: TSMC N6, DirectX 12 Ultimate



Xe-HPC Core

Xe HPC Core

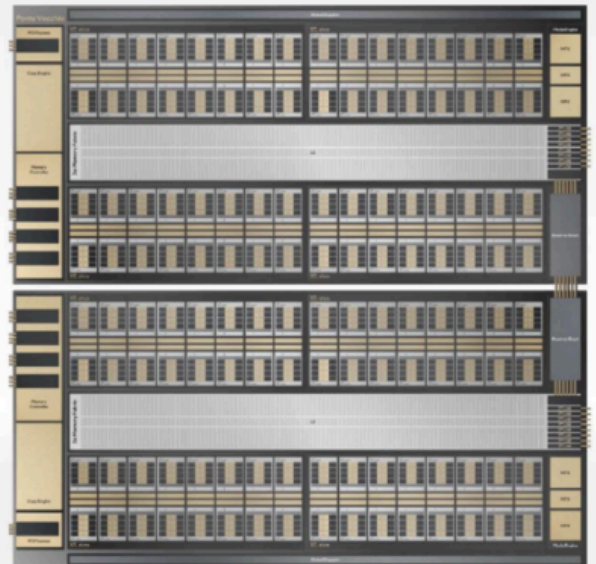
Vector Engine (ops/clock)
256 FP32
256 FP64
512 FP16



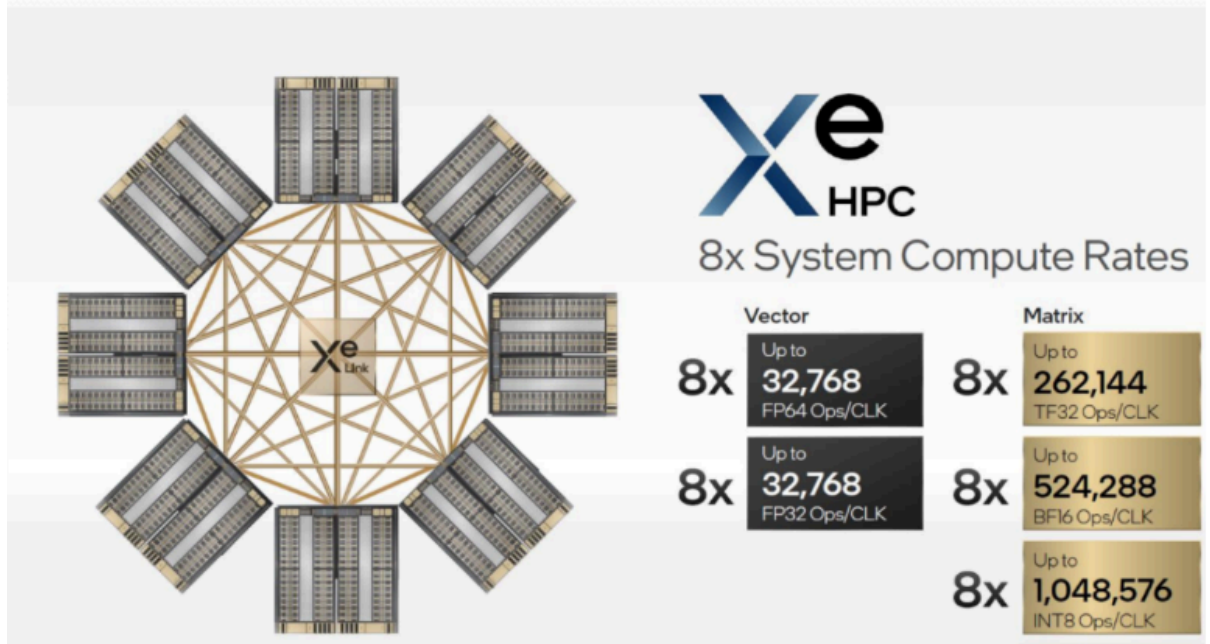
Matrix Engine (ops/clock)
2048 TF32
4096 FP16
4096 BF16
8192 INT8

Xe HPC 2 - Stack

8 Slices	2 Media Engines
128 Xe - cores	8 HBM2e controllers
128 Ray Tracing Units	16 Xe Links
8 Hardware Contexts	



For workloads and configurations visit www.intel.com/ArchDay21claims. Results may vary.



50. Микроархитектуры Intel Nehalem и Westmere

Микроархитектура: Nehalem

Кодовое имя:

- tock – Nehalem (август 2008 г. – март 2010 г.)

- tick – Westmere (январь 2010 г. – август 2011 г.)

release date: ноябрь 2008

особенности:

- техпроцесс: 45 нм

- транзисторы с диэлектриками high-k и металлическими затворами

(1 поколение)

- поддержка Hyper-Threading

- Intel QuickPath Interconnect

- Сменил Front Side Bus

- macro-op fusion в 64-битном режиме

- отсутствие серии Core i3

- оптимизация латентности ряда инструкций

Intel QuickPath Interconnect

- Реализовано во всех процессорах Intel

- Предшественник: Front-Side Bus

- Назначение:

- связь процессора с чипсетом

- связь процессоров друг с другом

- Характеристика:

- последовательная, высокоскоростная двунаправленная шина

Шина:

- 20 бит (20 отдельных пар линий), полный дуплекс

- 16 бит отводится для передачи данных

- 2 линии зарезервированы для передачи служебных сигналов

- 2 линии для передачи кодов коррекции ошибок CRC
- 2 пары линий, применяемых для синхронизации сигналов (одна на прием и одна на передачу)

- итого: 42 пары линий (84-контактная)

- независимый тактовый генератор в обоих направлениях

Пропускная способность шины:

- 3.2 GHz

× 2 bits/Hz (double data rate)

× 16(20) (data bits/QPI link width)

× 2 (unidirectional send and receive operating simultaneously)

÷ 8 (bits/byte)

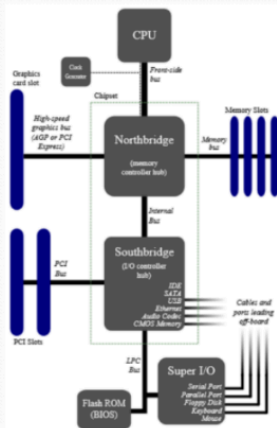
= 25.6 GB/s = 6,4 (GT/s) * 4

- Введено понятие:

«трансферы в секунду» - количество передач запакетированных данных по шине в секунду

Intel Front Side Bus

- Разработана в 1990-е (1980-е ?)
- Использовалась как связующая шина между CPU и чипсетом

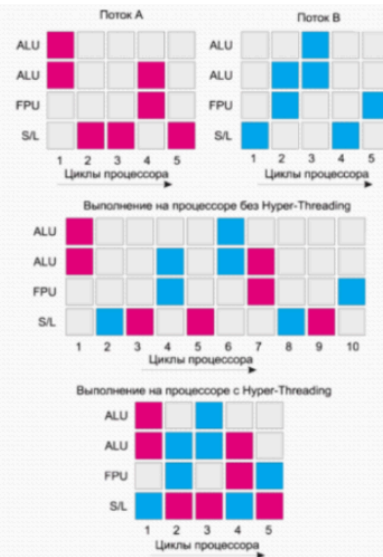


Simultaneous Multi-Threading (SMT)

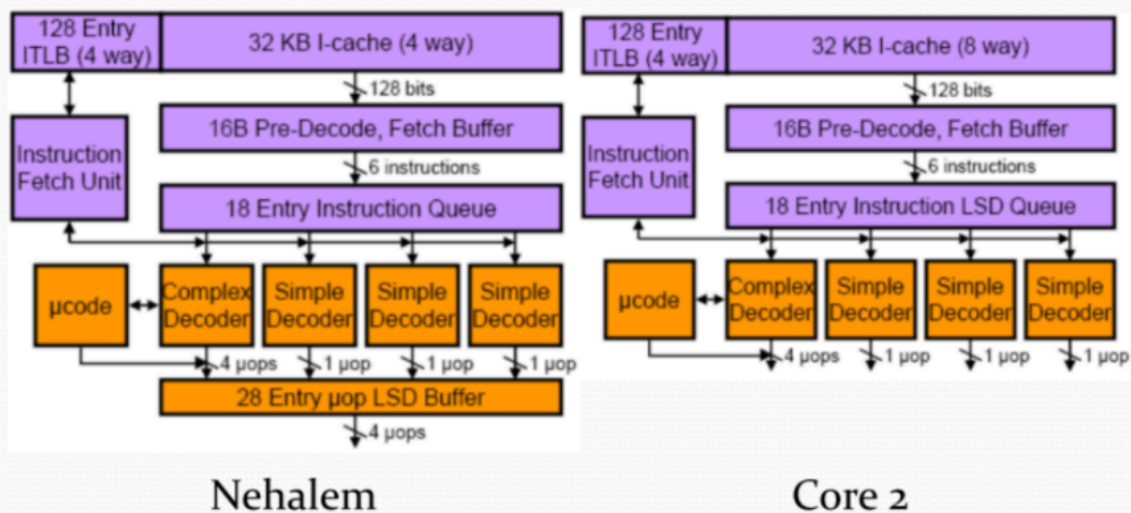


- Продублированы:
 - процессорные регистры
 - Return Stack Buffer

Simultaneous Multi-Threading (SMT)



Front-end: выборка инструкций



Front-end: выборка инструкций

- relative instruction point (RIP)
 - дублируется для каждого контекста нити
 - определяет следующую инструкцию для исполнения
- предсказание переходов
 - “best in class” © Intel
 - 2-уровневый BTB (Branch Target Buffer)
 - return stack buffer (RSB) сохраняет адрес возврата
- ITLB статически распределен между двумя потоками
 - 128 entries for 4KB pages
 - 4-ассоциативный кэш
- Кэш инструкций распределен между двумя потоками
- LSD (Loop Stream Detector) использовался в Core 2 для детектирования небольших циклов

Front-end: декодирование

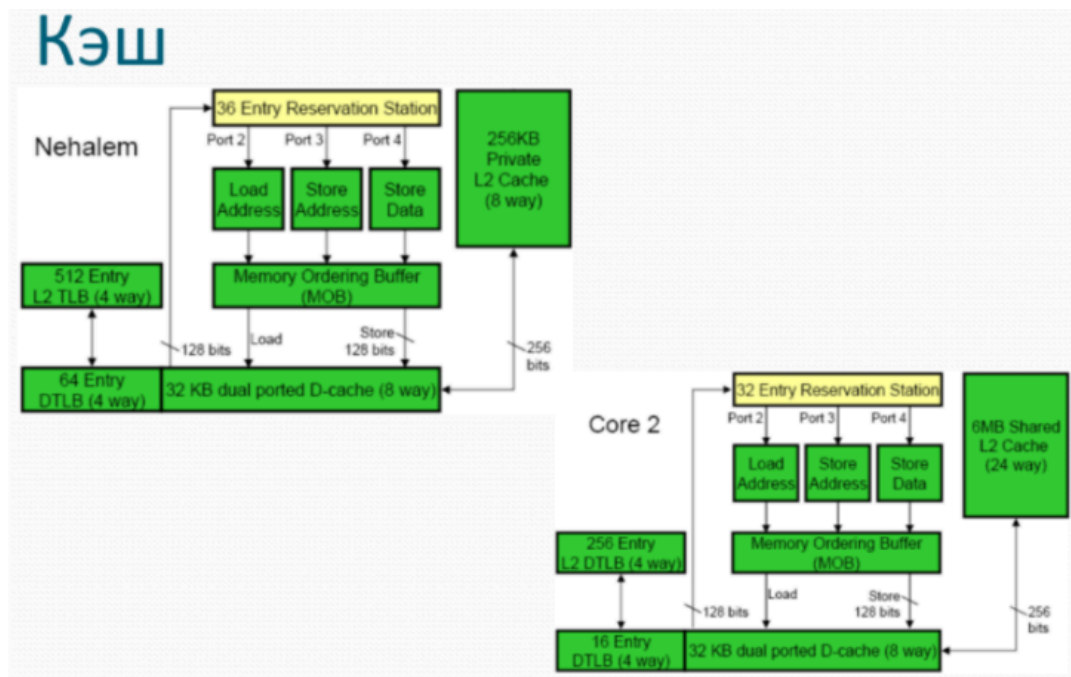
- Расширено число инструкций, декодируемых как 1 микрооперация
 - многие условные инструкции переходов
 - Core 2 только JCC, CMP, TEST
- macro-op fusion расширен на 64 битный режим

- расширено детектирование циклов:
 - число итераций менее 28 микроопераций?
 - да:
 - выполняется загрузка в LSD всех микроопераций цикла
 - внеочередное исполнение инструкций
- финальный этап декодирования:
 - dedicated stack engine
 - удаление всех микроопераций, модифицирующих стек
 - причина: увеличение числа микроопераций с возможностью внеочередного исполнения



Блок внеочередного исполнения

- Register alias table (RAT)
 - определяет архитектурный регистр в Re-Order Buffer (ROB) или Retirement
- Register File (RRF)
 - используется для поддержания механизма спекуляций
 - переименовывает до 4 инструкций за такт
- ROB и RS разделены между 2 потокам
 - при этом только ROB разделен статически



Кэш:

Увеличено число инструкций, которые могут выполняться параллельно:

- буфер загрузки – 48 инструкций
- буфер сохранения – 32 инструкции

Совмещенный для 2 потоков

распределение динамическое

Кэш

- L1D – 4 такта (в поколении Core 2 – 3 такта) (до 10 outstanding misses)
- L2 – менее 12 тактов (до 16 misses in-flight)

Инклюзивный кэш

Кэш L3:

- независимое питание
- независимый тактовый генератор
- минимальная латентность загрузки – 30-40 тактов
- наличие «core bit» - определяет ядро, которое может обрабатывать данные
- Когерентность кэша – протокол MESIF
- Выравненный и не выравненный доступ
 - латентность и пропускная способность инструкций одинакова

- условие: данные физически выравнены

Латентность кэша

Sandra 2009	Nehalem 3.2GHz	Penryn 3.2GHz
L1 Cache Latency	4 cycles	3 cycles
L2 Cache Latency	11 cycles	18 cycles
L3 Cache Latency	52 cycles	-

TLB

- полноценный 2-уровневый кэш
- динамическое распределение между контекстами потоков
- 1-й уровень DTLB
 - 64 строки для 4Kb страниц в памяти
 - 32 строки для 2M/4M страниц
- 2-й уровень TLB
 - общий для данных и инструкций
 - 512 строк для 4Kb страниц в памяти
 - не реализован для 2M/4M страниц
- позволяет транслировать до 9216 Кб памяти для все ядра процессора
 - достаточно, чтобы загрузить L3 кэш (8Mb)
 - проблема Core 2: транслирование 2176Кб памяти на 6Mb L2

Поддержка виртуализации:

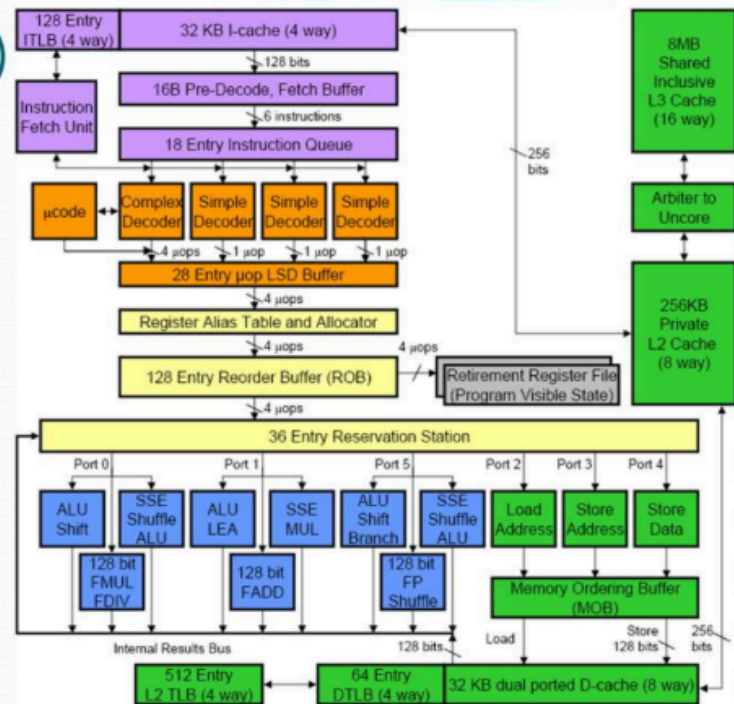
- Virtual Processor ID
 - процессы в реальной ОС и виртуальной обладают разными идентификаторами
- Extended Page Tables
 - трансляция физических адресов из виртуальной системы и в реальную

Intel Westmere - release date: январь 2010

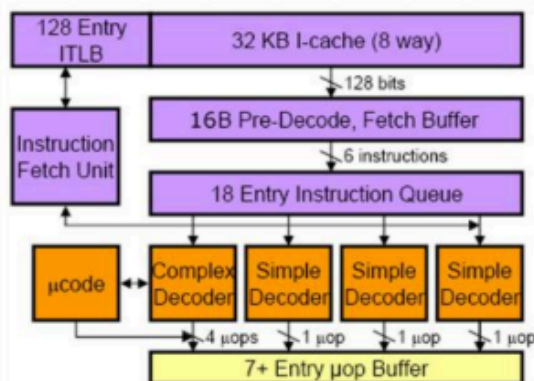
Особенности:

- техпроцесс: 32 нм
- транзисторы с диэлектриками high-k и
- металлическими затворами (2 поколение)
- расширенный набор инструкций: AES
- интегрированная графика

Nehalem (i7)

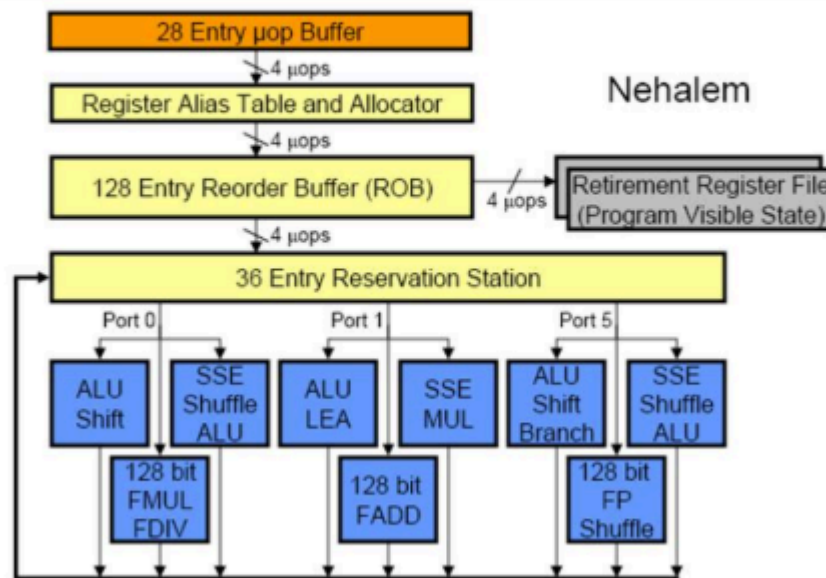


Выборка и декодирование инструкций Core 2



- Выборка из кэша инструкций производится с темпом 16 байт/такт.
- Выбранные данные попадают в буфер предварительного декодирования, в котором определяется размер инструкций.
- Выборка инструкций из этого буфера может производиться с максимальным темпом до 6 инструкций. Реальная скорость зависит от размера инструкций. Так для SSE инструкций и для x64, где средняя длина инструкции 4.5-5 байт, скорость выборки не будет превышать.
- Инструкции из буфера предварительного декодирования попадают в очередь инструкций. Максимум в очереди может храниться 18 инструкций. Размер буфера 64 байта, в среднем по 3.5 байта на инструкцию (средний размер инструкций x86). Максимальная скорость выборки из очереди 32 байта за такт.
- Буфер инструкций позволяет обнаруживать и сохранять циклы, которые не превосходят его по размеру. В этом случае инструкции будут выбираться только из очереди (не обращаясь кэш) с максимальным темпом.
- В очереди инструкций выполняется склейка инструкций. Склеиваются инструкции test, cmp со следующей за ними инструкцией условного перехода. Склейка позволяет поднять скорость декодирования в среднем на 10 %. Не работает для x64.

Вычислительное ядро Nehalem



Поддержка технологии SMT Nehalem

- Планирование инструкций выполняются по очереди. Один такт первый тред, второй такт другой тред. Инструкции для каждого тредя выбираются из отдельных LSD буферов.
- Для поддержки SMT в ядре был модифицирован ROB. ROB разделяется между двумя тредями поровну, по 64 ячейки.
- Станции резервирования являются общими для двух тредов и разделяются динамически.
- Скорость отставки для двух тредов не превышает 4 микроопераций за такт.
- Архитектурные регистры являются отдельными для каждого тредя

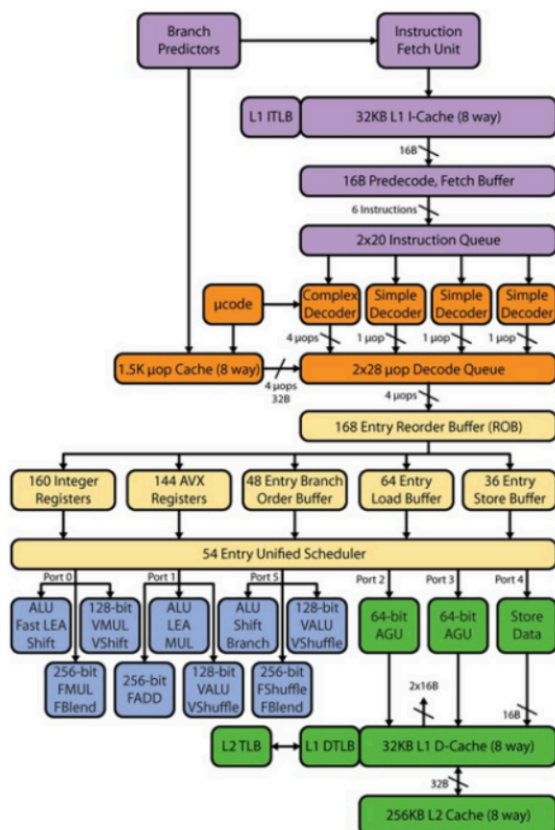
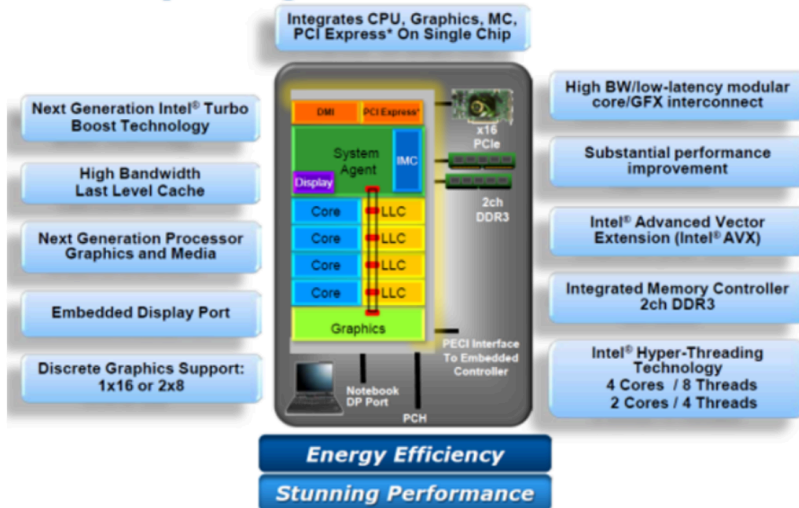
51. Микроархитектуры Intel Sandy Bridge и Ivy Bridge

Sandy Bridge

1. **Процессор:** Четырёхъядерный 64-битный процессор.
2. **Исполнение команд:** Изменяемая (out-of-order) последовательность исполнения команд.
3. **Потоки данных:** Поддержка двух потоков на ядро (**Hyper Threading**).
4. **Исполнение команд:** Выполнение четырёх команд за такт.
5. **Графика:** Интегрированное графическое ядро.
6. **Контроллер памяти:** Интегрированный контроллер памяти **DDR3**.

7. **Кольцевая шина:** Поддержка новой кольцевой шины.
8. **AVX:** Поддержка 3- и 4-операндных 128/256-битных векторных команд **AVX**.
9. **Техпроцесс:** Производство по **32-нм** технологии.

Sandy Bridge: Overview



Ivy Bridge

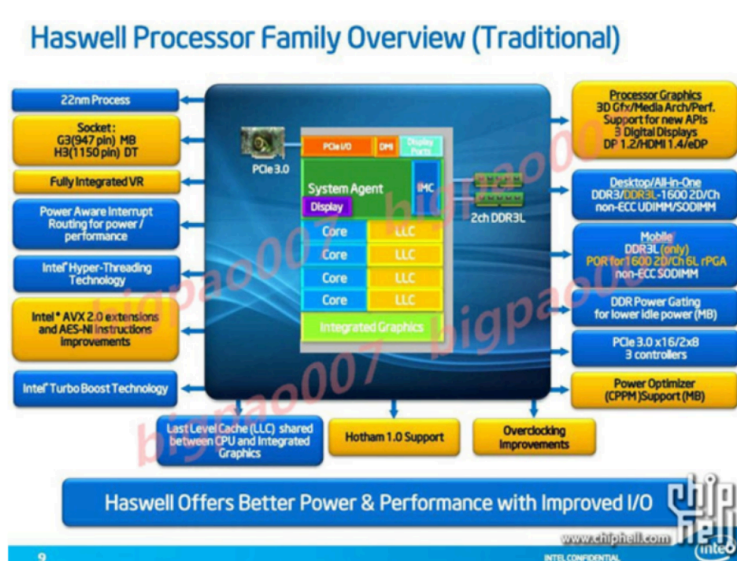
1. **Техпроцесс:** 22-нм, улучшение производительности и снижение энергопотребления.
2. **Графическое ядро:** 16 исполнительных блоков (EU), поддержка DirectX 11.

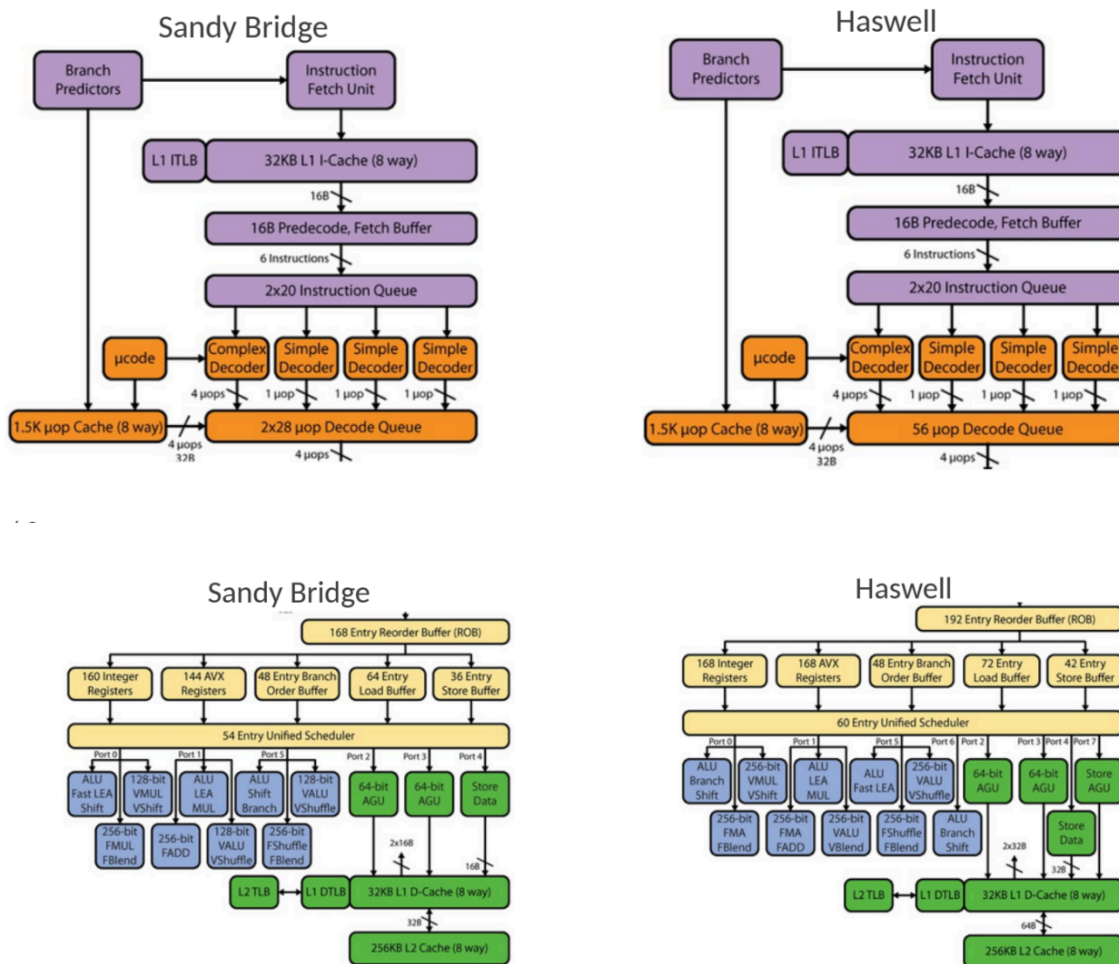
3. **Кольцевая шина:** Ring Interconnect для объединения ядер, графического ядра и системного агента через общий кэш L3.
4. **Память:** Поддержка DDR3 до 2800 MT/s и DDR3L.
5. **PCIe 3.0:** Встроенный контроллер PCIe 3.0 (кроме i3).
6. **USB 3.0:** Поддержка USB 3.0 в чипсетах 7 серии (4 порта).
7. **Thunderbolt:** Встроенная поддержка Thunderbolt.
8. **Энергосбережение:** Конфигурируемое TDP, режим пониженного энергопотребления.
9. **Генератор случайных чисел:** Аппаратный генератор, поддержка ANSI X9.82, NIST SP 800-90.
10. **Видео:** Интегрированное GPU с поддержкой HDMI 1.4a и до 3 мониторов.

52. Микроархитектуры Intel Haswell и Broadwell

Intel Haswell:

1. **Техпроцесс:** 22-нм.
2. **Энергопотребление:** Снижено по сравнению с Ivy Bridge.
3. **Поддержка AVX2:** Улучшенная производительность для векторных вычислений.
4. **Графика:** Десктопные модели могут иметь встроенную видеокарту
5. **Turbo Boost 2.0:** Динамическое увеличение тактовой частоты.
6. **Поддержка памяти:** DDR3 и DDR3L наследовано от Ivy Bridge.
7. **Год анонса:** 2013.





Intel Broadwell:

1. **Техпроцесс:** 14-нм.
2. **Энергопотребление:** Значительно снижено по сравнению с Haswell.
3. **Поддержка AVX2:** Сохраняется поддержка для векторных вычислений.
4. **Графика:** Десктопные модели могут иметь встроенную видеокарту
5. **Кэш:** Улучшена система кэширования для снижения задержек.
6. **Turbo Boost 2.0:** Поддержка динамического увеличения частоты.
7. **Поддержка памяти:** DDR3L и LPDDR3 с улучшенной пропускной способностью наследовано от Ivy Bridge.
8. **Год анонса:** 2014.

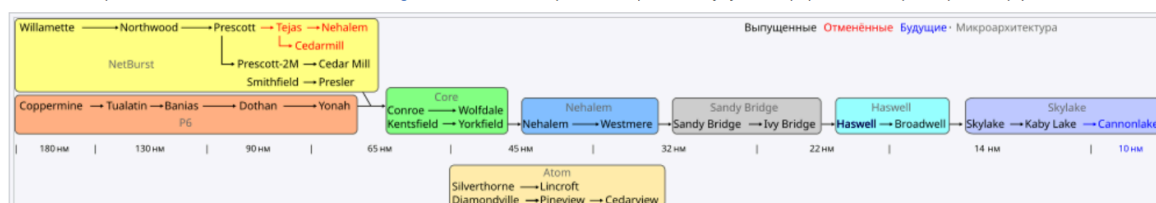
Стадий конвеера - 14.

FinFET транзисторы с трехмерной конструкцией (Tri-Gate)

Transactional Synchronization Extensions (TSX)

Broadwell по сути копия Haswell под новый техпроцесс.

Хронология до Haswell



53. Микроархитектуры Intel Skylake и его наследники

Микроархитектура: Skylake

Кодовое имя: tock – Skylake, tick – Kaby Lake

Intel Skylake

анонс: август 2015

особенности:

- универсальная микроархитектура
- техпроцесс: 14 нм
- FinFET с трехмерной конструкцией (Tri-Gate)
- изменения чипсета: eMMC, SDXC, CSI
- сигнальный процессор для обработки изображений
- усовершенствованная модульная микроархитектура
- поддержка стандарта DDR4
- AVX-512 для серверов

Intel Skylake Microarchitecture

- Увеличенная пропускная способность ввода-вывода чипсета, улучшенные интерфейсы для планшетов, обновленный аудио DSP, концентратор датчиков
- Дисплей с более высоким разрешением
- Увеличенное и расширенное ядро, улучшенная производительность инструкций за такт, повышенная энергоэффективность
- Улучшенная кольцевая шина/LLC для повышения пропускной способности
- Интегрированный процессор обработки изображений (ISP) для камеры
- Расширенные возможности разгона
- Более быстрая память DDR
- Продвинутое графика процессора GT3 + eDRAM, GT4 + eDRAM; поддержка OpenCL™ 2.0 API, DirectX 12, OpenGL 4.4

Усовершенствования конвейера

- сохранено одновременное декодирование до шести x86 CISC-инструкций
 - усовершенствована система выборки инструкций
 - блок предсказания ветвлений

	Sandy Bridge	Haswell	Skylake
Out-of-order Window	168	192	224
In-flight Loads	64	72	72
In-flight Stores	36	42	56
Scheduler Entries	54	60	97
Integer Register File	160	168	180

FP Register File	144	168	168
Allocation Queue	28\thread	56	64\thread

SKL Core Microarchitecture at a Glance

Оптимизация сегмента

- Специальные конфигурации IP для сервера и клиента

Улучшенный фронтенд

- Повышенная производительность и улучшенный предсказатель ветвлений
- Широкая подача инструкций с более глубокими буферами
- Более быстрая предварительная выборка (prefetch)

Более глубокие Out-of-Order буферы

- Извлечение большего параллелизма инструкций

Улучшенные исполнительные блоки

- Сокращенные задержки
- Увеличенное количество блоков
- Отключение питания при простое
- Ускорение AES-GCM и AES-CBC на 17% и 33% соответственно

Повышенная пропускная способность загрузки/хранения данных

- Улучшения в предварительной выборке
- Более глубокие буферы записи, заполнения и обратной записи
- Улучшенная обработка пропущенных страниц
- Лучшая пропускная способность при промахх кэша L2
- Новые инструкции для улучшенного управления кэшем

Улучшенный Hyper-Threading

- Расширение механизмов завершения инструкций

QPI 2.0

двойной режим работы:

- старая версия
- с удвоенной пропускной способностью

eDRAM-буфер

- новое название - Memory Side Cache
- исключено взаимодействие с кэшем
- снята необходимость в поддержке когерентности
- кешируются любые данные, в т.ч. некешируемые

Сигнальный процессор для обработки изображений

Полное решение для обработки изображений и работы с камерой

- Полная интеграция оборудования и программного обеспечения
 - Оптимизированная поддержка модулей и сенсоров: поддержка до 4 камер (2 одновременно), сенсоры до 13 МП
 - Система с премиальным качеством обработки изображений, оптимизированная по компонентам (ISP + CPU + GPU + дисплей)
 - Сотрудничество с экосистемой сенсоров для ускорения вывода на рынок (TTM), снижения затрат на разработку и уменьшения размеров устройств
- Интегрированный процессор обработки изображений (ISP) для большей эффективности

- Снижение стоимости материалов (BOM)
- Уменьшение размера модуля
- Поддержка более компактных и тонких форм-факторов
- Оптимизация энергопотребления
 - Поддержка передовых функций обработки изображений и работы с камерой

Энергоэффективность

- убран интегрированный преобразователь питания для CPU
- увеличено число энергетических доменов
 - 256-битные исполнительные устройства, отвечающие за исполнение AVX⁵¹²-команд
- технология Speed Shift

Переходы между режимами сна

- переключение частоты между номинальным состоянием и турборежимом осуществляет CPU
- переход в экономичные состояния:
 - ОС обращается к микропрограмме и выясняет поддерживаемые режимы
 - ОС принимает решение и переключает режим на CPU
- восстановление:
 - CPU предварительно информирует ОС об изменении условий
 - ОС обрабатывает запрос
 - ОС передает команду на переключение

Intel Speed Shift

- Skylake реализует алгоритм P-state на основе спроса с обнаружением отклика
 - Операционная система задает минимальные/максимальные значения и предпочтение мощности или производительности → SoC определяет рабочую точку P-state для оптимального баланса между энергопотреблением и производительностью
- Технология Intel® Speed Shift обеспечивает повышенную отзывчивость системы для коротких операций
- Процессор экономит энергию, самостоятельно приостанавливая выполнение операций на короткие промежутки времени
 - В сценариях ограничения мощности, например, при полной загрузке всех ядер, процессоры с низким TDP не могут поддерживать высокую частоту. Более того, они опускаются ниже самой эффективной частоты.
 - Механизм Duty Cycling для SoC позволяет ускориться до наиболее эффективной частоты, таким образом восстанавливая утраченную производительность и одновременно экономя энергию благодаря более эффективной точке выполнения.

Графическое ядро Skylake

- построено на базе одного или нескольких модулей (по три секции каждое)

- внемодульная часть вынесена теперь в отдельный энергетический домен с собственной частотой
- поддержка до 3 модулей (72 исполнительных устройства)
- H.265 / HEVC
- OpenCL 2.0

9-е поколение GPU

Графика процессоров Intel® Core™ 6-го поколения на 14-нм техпроцессе Добавляет поддержку новейших API:

- DirectX 12/11.3
- OpenCL™ 2.0
- OpenGL 4.4

Сохраняет схожую архитектурную сегментацию с архитектурой Broadwell:

- Unsliced, slice, subslice и т. д.

Модульная организация

- Similar subslice configuration to Gen8
 - 24 EU slice
 - 8:1 EU sampler ratio
- EU simplified to "scalar" mode
 - New compiler stack to support
- Mixed float16/float32 support
 - Float16 denormals
- Sampler improvements
 - YUV formats, performance
- Bindless resources, tiled resources

Безопасное сжатие целевого рендеринга (Lossless Render Target Compression)

- Сжатие данных перед отправкой в память
- Экономия пропускной способности и энергии DDR
- Гранулярность на уровне строк кэша
- Максимальное отношение сжатия: 2:1
- Улучшение производительности: 3-11%
- Поддерживается всеми SKU, начиная с Gen9

Проблемы Skylake

проект GIMPS (Great Internet Mersenne Prime Search)

- зависание системы под высокими вычислительными нагрузками
- затрагивает операционные системы Windows и Linux
- сбой вызывает определённое значение экспоненты (14 942 209)

Проблемы Skylake-U

Полный контроль системы с помощью интерфейса USB 3.0

- © Positive Technologies
- проблемы в отладочном интерфейсе через DCI (Direct Connect Interface)

Intel Kaby Lake

- release date: 30 августа 2016
- Kaby Lake R – 21 августа 2017 для устройств пониженной мощности
- Whiskey Lake (4-й кв. 2018) – мобильный сегмент с ультра-низким энергопотреблением (15 Вт)
- Amber Lake (4-й кв. 2018) – мобильный сегмент с ультра-низким энергопотреблением (5-7 Вт)
- особенности:
- техпроцесс: 14 нм
- Усовершенствованный FinFET с трехмерной конструкцией (Tri-Gate)
- USB 3.1 / Thunderbolt 3
- Intel Optane (3D Xpoint)

Intel Coffee Lake

- release date: 5 октября 2017 г.
- особенности:
 - техпроцесс: 14 нм++
 - сегмент: десктоп / мобильный решения
 - 4-6 ядер
 - L3 кэш – 12 Мб
 - GPU Gen 9,5
- чипсет:
 - USB 3.1 (10 Gib/s)
 - Intel wireless controller (IEEE 802.11ac)
 - SDXC 3.0
- основа – архитектура Skylake

Intel Comet Lake

- release date:
 - мобильный сегмент: 21.08.2019
 - desktop: 30.04.2020
- тех.процесс: 14 нм
- отличия от Kaby Lake:
 - +25% ядер (до 10)
 - +25% LLC (до 20 MiB)
- чипсет:
 - 2.5G Ethernet
 - WiFi 6 AX201 (GiG+)

Intel Rocket Lake

- release date: 16.03.2021 (desktop/server сегмент)
- особенности:
 - техпроцесс: 14 нм++
 - ядро: Cypress Cove
 - PCIE 4.0
 - GPU: Gen 12 (XE)
 - DisplayPort 1.4a
 - HDMI 2.0b

54. Микроархитектура Intel Alder Lake

Анонс: ноябрь 2021 года;

Тех. процесс: Intel 7 (10nm ESF);

Ядра:

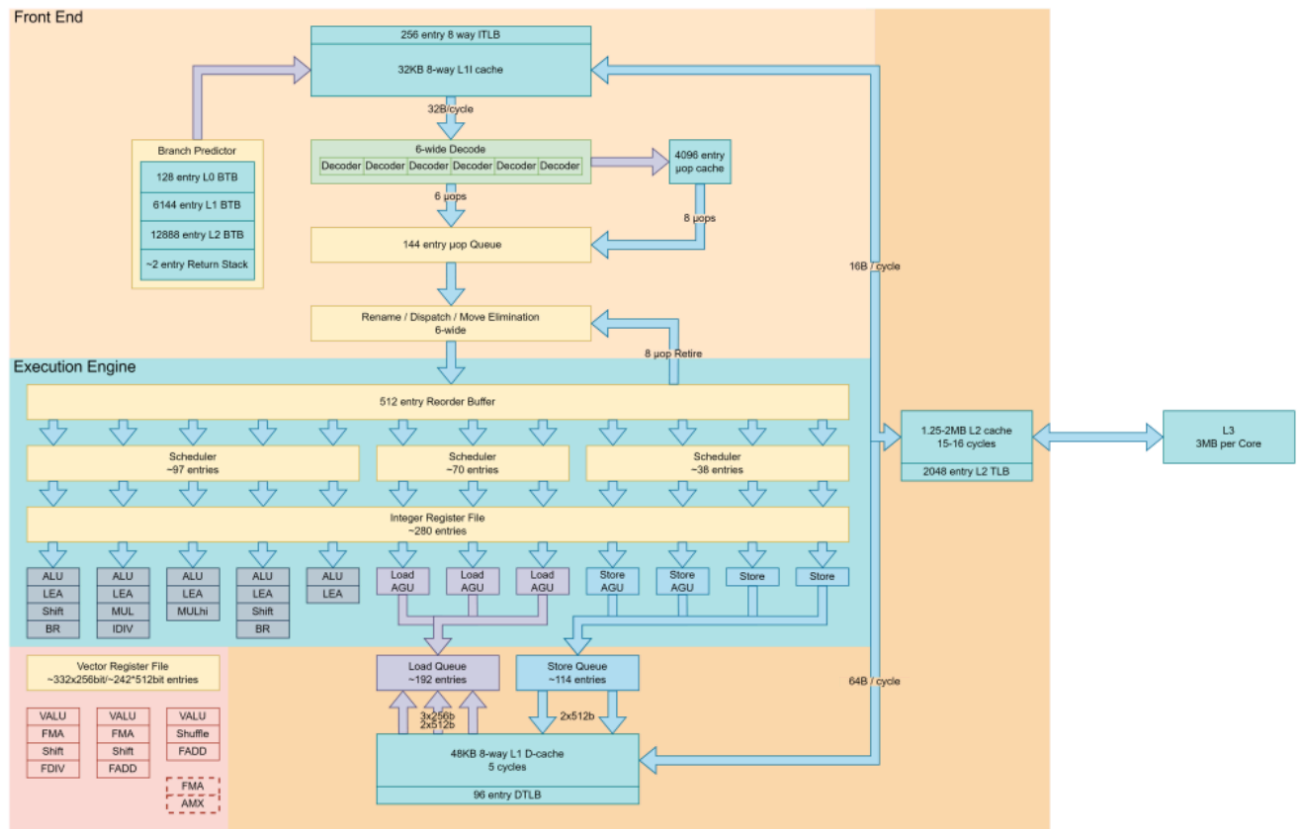
— Golden Cove (performance core, до 8);

— Gracemont (efficiency core, до 8);

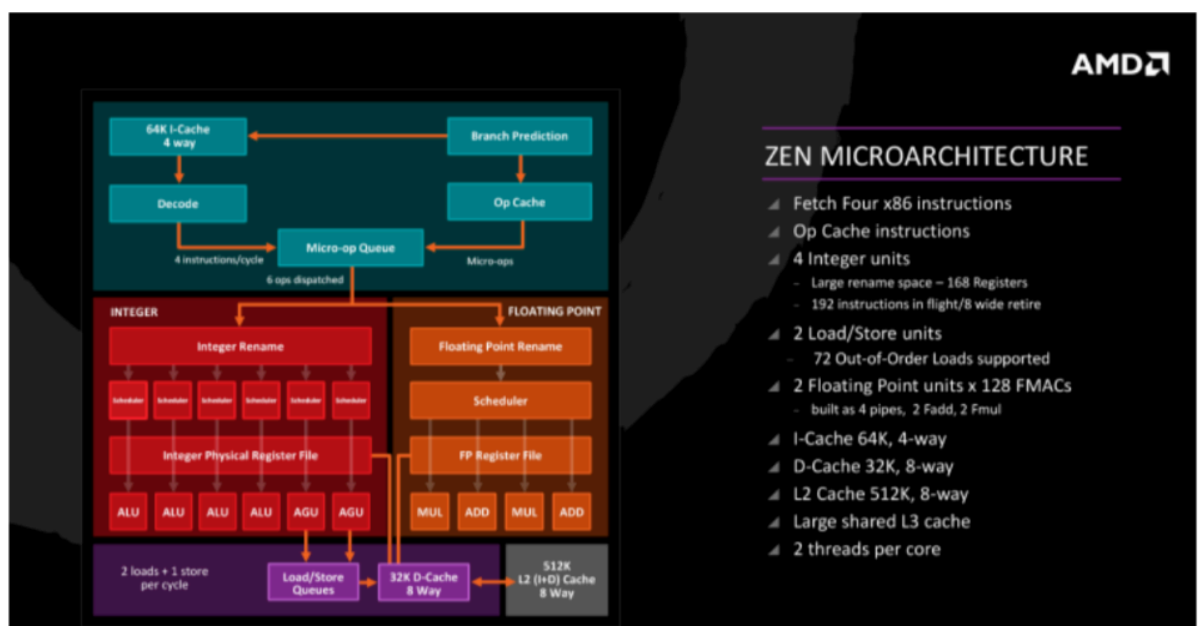
DDR5;

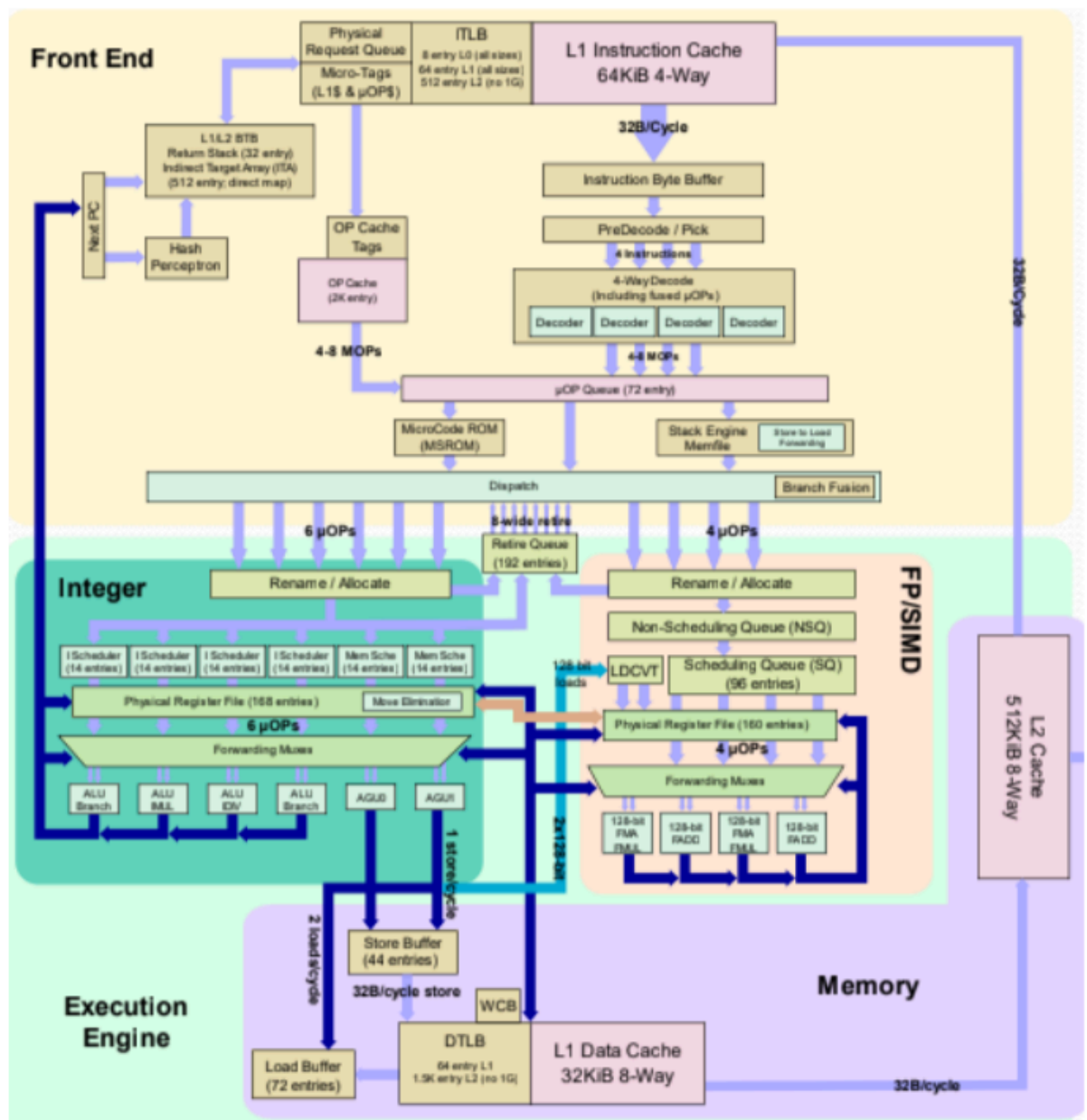
Intel Xe-LP.

Golden Cove



55. Микроархитектура AMD Zen





- классическая «широкая» микроархитектура
 - увеличение IPC без увеличения потребляемой мощности
- единая микроархитектура для всего рынка ЦП
- технология AMD SenseMI
 - AMD Pure Power
 - AMD Precision Boost
 - AMD XFR (eXtended Frequency Range)
 - Neural Net Prediction
 - Smart Prefetch
- стандарт DDR4 (до 2/8 каналов)
 - HBM в APU - так и не была реализована
- полностью независимые ядра
 - поддержка SMT
 - кэш микроопераций
 - write-back L1 кэш
 - кэш L2
 - собственный блок FPU
- целочисленный конвейер - 19 стадий

56. Микроархитектура AMD Zen 2 и наследников

Поколение	год	Техпроцесс	IPC прирост	Максимальный L3-кэш на CCD	PCIe	Память
Zen 2	2019	7-нм	+15%	32 МБ	4.0	DDR4
Zen 3	2020	7-нм	+19%	32 МБ	4.0	DDR4
Zen 4	2022	5-нм	+10-15%	32 МБ	5.0	DDR5
Zen 5	2024	4-нм (ожд.)	-	-	5.0+	DDR5

Zen 2 предлагает значительное улучшение по сравнению с Zen+ в плане производительности на такт (IPC, Instructions Per Cycle):

Улучшения ядра

- В Zen 2 каждый комплекс ядер (CCX, Core Complex) имеет до 32 МБ L3-кэша (в 2 раза больше по сравнению с Zen и Zen+).
- **Кэш микроопераций** расширен до **4000 инструкций (4K)**, что уменьшает зависимость от декодирования инструкций и повышает производительность в повторяющихся циклах и часто используемых блоках кода.
- Каждое ядро поддерживает **2 потока** благодаря технологии SMT, которая была перенесена из Zen и Zen+.
- Улучшенный механизм предсказания ветвлений **TAGE (Tagged Geometric History Length)** и предвыборки данных позволяет ядру загружать данные быстрее и точнее.
- **Широкий передний конец:** Zen 2 может декодировать до 4 инструкций за такт, улучшая производительность при обработке параллельных инструкций.

ALU и FPU

- Имеется **4 целочисленных блока (ALU)** для выполнения арифметических и логических операций.
- Расширенное пространство переименования регистров — **180 регистров**, что позволяет лучше оптимизировать операции и уменьшать задержки.
- Увеличено количество блоков **AGU (Address Generation Units)** с 2 до 3, что улучшает обработку адресов памяти.
- Процессор может выполнять **2 загрузки и 1 запись за такт**
- Имеется 2 блока FPU, каждый из которых поддерживает 256-битные операции FMA (Fused Multiply-Add).

FPU реализован как 4 отдельных канала:

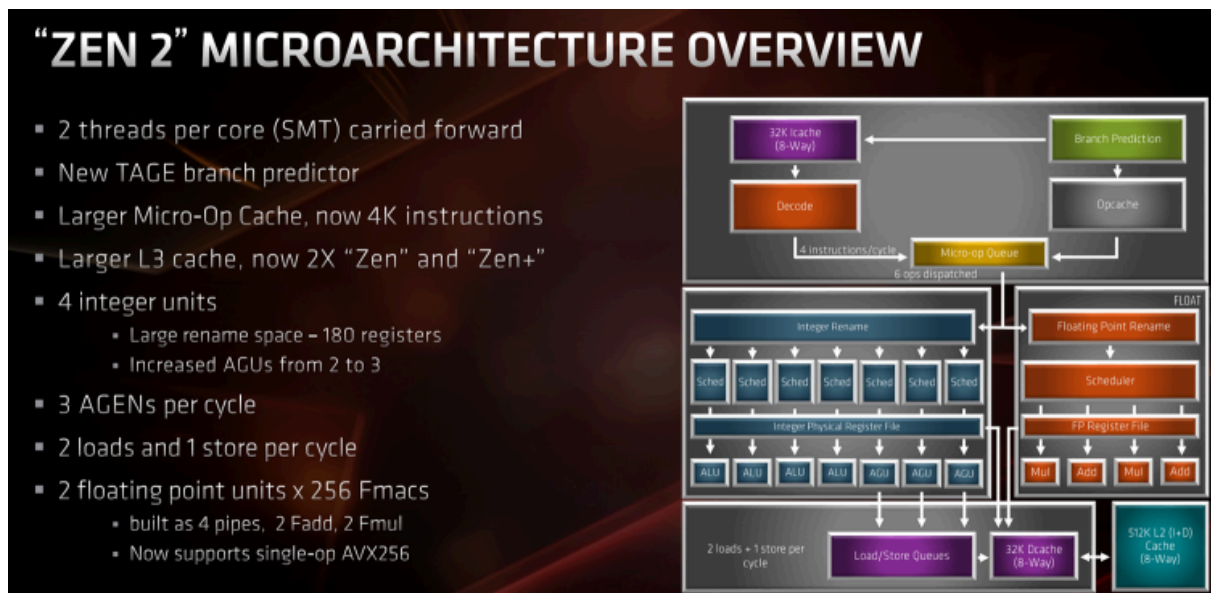
- 2 для операций сложения (Add).
- 2 для операций умножения (Mul).

Поддерживается **AVX256**, что удваивает производительность для задач, связанных с обработкой векторных данных

- **256-битный FPU:** FPU в Zen 2 поддерживает выполнение до 2 AVX2-инструкций или одной 256-битной инструкции за такт, что удваивает производительность в задачах с интенсивным использованием векторных вычислений.

Архитектура обеспечивает улучшенную энергоэффективность и повышенную пропускную способность за счёт увеличения скорости работы кэшей и блоков исполнительных устройств.

Интеграция с шиной Infinity Fabric позволяет эффективно связывать вычислительные чиплеты и чип ввода-вывода (IOD).



57. Микроархитектура семейства видеокарт AMD Radeon 300

кодовое имя	«Fiji»	«Grenada» «Hawaii»	«Antigua» «Tonga»	«Trinidad» «Curacao» «Pitcairn»	«Tobago» «Bonaire»
технология (нм)	28				
транзисторов (млрд)	8,9	6,2	5,0	2,8	2,1
универсальных процессоров	4096	2816	2048	1280	896
текстурных блоков	256	176	128	80	56
блоков блендинга	64		32		16
шина памяти	4096	512	384	256	128
типы памяти	HBM	DDR3 GDDR5			
системная шина	PCI Express 3.0				
интерфейсы	DVI Dual-Link HDMI DisplayPort				
вершинные шейдеры	5.0				
пиксельные шейдеры	5.0				
точность вычислений	FP32/FP64				
Сглаживание	MSAA CFAA SSAA EQAA MLAA				

Вычислительный блок GCN:

- четыре подраздела
- работает над своим потоком команд каждый такт
- встроенные векторные и скалярные блоки

GCN 3-го поколения

- улучшенная производительность обработки геометрии и тесселяции
- новые методы сжатия данных без потерь в кадровом буфере
- новые мультимедийные 16-битные инструкции
- увеличенный объем L2 кэша до 2 МБ
- улучшенное планирование и распределение задач
- новые инструкции для параллельной обработки данных

Графический процессор Fiji

- 1 командный процессор
- 8 движков асинхронных вычислений Asynchronous Compute Engine
- 4 шейдерных движка Shader Engine
- собственный процессор для обработки геометрических данных и растеризатор
- по четыре укрупненных блока ROP
- обработка по 16 пикселей за такт
- 16 compute unit
 - 64 ALU
- 4096 ALU
- Тактовая частота: 1050 МГц
- Теоретическая производительность (FP32):
 - 8,6 TFlops
- Теоретическая производительность (FP64):
 - 538 MFlops

58. Микроархитектура семейства видеокарт AMD Radeon 400

кодовое имя	Polaris 10	Polaris 11
технология (нм)	14	
транзисторов (млрд)	5,7	?
универсальных процессоров	2304	896
текстурных блоков	144	56
блоков блендинга	32	16
шина памяти	256	128
типы памяти	DDR3 GDDR5	
системная шина	PCI Express 3.0 x16	PCI Express 3.0 x8
интерфейсы	DVI Dual-Link HDMI 2.0b DisplayPort 1.4	
D3D Feature Level	12_0	
точность вычислений	FP32/FP64	

Графический процессор Polaris 10

- 4-е поколение архитектуры Graphics Core Next
- технологический процесс: 14 нм 3D FinFET (Fin Field Effect Transistor)
(производство на базе Samsung Electronics и GlobalFoundries)
- поддержка VR
- DirectX 12, Vulkan API
- Radeon WattMan
- HDR

Состав Polaris 10

- 1 Graphics Command Processor
- 4 асинхронных вычислительных движка Asynchronous Compute Engines (ACE)
- 2 планировщика задач Hardware Scheduler (HWS)
- 36 вычислительных блоков Compute Unit (CU)
- 4 геометрических процессора
- 144 текстурных модуля TMU (включающих по четыре блока загрузки и сохранения данных LSU на каждый TMU)
- 32 блока ROP
- 8 32-битных контроллера памяти GDDR5-памяти
- эффективная частота до 8 ГГц
- CU – Compute Unit
- состав:
 - выделенное локальное хранилище для обмена данными
 - или расширения локального регистрового стека
 - L1-кэш
 - полноценный текстурный конвейер с блоками выборки и фильтрации
 - аппаратно разбит на блоки:
 - выполняют собственный поток команд

- планирование и распределение работы

Усовершенствования GCN 4-го поколения

- улучшенная обработка геометрии
 - Primitive Discard Accelerator - отбрасывает невидимые треугольники
 - поддержка нескольких проекций при рендеринге VR с разным разрешением
 - обновленный контроллер памяти с улучшенным сжатием данных
 - Delta Color Compression
 - режимы сжатия с соотношением 2:1, 4:1 и 8:1
 - модифицированная предвыборка инструкций и улучшенная буферизация
 - планирование и приоритезация вычислительных задач в асинхронном режиме
 - поддержка операций над данными в формате FP16/Int16
- Hardware scheduler (HWS)
- используется при асинхронных вычислениях
 - решаемые задачи:
 - разгрузка CPU от задач планирования
 - приоритезация задач реального времени
 - параллельное выполнение задач и процессов
 - менеджмент ресурсов
 - координация и балансирование загрузки
 - исполнительных блоков
 - обновление при помощи микрокода

59. Микроархитектура семейства видеокарт AMD Radeon 500

кодовое имя	Vega 10	Polaris 10/20	Polaris 11/21	Polaris 12
технология (нм)	14			
транзисторов (млрд)	12,5	5,7	3,0	2,2
универсальных процессоров	4096	2304	1024	512
текстурных блоков	256	144	64	32
блоков блендинга	64	32	16	
шина памяти	2048	256	128	
типы памяти	HBM2	GDDR5		
системная шина	PCI Express 3.0 x16			
интерфейсы	DVI Dual-Link HDMI 2.0b DisplayPort 1.4			
D3D Feature Level	12_1		12_0	
точность вычислений	FP16/FP32/FP64		FP32/FP64	

Состав Vega 10

- 4 движка асинхронного исполнения команд
- 4 модифицированных геометрических движка
- 64 вычислительных блока Compute Unit нового поколения (Graphics Core Next)

- шина Infinity Fabric
- HBM2-память
 - 2048-битной шина
 - тактовая частота: ~1 ГГц
 - пропускная способность: ~484 ГБ/с.

Next-Generation Compute Unit

- 4096 потоковых процессоров
- 256 текстурных блоков
- 64 блока ROP
- Rapid Packed Math
 - 128 операций FP32
 - 256 операций FP16

HBM2-память

режим кэш-памяти для данных:

- виртуальная адресация до 512 терабайт данных
- 49-битный доступ
- управление через High Bandwidth Cache Controller (HBCC)
- размер страниц настраивается
- единый кэш-памяти второго уровня (L2)
- режимы работы:
- inclusive и exclusive
- с иерархической или плоской моделью памяти

60. Микроархитектура семейства видеокарт AMD Radeon 500 (у него так в списке вопросов дублируется, не трогать)

кодовое имя	Vega 10	Polaris 10/20	Polaris 11/21	Polaris 12
технология (нм)	14			
транзисторов (млрд)	12,5	5,7	3,0	2,2
универсальных процессоров	4096	2304	1024	512
текстурных блоков	256	144	64	32
блоков блендинга	64	32	16	
шина памяти	2048	256	128	
типы памяти	HBM2	GDDR5		
системная шина	PCI Express 3.0 x16			
интерфейсы	DVI Dual-Link HDMI 2.0b DisplayPort 1.4			
D3D Feature Level	12_1		12_0	
точность вычислений	FP16/FP32/FP64		FP32/FP64	

Состав Vega 10

- 4 движка асинхронного исполнения команд
- 4 модифицированных геометрических движка
- 64 вычислительных блока Compute Unit нового поколения (Graphics Core Next)
- шина Infinity Fabric

- HBM2-память
 - 2048-битной шина
 - тактовая частота: ~1 ГГц
 - пропускная способность: ~484 ГБ/с.

Next-Generation Compute Unit

- 4096 потоковых процессоров
- 256 текстурных блоков
- 64 блока ROP
- Rapid Packed Math
 - 128 операций FP32
 - 256 операций FP16

HBM2-память

- режим кэш-памяти для данных:
 - виртуальная адресация до 512 терабайт данных
 - 49-битный доступ
 - управление через High Bandwidth Cache Controller (HBCC)
 - размер страниц настраивается
 - единый кэш-памяти второго уровня (L2)
 - режимы работы:
 - inclusive и exclusive
 - с иерархической или плоской моделью памяти

61. Микроархитектура семейства видеокарт AMD Radeon RX Vega

- Используется память типа **HBM2 (High Bandwidth Memory 2)**, которая отличается высокой пропускной способностью и энергоэффективностью.
- Особенность — технология **HBCC (High-Bandwidth Cache Controller)**, которая эффективно управляет большим объемом памяти, включая системную оперативную память и SSD, как расширение видеопамати.
- Микроархитектура Vega использует **вычислительные блоки (Compute Units, CUs)**, которые состоят из потоковых процессоров, текстурных блоков и специализированных ALU (арифметико-логических устройств).
- Поддерживается **Rapid Packed Math** — возможность выполнения 16-битных операций FP16 в два раза быстрее, чем 32-битных FP32, что особенно полезно для задач искусственного интеллекта и некоторых игровых процессов.

Состав Vega 10 (который содержит все данные особенности)

- 4 движка асинхронного исполнения команд

- 4 модифицированных геометрических движка
- 64 вычислительных блока Compute Unit нового поколения (Graphics Core Next)
- шина Infinity Fabric
- HBM2-память
 - 2048-битной шина
 - тактовая частота: ~1 ГГц
 - пропускная способность: ~484 ГБ/с.

Next-Generation Compute Unit

- 4096 потоковых процессоров
- 256 текстурных блоков
- 64 блока ROP
- Rapid Packed Math
 - 128 операций FP32
 - 256 операций FP16

HBM2-память

режим кэш-памяти для данных:

- виртуальная адресация до 512 терабайт данных
- 49-битный доступ
- управление через High Bandwidth Cache Controller (HBCC)
- размер страниц настраивается
- единый кэш-памяти второго уровня (L2)
- режимы работы:
 - inclusive и exclusive
 - с иерархической или плоской моделью памяти

62. Микроархитектура семейства видеокарт AMD Radeon RX 5000

Кодовое имя	Vega 20	Navi 10
Технология (нм)	7	
Транзисторов (млрд)	13,2	10,3
Площадь ядра (мм ²)	331	251
Универсальных процессоров	4096	2560
Текстурных блоков	256	160
Блоков блендинга	64	64
Шина памяти	4096	256
Типы памяти	HBM2	GDDR6
Системная шина	PCI Express 4.0	PCI Express 3.0
Интерфейсы	DVI Dual-Link HDMI 2.0b Display Port 1.4	

RDNA:

- 40 вычислительных блоков нового дизайна
- 2560 блоков ALU

- 160 блоков TMU
- 64 блоков ROP
- 4 асинхронных вычислительных движка
- процессор обработки геометрии с четырьмя блоками обработки примитивов

RDNA CU:

- удвоено количество скалярных блоков и блоков управления
- скалярные инструкции могут выполняться каждый цикл, а не раз в четыре
- выделенный скалярный блок и выделенный блок выдачи инструкций
- улучшена предвыборка
- два режима исполнения: Wave32 и Wave64
- ширина блока SIMD: 32 слота (SIMD32)

63. Микроархитектура семейства видеокарт AMD Radeon RX 6000

1) RDNA 2.0:

- технология Infinity Cache - служит для увеличения пропускной способности памяти и уменьшения задержек. Infinity Cache представляет собой кэш-память, расположенную непосредственно на графическом процессоре (GPU), что позволяет ускорить доступ к данным.

AMD infinity cache структура памяти:

- 1) 384-разрядная память GDDR6х(19,5 Гбит/с)
- 2) 256-разрядная память GDDR6 (16 Гбит/с) + 128 МБ памяти Infinity Cache (высокоскоростной буфер объемом 128 МБ)
- 3) фактически 1664 Гбит/с
- 4) фактическая пропускная способность в 3,25 раза выше по сравнению с 256-разрядной памятью GDDR6 (16 Гбит/с)

- Smart Access Memory - технология, позволяющая процессору AMD Ryzen иметь полный доступ к видеопамяти, что приводит к улучшенной производительности в играх. Это достигается за счет использования функции Resizable BAR.

- Ray Accelerator - специальные аппаратные блоки, оптимизированные для трассировки лучей в реальном времени, которые повышают качество освещения и теней в играх

2) Техпроцесс 7 нм - обеспечивает высокую плотность транзисторов и энергоэффективность

3) PCIe 4.0, GDDR6

4) поддержка

- DirectX 12 Ultimate - Полная поддержка современных графических технологий, включая DirectX Raytracing, Variable Rate Shading, Mesh Shaders и Sampler Feedback, что позволяет разработчикам создавать более сложные и реалистичные игровые миры:

- DirectX Raytracing

- Variable Rate Shading

- Mesh Shaders

- Sampler Feedback

- DirectStorage - Поддержка технологии, позволяющей значительно сократить время загрузки игр за счет более быстрого доступа к данным с NVMe SSD

64. Микроархитектура семейства видеокарт AMD Radeon RX 7000

Кодовое имя	Navi 31	Navi 32	Navi 33
Технология (нм)	5		
Транзисторов (млрд)	24	16,7	10,3
Площадь ядра (мм2)	308	237	139
Универсальных процессоров	6144	3840	2048
Текстурных блоков	384	240	128
Блоков блендинга	96	80	64
Шина памяти	512	256	128
Типы памяти	GDDR6X		
Системная шина	PCI Express 5.0		
Интерфейсы	HDMI 2.1 Display Port 2.0		

Архитектура RDNA 3

- параллельная архитектура (новая архитектура с поддержкой мультипоточности, оптимизированная для игр)

- Ray Tracing - усовершенствованные Ray Accelerators, обеспечивающие высокую производительность трассировки лучей

- Infinity Cache: увеличенный кэш до 128 МБ, что обеспечивает высокую пропускную способность

- техпроцесс 5 нм - улучшенная энергоэффективность и производительность

RDNA 3 CU

- увеличение числа вычислительных блоков до 64 на CU.
- поддержка нового формата данных для повышения производительности
- поддержка режима Wave64 для более эффективного выполнения инструкций
- оптимизированная предвыборка данных для уменьшения задержек.

Поддержка технологий:

- DirectX 12 Ultimate - полная поддержка новых графических технологий, включая DirectStorage и Mesh Shader
- Variable Rate Shading - улучшение производительности за счет динамического изменения частоты рендеринга
- FidelityFX Super Resolution 2.0 - технология суперразрешения для повышения производительности в играх

65. Микроархитектура IBM Power8

POWER – микропроцессорная архитектура с ограниченным набором команд (RISC), разработанная и развиваемая компанией IBM. Название позже было расшифровано как Performance Optimization With Enhanced RISC (оптимизация производительности на базе расширенной архитектуры RISC).

POWER8 – семейство суперскалярных микропроцессоров с архитектурой POWER, представленное компанией IBM в августе 2013 года. Производятся по техпроцессу 22 нм, имеют тактовые частоты от 2,5 до 5 ГГц, содержат 6 или 12 ядер. Системы на базе POWER8 выпускаются с июня 2014 года.

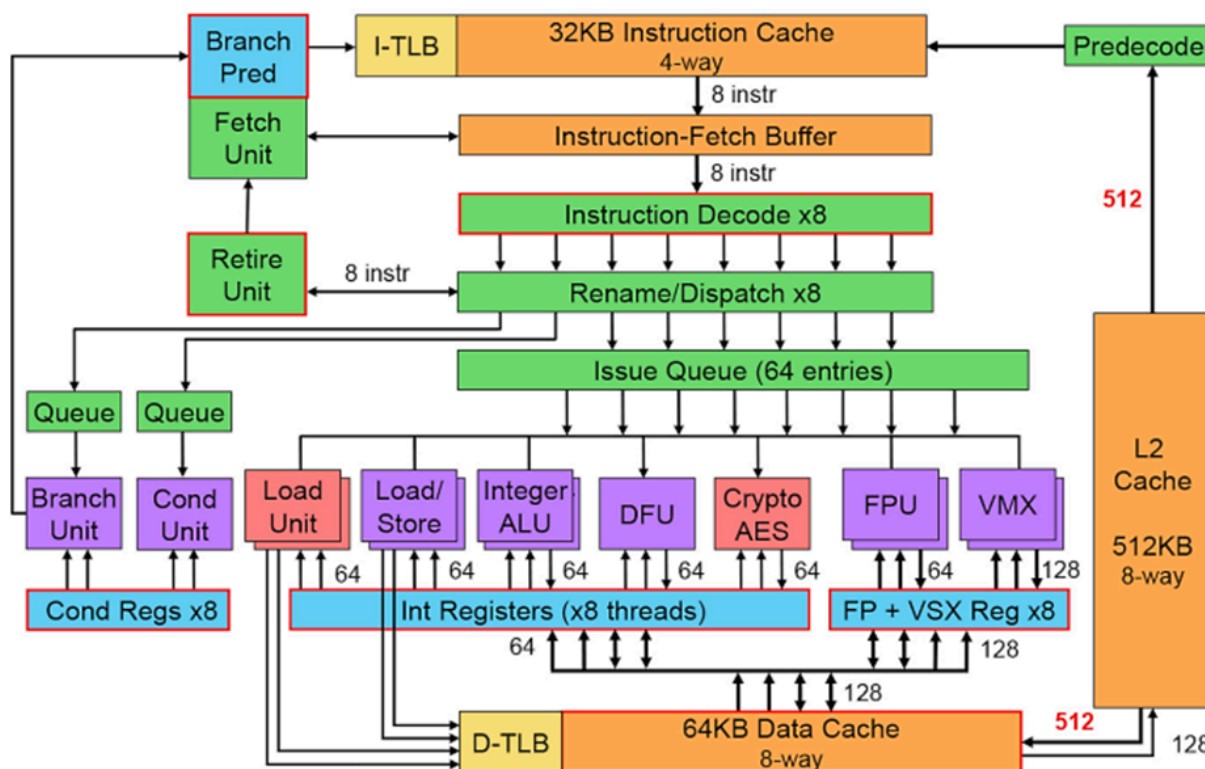
POWER8 разработан как микропроцессор с расширенной поддержкой многопоточности. Так, каждое ядро POWER8 имеет аппаратную поддержку одновременного исполнения до 8 потоков, следовательно, 12-ядерный микропроцессор поддерживает до 96 потоков (технология SMT8, аналог Intel Hyper-threading). В процессоре используется значительное количество eDRAM памяти в качестве кэшей. На каждое ядро выделены кэши L1 размером 64 и 32 КБ (данные и инструкции соответственно), кэши L2 размером 512 КБ; процессор также имеет общий кэш L3 размером 48 (6-ядерные модели) или 96 МБ

(12-ядерные модели). Кэш-память четвёртого уровня L4 не включена в кристалл CPU, но её достаточно много – до 128 МБ. В процессор встроены контроллеры памяти DDR3/DDR4 и системные каналы ввода-вывода (CAPI port на основе PCI Express 3.0, в том числе для подключения ASIC, FPGA, GPU). Питанием процессора управляет встроенный микроконтроллер на базе PowerPC 405 с 512 килобайтами SRAM памяти, настраивая 1764 встроенных регуляторов напряжения.

Для многих видов нагрузок процессор POWER8 показывает прирост производительности в 2-3 раза по сравнению с предыдущим процессором POWER7. Максимальная производительность при обработке чисел двойной точности оценивалась в 290 ГФлопс, одинарной точности – 580 ГФлопс (8 операций DP или 16 операций SP за такт на ядро при использовании FMA).

Производится по техпроцессу 22 нм по технологии кремний на изоляторе с 15 слоями металлизации. 12-ядерный вариант содержит 4,2 миллиарда транзисторов и имеет площадь кристалла в 650 мм². 6-ядерный вариант меньше – всего 362 мм².

Микроархитектура процессора IBM Power8 приведена на рисунке.



66. Микроархитектура IBM Power9

POWER – микропроцессорная архитектура с ограниченным набором команд (RISC), разработанная и развиваемая компанией IBM. Название позже было расшифровано как Performance Optimization With Enhanced

RISC (оптимизация производительности на базе расширенной архитектуры RISC).

POWER9 – семейство суперскалярных многопоточных мультипроцессоров на базе архитектуры POWER, анонсированное в августе 2016 года. Представлены во второй половине 2017 года. Изготовлены по техпроцессу 14 нм. Используются в суперкомпьютерах Summit и Sierra в США и европейском MareNostrum.

POWER9 выпускается в 4 вариантах:

- два варианта по 12 ядер с 8 потоками на ядро (SMT8) – предназначены для платформ с расширенной виртуализацией
- два варианта с 24 ядрами с поддержкой 4 потоков на ядро (SMT4) – предназначены для решения обычных задач в среде Linux.

В каждом из вариантов предлагается 2 типа процессоров:

Версия процессора SO (Scale-out) позиционируется как решение для двухпроцессорных конфигураций с встроенным контроллером памяти на 8 каналов небуферизованной DDR4.

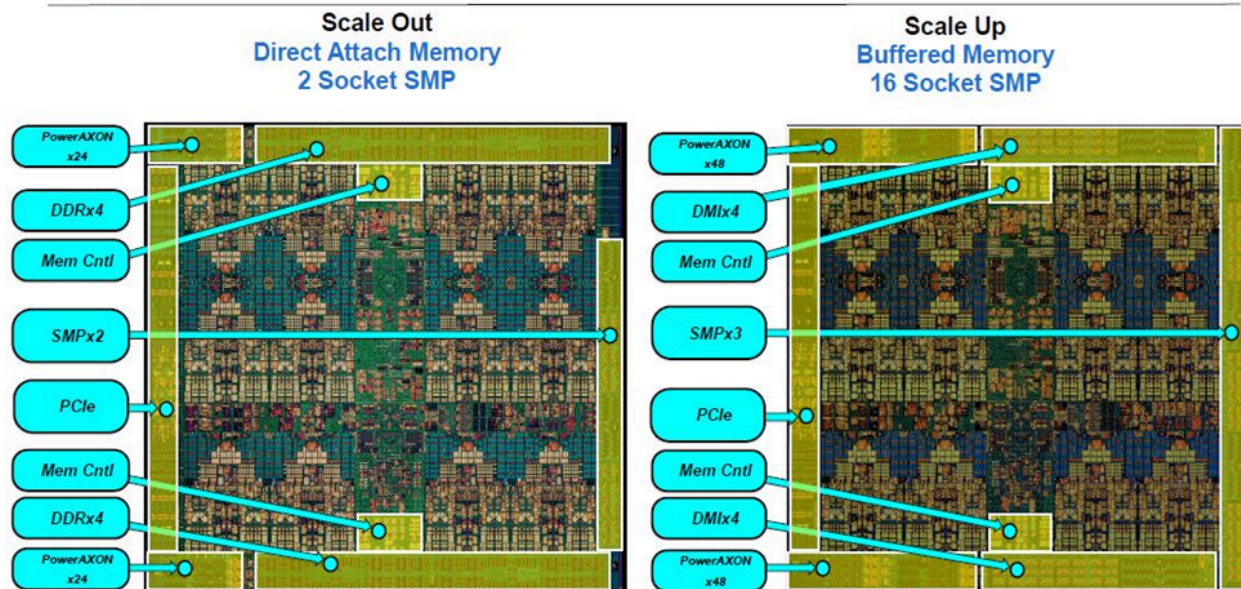
Версия процессора SU (Scale-up) предложены как решение для конфигураций с числом процессоров 4 и более и доступом к буферизованной DDR4-памяти через внечиповый контроллер памяти Centaur.

Общий объем L3 памяти составляет 120 МБ. Скорость доступа к L3 составляет вплоть до 7 ТБ/с. Данные процессоры предназначены исключительно для серверов, а не обычных ПК

Процессоры используют шину PCI Express 4.0 для подключения периферии. Также используется интерфейс NVIDIA NVLink 2.0 и интерфейс CAPI 2.0. Работа с внешними устройствами будет осуществляться посредством интерфейса IBM Bluelink (25 Гбит/с).

Архитектура поддерживается операционными системами FreeBSD, IBM AIX, IBM i, Linux, OpenBSD. Ядро Linux поддерживает архитектуру начиная с версии 4.6 от марта 2016 года. Дистрибутивы Linux: RHEL, SUSE, Debian, CentOS.

Задачи, которые можно решать при помощи Power9 – глубокое обучение, слабая форма ИИ, создание самообучающихся систем.



67. Микроархитектура IBM Power10

POWER – микропроцессорная архитектура с ограниченным набором команд (RISC), разработанная и развиваемая компанией IBM. Название позже было расшифровано как Performance Optimization With Enhanced RISC (оптимизация производительности на базе расширенной архитектуры RISC).

POWER10 – семейство суперскалярных симметричных мультипроцессоров на базе архитектуры POWER разработки IBM, следующее поколение процессоров после POWER9.

Впервые появились в 2020 году, где отмечены как нацеленные на задачи аналитики и применения для технологий больших данных. Сейчас основные сферы применения процессора на архитектуре POWER10 – искусственный интеллект, большие данные, серверы для гибридных облачных решений и суперкомпьютеры.

Использует набор инструкций Power ISA версии 3.1, открытый в 2019 году в рамках инициативы OpenPOWER.

Как и в POWER9, планируются два варианта процессоров — 15-ядерные с восемью потоками на ядро (SMT8) и 30-ядерные с четырьмя (SMT4). В каждом из SMT8-ядер по 8 SIMD-блоков и по 4 матричных блока, поддерживающих одновременные операции с различной точностью и 2 МБ кэша второго уровня. Кэш третьего уровня в процессоре – 128 МБ. Соединение с памятью – 16 линий OMI x8 суммарной пропускной способностью 1 ТБ/с; также процессор оснащается 16-ю интерфейсами PowerAxon x8, тоже суммарной пропускной способностью 1 ТБ/с, которые могут быть сконфигурированы для процессорного

межсоединения NUMA или для интерфейса OpenCAPI; кроме того, встроены два контроллера PCI Express 5.0 по 16 линий каждый. В процессоре на 18-слойной подложке площадью 602 мм² располагается 18 млрд транзисторов. В зависимости от решения по межпроцессорному соединению (2D-тор или полносвязное) возможна установка 16 процессоров или 4 процессора на систему.

Процессор IBM POWER10 поддерживает полное шифрование содержимого оперативной памяти и имеет специальные аппаратные блоки для ускорения криптографических процедур. IBM уточнила, что у POWER10 зафиксирован четырехкратный прирост по скорости шифрования AES на ядро по сравнению с POWER9. Также в POWER10 поддерживается защита и изоляция контейнеров на аппаратном уровне.

Блок вычислений с плавающей точкой в POWER10 относительно предшественника работает в 10 раз быстрее в режиме FP32, в 15 раз – в режиме BFloat16 и в 20 раз – в INT8. Последнее используется для разработки нейросетей.

Серийный выпуск стартовал в IV квартале 2021 года по техпроцессу 7 нм на фабрике Samsung. Производитель заявляет, что новинка имеет трехкратное преимущество в энергоэффективности по сравнению с POWER9.

68. Микроархитектура IBM Power11

69. Микроархитектура NVIDIA Maxwell

70. Микроархитектура NVIDIA Pascal

кодовое имя	GP102	GP104	GP106
технология, нм	16		
транзисторов, млрд	12	7,2	4,4
универсальных процессоров	3584	2560	1280
текстурных блоков	224	160	80
блоков блендинга	96	64	48
шина памяти	384	256	192
типы памяти	GDDR5, GDDR5X		
системная шина	PCI Express 3.0		
интерфейсы	DVI Dual Link HDMI 2.0b DisplayPort 1.4		
D3D Feature Level	12_1		
точность вычислений	FP32/FP64		

Архитектура NVIDIA Pascal была представлена в 2016 году

Основные характеристики GPC104 (GeForce GTX 1080):

- 4 кластера GPC, каждый из которых содержит несколько блоков SM (Streaming Multiprocessor). В GeForce GTX 1080 имеется 20 SM, что позволяет обрабатывать большое количество параллельных вычислений.

Структура SMM (Streaming Multiprocessor):

- каждый SM содержит 128 CUDA-ядер, предназначенных для выполнения параллельных вычислений. Эти ядра могут обрабатывать различные задачи, включая рендеринг и вычисления.

- размер регистрового файла составляет 256 КБ. Регистры обеспечивают быстрый доступ к данным для CUDA-ядер, что увеличивает общую производительность.

- каждый SM имеет 96 КБ общей памяти (smem), которая может использоваться для совместного доступа между CUDA-ядрами. Это позволяет уменьшить задержки при обмене данными и повышает эффективность выполнения программ.

- 48 КБ кэша L1, который помогает ускорить доступ к часто используемым данным, снижая необходимость обращения к более медленной глобальной памяти.

- каждый SM содержит 8 текстурных блоков (TMU), которые отвечают за выборку текстур из памяти и их фильтрацию. Это позволяет GPU быстро обрабатывать текстуры, что критично для графических приложений и игр.

Особенности и технологии:

1) GPU Boost 3.0 - технология, позволяющая автоматически увеличивать тактовую частоту GPU в зависимости от нагрузки и температуры. Это позволяет графическим процессорам достигать максимальной производительности без вмешательства пользователя.

2) Память GDDR5 - GTX 1080 использует память GDDR5X с высокой пропускной способностью, что обеспечивает быструю передачу данных между GPU и видеопамятью. Это критично для современных игр и приложений, требующих большой пропускной способности.

3) Улучшенное сжатие данных в памяти - Pascal использует более эффективные алгоритмы сжатия данных, что снижает объем передаваемых данных и увеличивает доступную пропускную способность памяти. Это приводит к улучшению производительности в сценариях, когда данные в памяти становятся узким местом.

71. Микроархитектура NVIDIA Turing

Кодовое имя	TU102	TU104	TU106	TU116	TU117
Технология, нм	12				
Транзисторов, млрд	18,6	13,6	10,8	6,6	4,7
Площадь кристалла, мм²	754	545	445	284	200
Универсальных процессоров	4608	3072	2304	1536	1024
Текстурных блоков	288	192	144	96	64
Блоков блендинга	96	64	64	48	32
Шина памяти	384	256	256	192	128
Типы памяти	GDDR6				GDDR5
Системная шина	PCI Express 3.0				
Интерфейсы	DVI Dual Link HDMI 2.0b DisplayPort 1.4				

Основные характеристики TU102

- 6 графических процессорных кластеров (GPC), каждый из которых содержит несколько SM (Streaming Multiprocessor).
- 36 текстурных кластеров, которые отвечают за выборку и фильтрацию текстур, улучшая качество рендеринга.
- 72 SM (streaming multiprocessors), каждый из которых состоит из следующих компонентов:
- 64 CUDA-ядра на SM, предназначенные для выполнения параллельных вычислений.

- 8 тензорных ядер на SM, оптимизированных для выполнения операций с низкой точностью, таких как матричные вычисления, которые важны для задач машинного обучения и трассировки лучей.

Структура SM

- 256 КБ памяти для хранения промежуточных данных, используемых CUDA-ядрами - регистровый файл

- Shared Memory (smem) и L1 Cache:

 - 96 КБ общей памяти, которая может использоваться для обмена данными между CUDA-ядрами.

 - увеличенная производительность благодаря:

 - удвоенной пропускной способности L1.

 - низкой задержке доступа к L1.

 - увеличенной емкости L1 до 2.7 раз по сравнению с предыдущими архитектурами.

 - увеличенной емкости L2, что позволяет хранить больше данных для быстрого доступа.

- Инструкции L0 Cache - новый кэш уровня 0 для хранения инструкций, что позволяет ускорить выполнение команд и снизить задержки.

- FP64 - каждое SM поддерживает 2 FP64 (двойной точности) операции за такт, что делает Turing более подходящим для вычислительных задач, требующих высокой точности.

Зависимость GPU от CPU

$GPU = f(CPU)$: производительность GPU во многом зависит от возможностей CPU. Каждый объект в сцене требует множества вызовов функций отрисовки (draw calls), каждый из которых обрабатывается на CPU. Это может стать узким местом, ограничивая способность GPU показывать всю свою мощность, особенно в сложных сценах с множеством объектов.

- Variable Rate Shading (VRS)

Шейдинг с переменным количеством сэмплов: технология VRS позволяет изменять количество выборок для каждого тайла буфера размером 4×4 пикселя. Это позволяет более эффективно использовать ресурсы GPU, сосредотачивая вычислительные мощности на более важных частях сцены, где требуется высокая детализация, и снижая ресурсоемкость на менее критичных участках.

72. Микроархитектура NVIDIA Ampere

A100, первый графический процессор с архитектурой NVIDIA Ampere, был выпущен в мае 2020 года. Он обеспечивает колоссальное ускорение для обучения ИИ, высокопроизводительных вычислений и анализа данных. В основе A100 лежит чип GA100 — чисто вычислительный и, в отличие от GA102, еще не игровой.

Графические процессоры GA10x основаны на архитектуре графических процессоров NVIDIA Turing. Turing — первая архитектура в мире, предлагающая высокопроизводительную трассировку лучей в реальном времени, графику с ИИ-ускорением и профессиональный рендеринг графики — все в одном устройстве.

Как и во всех GeForce RTX, в основе GA102 лежит процессор, содержащий три различных типа вычислительных ресурсов:

- **CUDA-ядра** для программируемого шейдинга;
- **RT-ядра**, ускоряющие расчет пересечений геометрии сцены с ограничивающими объемами (BVH) во время трассировки лучей;
- **Тензорные ядра**, значительно ускоряющие вывод и обучение нейронной сети.

Высокоуровневая архитектура GPC, TPC и SM

Как и предшественники, GA102 состоит из графических кластеров Graphics Processing Cluster (GPC), кластеров обработки текстур Texture Processing Cluster (TPC), потоковых мультипроцессоров (SM), блоков растеризации Raster Operator (ROP) и контроллеров памяти. Полный чип имеет семь блоков GPC, 42 TPC и 84 SM.

GPC — это доминирующий высокоуровневый блок, включающий все ключевые графические составляющие. Каждый GPC имеет выделенный движок Raster Engine, а теперь еще и по два раздела ROP из восьми блоков каждый, что является новшеством архитектуры Ampere. Кроме того, GPC содержит шесть TPC, в каждом из которых расположено по два мультипроцессора и по одному PolyMorph Engine.

В свою очередь, каждый SM в GA10x содержит 128 CUDA-ядра, четыре тензорных ядра третьего поколения, регистровый файл 256 КБ, четыре текстурных блока, одно ядро трассировки лучей второго поколения и 128 КБ L1/общей памяти, которые могут быть настроены для различных мощностей в зависимости от потребностей вычислительных или графических задач.

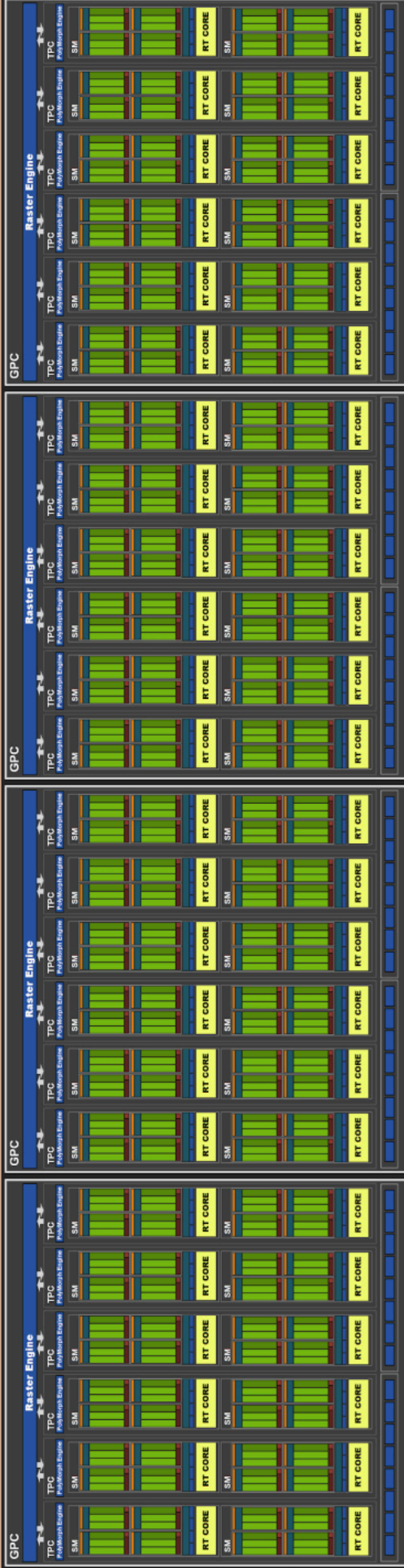
В предыдущих графических процессорах NVIDIA ROP были привязаны к контроллеру памяти и кэшу L2. Начиная с GA10x, они являются частью GPC, что повышает производительность растровых операций за счет увеличения общего числа ROP.

Графические процессоры GA102 поддерживают интерфейс NVIDIA NVLink третьего поколения, включающий в себя четыре канала x4. Так, с помощью NVLink можно соединить два графических процессора.

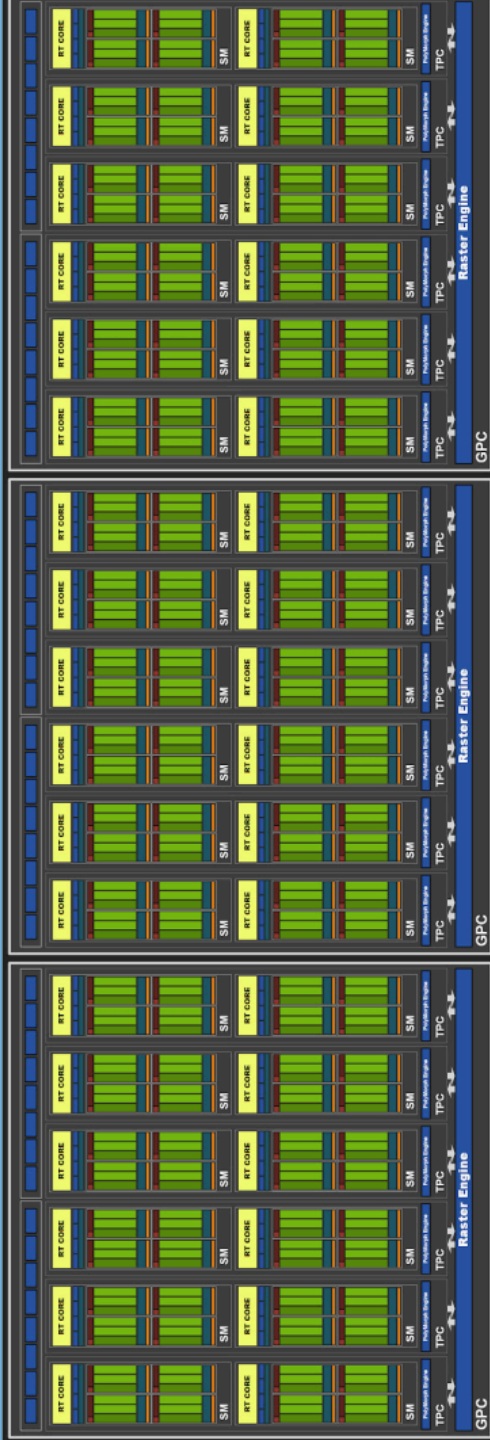
Графические процессоры GA10x оснащены интерфейсом PCI Express 4.0 (ниже на картинках - *Полный GPU GA102 с 84 блоками SM, Поточковый мультипроцессор GA10x*)

PCI Express 4.0 Host Interface

Giga Thread Engine



L2 Cache



High-Speed Hub

NVLink – Four x4 Links

SM



73. Микроархитектура NVIDIA Hopper

Техпроцесс и производительность:

- **4-нм техпроцесс от TSMC:** Hopper H100 изготовлен с использованием 4-нм технологического процесса TSMC, содержащего 80 миллиардов транзисторов, что обеспечивает высокую плотность и энергоэффективность.
- **Производительность:** H100 достигает до 60 ТФлопс в вычислениях двойной точности (FP64) и до 1 Петафлопса в вычислениях с пониженной точностью (FP8), что делает его мощным инструментом для разнообразных вычислительных задач.

Tensor Cores 4-го поколения:

- **Поддержка FloatingPoint8:** Новые тензорные ядра поддерживают формат FP8, что значительно ускоряет обучение и инференс моделей искусственного интеллекта, особенно трансформеров.
- **Улучшенные алгоритмы разреженности:** Оптимизация работы с разреженными матрицами позволяет увеличить производительность до 2 раз при выполнении соответствующих операций.

Модульная структура:

- **Multi-Instance GPU (MIG):** Расширенные возможности MIG позволяют создавать до семи изолированных экземпляров на одном GPU, обеспечивая безопасные многопользовательские конфигурации в облачных средах.
- **NVLink 4.0:** Обеспечивает скорость передачи данных до 900 ГБ/с между GPU, что улучшает масштабируемость и эффективность многопроцессорных систем.

HBM3 память:

- **Пропускная способность:** H100 является первым GPU с поддержкой HBM3, обеспечивая пропускную способность памяти до 3 ТБ/с, что на 50% больше по сравнению с 2 ТБ/с у предшественника, NVIDIA Ampere A100
- **Ускорение работы с большими наборами данных:** Высокая пропускная способность памяти позволяет эффективно обрабатывать большие объемы данных, что особенно важно для задач искусственного интеллекта и высокопроизводительных вычислений.

Transformer Engine:

- **Оптимизация трансформеров:** Специально разработанный для ускорения обучения и инференса моделей на основе трансформеров, широко используемых в обработке естественного языка и других областях.
- **Автоматическое переключение между FP8 и FP16:** Обеспечивает баланс между точностью и производительностью, автоматически выбирая оптимальный формат чисел с плавающей запятой для различных этапов вычислений.

Dynamic Programming и конфиденциальность:

- **Улучшенная работа с графовыми алгоритмами и геномикой:** Новые инструкции **DPX** и оптимизации ускоряют выполнение задач динамического программирования, таких как алгоритмы Смита–Ватермана, до 40 раз.
- **Confidential Computing:** Аппаратные функции для защиты данных и вычислений

NVIDIA Hopper (2022)



- GH100
- 144 SM

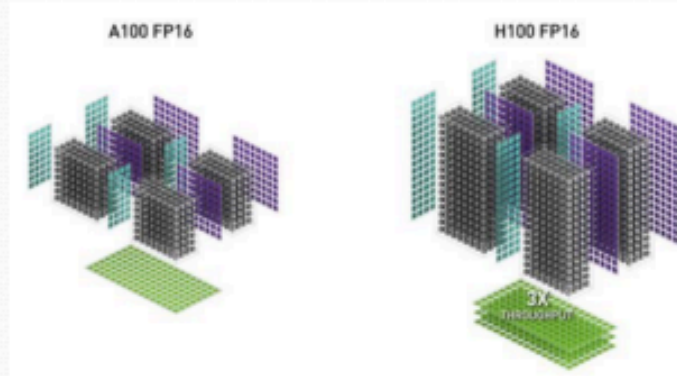
H100 SM

- DPX instructions



H100 FP16 Tensor Core

- Mixed FP8 и FP16
- в случае FP8 пропускная способность в два раза превышает FP16
- TensorFloat32 (TF32) остается в TensorFlow и PyTorch по умолчанию
 - Transformer Engine FP8 или FP16



Grace CPU

ANNOUNCING NVIDIA GRACE SUPERCHIPS
3rd Pillar of NVIDIA's 3 Chip Data Center Strategy

GRACE HOPPER SUPERCHIP
For Giant-Scale AI and HPC



- CPU+GPU Designed For Giant-Scale AI and HPC
- 600GB Memory GPU for Giant Models
- New 900 GB/s Coherent Interface
- 30X Higher System Memory B/W to GPU in a Server
- Runs NVIDIA Computing Stacks
- Available 1H 2023

GRACE CPU SUPERCHIP
For HPC and AI Infrastructure



- High Performance CPU for HPC and AI
- 144 Cores | 740 SPECrate@2017_int_base est.
- First LPDDR5X Memory With ECC, 1TB/s Memory Bandwidth
- 2X Perf/Watt Over Traditional Servers
- Runs NVIDIA Computing Stacks
- Available 1H 2023

74. Микроархитектура NVIDIA Ada Lovelace

75. Микроархитектура NVIDIA Volta